

پایتون برای علوم داده

Python for Data Science

دکتر حسین حق‌بین

گروه آمار، دانشکده‌ی مهندسی سیستم‌های هوشمند و علوم داده

جلسه ۱: معرفی و نصب محیط

جلسه ۲: انواع داده

جلسه ۳: عملگرها

جلسه ۴: ورودی و خروجی

جلسه ۵: شرط‌ها

جلسه ۶: حلقه‌ها

جلسه ۷: توابع

جلسه ۸: لیست‌ها

جلسه ۹: تاپل‌ها و مجموعه‌ها

جلسه ۱۰: دیکشنری‌ها

جلسه ۱۱: رشته‌ها

جلسه ۱۲: پروژه‌ی ۱ — تحلیل کارنامه‌ی دانشجویان یک دانشکده

2026-11-08

جلسه ۱: معرفی و نصب محیط

Introduction & Setup

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🧭 این جزوه را چطور بخوانید؟

این جزوه برای **مطالعه‌ی شخصی** نوشته شده است؛ یعنی باید بتوانید آن را بدون حضور در کلاس، از اول تا آخر بخوانید و اجرا کنید. چون این اولین جلسه است، روش خواندن همه‌ی جزوه‌های دوره را همین‌جا توضیح می‌دهیم.

چرا این روش؟

امروز ابزارهای هوش مصنوعی می‌توانند در چند ثانیه کد بنویسند. پس مهارت اصلی شما **حفظ کردن دستورها و تایپ خط به خط کد** نیست؛ مهارت اصلی این است که:

- بتوانید یک قطعه کد را **بخوانید** و بفهمید چه کار می‌کند،
- قبل از اجرا، **خروجی آن را پیش‌بینی کنید**،
- وقتی خطا می‌دهد، **پیام خطا را بفهمید** و علت را پیدا کنید،
- و درباره‌ی کدی که (مثلاً یک هوش مصنوعی) نوشته، **قضاوت کنید** که درست است یا نه.

امتحان این درس هم دقیقاً همین مهارت‌ها را می‌سنجد.

علامت‌های جزوه

علامت	معنی	شما چه کار کنید؟
💡	مفهوم	توضیح را با دقت بخوانید.
🔍	کد را بخوان	کد را خط به خط با توضیح تطبیق دهید.
❓	پیش‌بینی کن	**قبل از اجرای سلول**، جواب خود را روی کاغذ بنویسید؛ بعد اجرا کنید و مقایسه کنید.
⚠️	اشتباه رایج	این‌ها همان جاهایی است که بیشتر دانشجوها اشتباه می‌کنند.
📌	آنچه باید بلد باشید	فهرست مهارت‌هایی که در امتحان از شما خواسته می‌شود.
📝	سؤالات مرور / نمونه سؤال امتحان	سؤال‌های امتحان شبیه همین‌ها خواهند بود.

چند توصیه

1. همه‌ی سلول‌های کد را **خودتان اجرا کنید**؛ فقط خواندن کافی نیست.
2. بعد از هر مثال، کد را کمی **تغییر دهید** و ببینید خروجی چطور عوض می‌شود.
3. از هوش مصنوعی برای **توضیح دادن** کدی که نمی‌فهمید کمک بگیرید، اما جواب سؤال‌های 📝 را اول خودتان حل کنید.

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- توضیح دهید پایتون چیست و چرا زبان اصلی علوم داده است،
- پایتون را روی ویندوز نصب کنید و از درستی نصب مطمئن شوید،
- با دو محیط **Jupyter Notebook** و **VS Code** کار کنید و تفاوت آن‌ها را بدانید،
- جزوه‌های دوره را از گیت‌هاب دریافت کنید و روی کامپیوتر خودتان اجرا کنید،
- مفهوم **سلول** و **ترتیب اجرای سلول‌ها** در نوت‌بوک را بفهمید،
- اولین برنامه‌ی خود را بنویسید و **پیام خطا** را بخوانید.

فهرست مطالب

1. پایتون چیست و چرا برای علوم داده؟
2. نصب Python روی ویندوز
3. Jupyter Notebook
4. VS Code
5. دریافت و اجرای جزوه‌های دوره
6. اولین برنامه و خواندن پیام خطا

۱. پایتون چیست و چرا برای علوم داده؟

💡 مفهوم

پایتون (Python) یک زبان برنامه‌نویسی سطح بالا است که خواندن کد آن تقریباً مثل خواندن یک متن انگلیسی ساده است. پایتون یک زبان **مفسری (interpreted)** است: برنامه‌ای به نام **مفسر پایتون** کد شما را خط به خط می‌خواند و همان لحظه اجرا می‌کند.

```
your code (.py / .ipynb)  --->  Python interpreter  --->  result
```

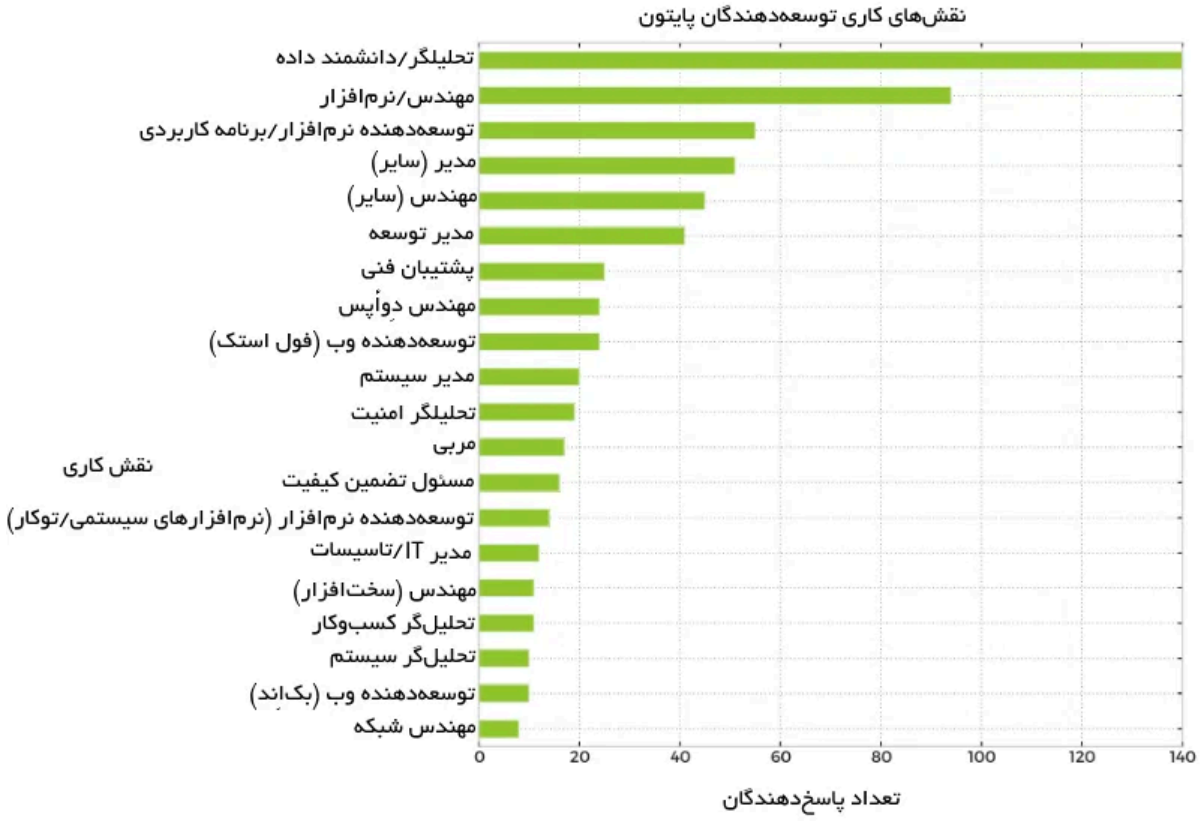
چرا پایتون در علوم داده؟

- **خوانایی:** کد کوتاه و قابل فهم است؛ پس تمرکز شما روی مسئله است، نه روی جزئیات زبان.
- **کتابخانه‌های قدرتمند:** تقریباً هر کاری در علوم داده یک کتابخانه‌ی آماده در پایتون دارد:

کار	کتابخانه	کجای این دوره؟
محاسبات عددی و ماتریسی	NumPy	بخش ۳
کار با جدول داده	Pandas	بخش ۳
رسم نمودار	Matplotlib, Seaborn	بخش ۳
یادگیری ماشین	scikit-learn	درس‌های بعدی
یادگیری عمیق	PyTorch, TensorFlow	درس‌های بعدی

- **جامعه‌ی بزرگ:** برای تقریباً هر خطایی که ببینید، کسی قبلاً همان سؤال را پرسیده و جواب گرفته است.
- **رایگان و چندسکویی:** مفسر پایتون و تقریباً همه‌ی کتابخانه‌های آن رایگان‌اند و روی ویندوز، مک و لینوکس اجرا می‌شوند.

نمودار زیر نتیجه‌ی یک نظرسنجی از توسعه‌دهندگان پایتون است. ببینید کدام نقش شغلی بیشترین سهم را دارد:



نقش‌های شغلی توسعه‌دهندگان پایتون؛ «تحلیلگر/دانشمند داده» با فاصله در صدر است.

این دوره شما را تا جایی می‌برد که بتوانید وارد یادگیری ماشین و یادگیری عمیق شوید.

🔍 کد را بخوان

هنوز هیچ دستوری از پایتون یاد نگرفته‌اید؛ با این حال سعی کنید کد زیر را **بخوانید** و حدس بزنید چه کار می‌کند. این همان مهارتی است که در کل دوره تمرین می‌کنیم.

```
scores = [17, 12.5, 19, 15]
average = sum(scores) / len(scores)
print("Average:", average)
```

```
Average: 15.875
```

- خط اول: چهار نمره را در یک **لیست** به نام `scores` نگه می‌دارد.
- خط دوم: مجموع نمره‌ها (`sum`) را بر تعداد آن‌ها (`len`) تقسیم می‌کند؛ یعنی **میانگین** را حساب می‌کند و در `average` می‌گذارد.
- خط سوم: متن `Average:` و مقدار میانگین را چاپ می‌کند.

❓ پیش‌بینی کن

اگر عدد `19` را در خط اول به `11` تغییر دهیم، خروجی چه می‌شود؟ اول حساب کنید، بعد کد بالا را تغییر دهید و اجرا کنید.

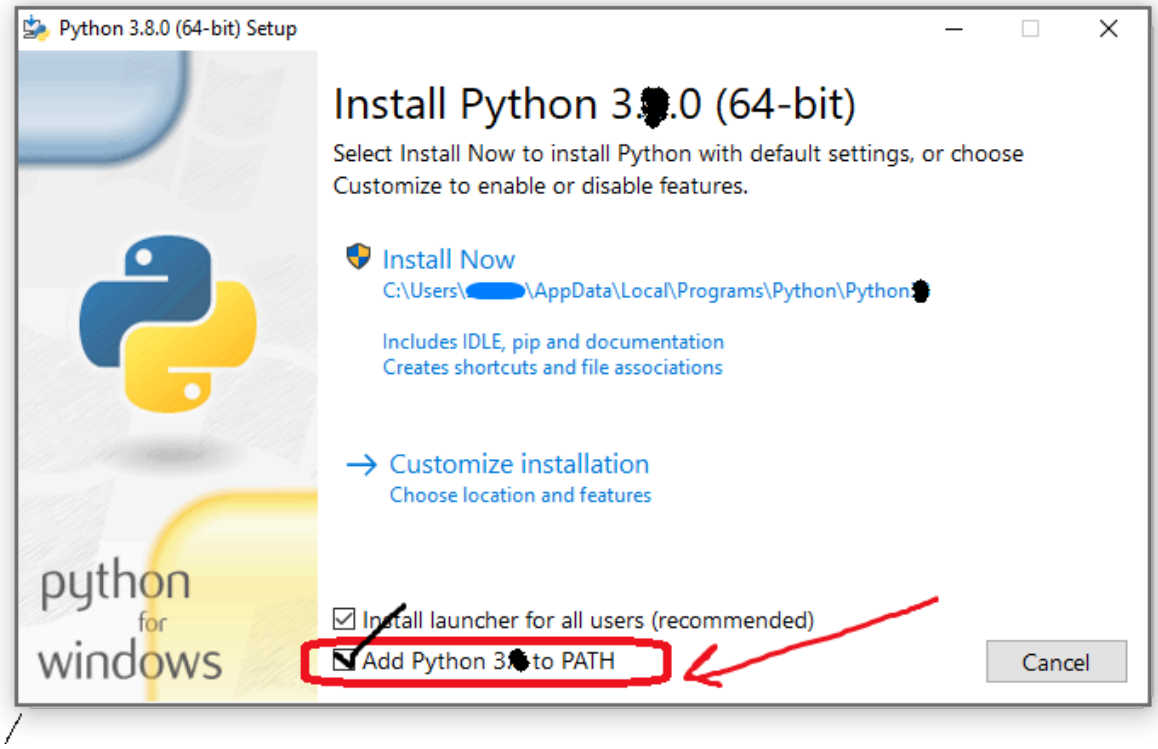
۲. نصب Python روی ویندوز

در این دوره همه‌ی کدها را روی کامپیوتر خودتان اجرا می‌کنید؛ پس اولین قدم نصب پایتون است.

مراحل نصب

1. به سایت رسمی پایتون بروید: python.org/downloads
2. روی دکمه‌ی **Download Python 3.x.x** کلیک کنید (آخرین نسخه‌ی پایدار؛ عدد دقیق نسخه مهم نیست، فقط باید با **3** شروع شود).
3. فایل دانلودشده را اجرا کنید.
4. در **اولین صفحه‌ی نصب**، قبل از هر کاری تیک گزینه‌ی زیر را بزنید:

   Add python.exe to PATH ☑
5. روی **Install Now** کلیک کنید و صبر کنید تا نصب تمام شود.



اولین صفحه‌ی نصب پایتون در ویندوز. قبل از زدن Install Now، تیک PATH (کادر قرمز) را بزنید. در نسخه‌های قدیمی‌تر، متن این گزینه Add Python 3.x to PATH است.

⚠️ اشتباه رایج: فراموش کردن تیک PATH

اگر تیک **Add python.exe to PATH** را نزنید، ویندوز نمی‌داند پایتون کجا نصب شده و در مرحله‌ی بعد پیام زیر را می‌بینید:

```
'python' is not recognized as an internal or external command,
operable program or batch file.
```

راه حل: فایل نصب را دوباره اجرا کنید، گزینه‌ی **Modify** را بزنید و در صفحه‌ی **Advanced Options** تیک **Add Python to environment variables** را بزنید.

بررسی درستی نصب

1. کلیدهای `Win` + `R` را بزنید، عبارت `cmd` را بنویسید و Enter بزنید تا **Command Prompt** باز شود.
2. دستور زیر را بنویسید و Enter بزنید:

```
python --version
```

اگر نصب درست باشد، شماره‌ی نسخه را می‌بینید؛ مثلاً:

```
Python 3.13.1
```

⚠️ اشتباه رایج: باز شدن Microsoft Store

گاهی با نوشتن `python` ، به جای اجرای پایتون، **Microsoft Store** باز می‌شود. علت این است که ویندوز یک «میان‌بر» به فروشگاه دارد. راه حل:

- در منوی Start عبارت **Manage app execution aliases** را جست‌وجو کنید،
- گزینه‌های **App Installer – python.exe** و **python3.exe** را خاموش کنید.

محیط تعاملی پایتون

در همان Command Prompt فقط بنویسید `python` و Enter بزنید. علامت `>>>` ظاهر می‌شود؛ یعنی مفسر پایتون منتظر دستور شماست:

```
>>> 2 + 3
5
>>> print("Hello")
Hello
>>> exit()
```

با `exit()` از محیط پایتون خارج می‌شوید. این محیط برای امتحان سریع یک دستور خوب است، ولی برای نوشتن برنامه از ابزارهای بخش‌های بعد استفاده می‌کنیم.

۳. Jupyter Notebook

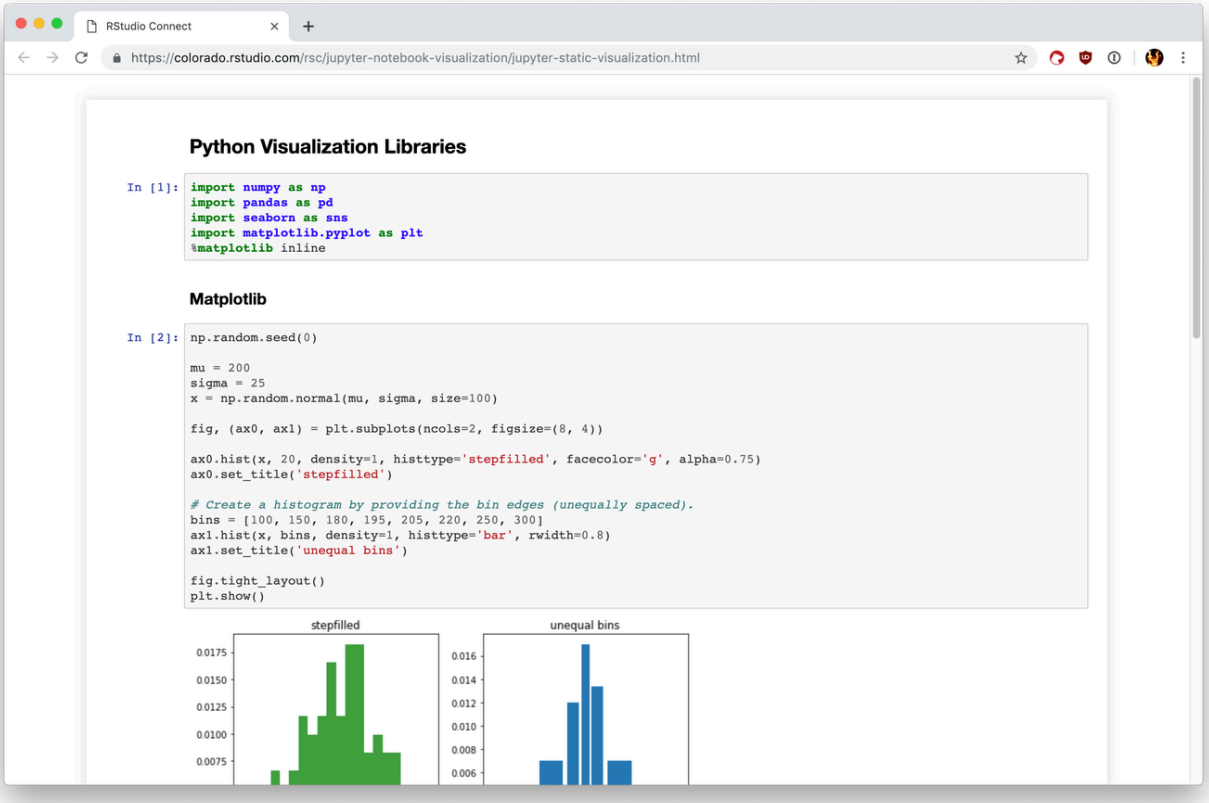
💡 مفهوم

نوت‌بوک (Notebook) فایلی با پسوند `.ipynb` است که از تعدادی **سلول (cell)** ساخته شده است. همین جزوه‌ای که می‌خوانید یک نوت‌بوک است. دو نوع سلول اصلی داریم:

نوع سلول	محتوا	با اجرا چه می‌شود؟
Code	کد پایتون	کد اجرا و خروجی زیر سلول نمایش داده می‌شود.
Markdown	متن، جدول، فرمول	متن قالب‌بندی‌شده نمایش داده می‌شود.

پشت هر نوت‌بوک یک **کرنل (kernel)** وجود دارد: یک مفسر پایتون که روشن می‌ماند و **همه‌ی متغیرها را در حافظه نگه می‌دارد**. به همین دلیل متغیری که در یک سلول ساخته‌اید، در سلول‌های دیگر هم در دسترس است.

نوت‌بوک برای علوم داده بسیار مناسب است، چون می‌توانید کد، خروجی، نمودار و توضیح را کنار هم داشته باشید:



نمونه‌ای از یک نوت‌بوک: سلول‌های کد (با شماره‌ی In [1] و In [2]) و نمودار خروجی، همه در یک صفحه.

نصب و اجرا (روی ویندوز)

بعد از نصب پایتون، در Command Prompt دستور زیر را اجرا کنید تا Jupyter نصب شود (فقط یک بار لازم است):

```
pip install notebook
```

سپس برای باز کردن Jupyter:

```
jupyter notebook
```

مرورگر باز می‌شود و فایل‌های پوشه‌ی جاری را نشان می‌دهد. از منوی **New** → **Python 3** یک نوت‌بوک جدید بسازید. تا زمانی که با Jupyter کار می‌کنید، پنجره‌ی Command Prompt را **نبندید**.

میان‌برهای مهم

کلید	کار
Shift + Enter	اجرای سلول و رفتن به سلول بعدی
Ctrl + Enter	اجرای سلول و ماندن در همان سلول
Esc	خروج از حالت ویرایش (Command mode)
A / B	افزودن سلول بالا / پایین (در Command mode)

کلید	کار
M / Y	تبدیل سلول به Markdown / Code
D D	حذف سلول

🔍 کد را بخوان: حافظه‌ی مشترک سلول‌ها

دو سلول زیر را به ترتیب اجرا کنید.

```
x = 10
```

```
x * 2
```

- سلول اول متغیر `x` را می‌سازد. خروجی ندارد، چون فقط یک **انتساب** است.
- سلول دوم از همان `x` استفاده می‌کند؛ چون کرنل `x` را در حافظه نگه داشته است.
- در نوت‌بوک، اگر **آخرین خط** یک سلول یک عبارت باشد (مثل `x * 2`)، مقدار آن خودکار نمایش داده می‌شود؛ حتی بدون `print`.

به عدد داخل کروشه‌ی کنار هر سلول (مثل `[3]`) دقت کنید: این عدد **ترتیب اجرا** را نشان می‌دهد، نه ترتیب قرار گرفتن سلول در صفحه.

❓ پیش‌بینی کن

سلول اول زیر را یک بار و سلول دوم را **دو بار پشت سر هم** اجرا کنید. خروجی بار دوم چیست؟

```
counter = 5
```

```
counter = counter + 1
counter
```

6

⚠️ اشتباه رایج: اجرای سلول‌ها به ترتیب نامنظم

چون کرنل همه‌چیز را به خاطر می‌سپارد، اگر سلول‌ها را بالا و پایین و چند بار اجرا کنید، مقدار متغیرها ممکن است با آنچه در صفحه می‌بینید فرق کند. هر وقت نتیجه‌ها عجیب شد:

منوی Kernel → Restart & Run All

این کار حافظه را پاک می‌کند و همه‌ی سلول‌ها را از اول به ترتیب اجرا می‌کند.

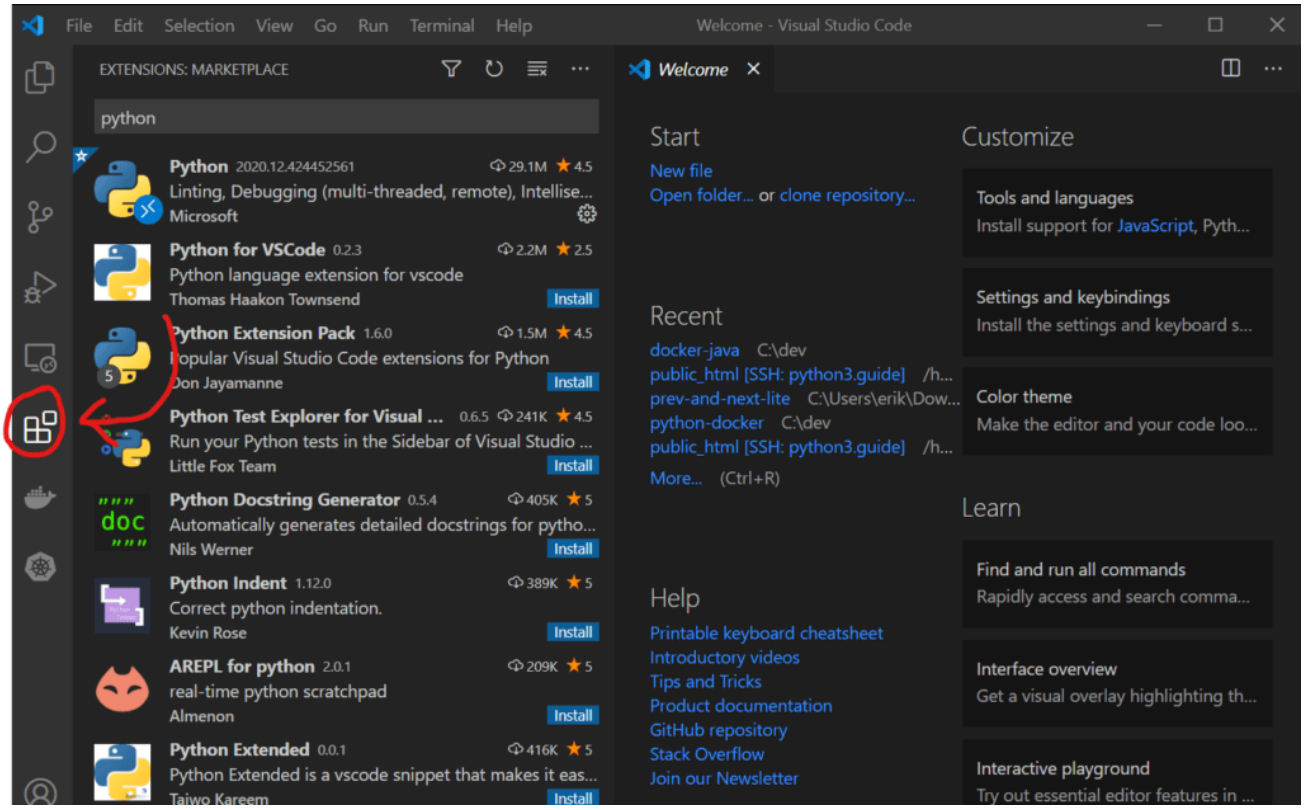
۴. VS Code

مفهوم

Visual Studio Code (VS Code) یک ویرایشگر کد رایگان از مایکروسافت است. هم فایل‌های نوت‌بوک (`.ipynb`) را باز می‌کند و هم فایل‌های اسکریپت پایتون (`.py`) را. امکاناتی مثل **تکمیل خودکار کد**، **نمایش خطاها** هنگام نوشتن و **اشکال‌زدایی (debugging)** دارد و برای پروژه‌های بزرگ‌تر، ابزار اصلی بیشتر برنامه‌نویس‌هاست.

نصب و آماده‌سازی (روی ویندوز)

1. از سایت code.visualstudio.com نسخه‌ی ویندوز را دانلود و نصب کنید.
2. VS Code را باز کنید و از نوار سمت چپ وارد بخش **Extensions** شوید (Ctrl + Shift + X).
3. عبارت `python` را جست‌وجو کنید و این دو افزونه از **Microsoft** را نصب کنید: **Python** و **Jupyter**.



بخش Extensions در VS Code (آیکون دایره‌کشیده در نوار سمت چپ). افزونه‌ای را نصب کنید که ناشر آن Microsoft است، نه افزونه‌های مشابه دیگر.

4. یک فایل `.ipynb` باز کنید. بالا سمت راست روی **Select Kernel** کلیک کنید و پایتونی را که نصب کرده‌اید انتخاب کنید.
5. اگر VS Code پیشنهاد نصب بسته‌ی **ipykernel** را داد، **Install** را بزنید.

⚠️ اشتباه رایج: نصب افزونه‌ی اشتباه

با جست‌وجوی `python` ده‌ها افزونه با اسم‌های مشابه ظاهر می‌شود. فقط افزونه‌ای را نصب کنید که زیر نامش **Microsoft** نوشته شده است.

نوت‌بوک یا اسکریپت؟

	نوت‌بوک `.ipynb`	اسکریپت `.py`
ساختار	سلول به سلول	یک فایل متنی، از بالا به پایین
اجرا	هر سلول جدا	کل فایل یک‌جا
مناسب برای	تحلیل داده، آموزش، آزمایش	برنامه‌های کامل، پروژه‌ها

برای امتحان اسکریپت: در VS Code یک فایل جدید به نام `hello.py` بسازید، خط `print("Hello from a script")` را در آن بنویسید، ذخیره کنید و دکمه‌ی ▶ (Run) بالا سمت راست را بزنید. خروجی در پنجره‌ی **Terminal** پایین صفحه نمایش داده می‌شود.

کدام محیط را انتخاب کنم؟

محیط	پیشنهاد برای
Jupyter Notebook	اجرای سریع جزوه‌ها در مرورگر
VS Code	هم جزوه‌ها و هم پروژه‌ها؛ **پیشنهاد اصلی این دوره**

ابزارهای دیگری هم وجود دارند که ممکن است اسمشان را بشنوید: **PyCharm** (محیط حرفه‌ای مخصوص پایتون)، **Spyder** (مخصوص کارهای علمی) و **Thonny** (بسیار ساده، برای مبتدی‌ها). در این دوره به آن‌ها نیاز ندارید.

۵. دریافت و اجرای جزوه‌های دوره

همه‌ی جزوه‌های این دوره در مخزن گیت‌هاب درس قرار دارند:

github.com/haghbinh/Python-Course

دریافت جزوه‌ها

1. صفحه‌ی مخزن را باز کنید.
2. روی دکمه‌ی سبز **Code** و سپس **Download ZIP** کلیک کنید.
3. فایل ZIP را در یک پوشه (مثلاً `Documents\Python-Course`) از حالت فشرده خارج کنید.
4. جزوه‌ها در پوشه‌ی `Notebooks` هستند.

باز کردن یک جزوه

- **در VS Code:** از منوی **File** → **Open Folder** پوشه‌ی `Python-Course` را باز کنید و روی فایل جزوه (مثلاً `S01_Setup_Intro.ipynb`) کلیک کنید.
- **در Jupyter:** در Command Prompt به پوشه‌ی درس بروید و Jupyter را اجرا کنید:

```
cd Documents\Python-Course
jupyter notebook
```

⚠️ اشتباه رایج: کار روی فایل اصلی

جزوه‌ها هر هفته به‌روز می‌شوند. اگر جواب تمرین‌ها را در همان فایل بنویسید و دوباره ZIP جدید را دانلود و جایگزین کنید، پاسخ‌هایتان از بین می‌رود. قبل از نوشتن، از جزوه یک **کپی با نام دیگر** بسازید (مثلاً `S01_my_answers.ipynb`).

> اگر فقط می‌خواهید جزوه را **بخوانید**، روی فایل آن در صفحه‌ی گیت‌هاب کلیک کنید؛ گیت‌هاب نوت‌بوک را همراه با متن و کدها نمایش می‌دهد، ولی امکان اجرای کد را ندارد.

۶. اولین برنامه و خواندن پیام خطا

💡 مفهوم

تابع `print` هر چیزی را که داخل پرانتز به آن بدهید روی خروجی نمایش می‌دهد. متن باید داخل **نقل‌قول** (`"..."` یا `'...'`) باشد.

```
print("Hello, World!")
```

```
Hello, World!
```

🔍 کد را بخوان

در کد زیر، هر چیزی که بعد از علامت `#` بیاید یک **توضیح (comment)** است و پایتون آن را اجرا نمی‌کند. توضیح‌ها برای انسان‌ها نوشته می‌شوند، نه برای کامپیوتر.

```
# print can take several values, separated by commas
print("Python", "for", "Data Science")

# the values are joined with a space by default;
# sep changes the separator
print("2026", "10", "30", sep="-")

print(2 + 3 * 4)   # Python computes the expression first, then prints it
```

```
Python for Data Science
2026-10-30
14
```

- خط اول کد سه متن را می‌گیرد و با یک **فاصله** بینشان چاپ می‌کند.
- با `sep="-"` به جای فاصله، علامت `-` بین مقدارها گذاشته می‌شود.
- در خط آخر، پایتون اول حاصل عبارت را حساب می‌کند (ضرب قبل از جمع) و بعد آن را چاپ می‌کند.

❓ پیش‌بینی کن

خروجی هر یک از دو خط زیر چیست؟ (به تفاوت `"2 + 3"` و `2 + 3` دقت کنید.)

```
print("2 + 3")
print(2 + 3)
```

```
2 + 3
5
```

💡 خواندن پیام خطا

خطا دیدن بخش عادی برنامه‌نویسی است. مهم این است که **پیام خطا را بخوانید**. پیام خطای پایتون را **از پایین به بالا** بخوانید:

- **خط آخر** مهم‌ترین خط است: **نوع خطا** و **توضیح** آن.
- خط‌های بالاتر نشان می‌دهند خطا **کجای کد** رخ داده است.

دو سلول زیر عمداً خطا دارند. اجرایشان کنید و پیام خطا را بخوانید.

```
pritn("Hello")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[9], line 1
----> 1 pritn("Hello")

NameError: name 'pritn' is not defined
```

🔍 خط آخر خطا `NameError: name 'pritn' is not defined` است؛ یعنی پایتون چیزی به نام `pritn` نمی‌شناسد. علت: **غلط املایی** در `print` .

```
print("Hello"))
```

```
  Cell In[10], line 1
    print("Hello"))
                   ^
SyntaxError: unmatched ')'
```

🔍 این بار خط آخر خطا `SyntaxError: unmatched ')'` است؛ یعنی **دستور زبان** کد غلط است: یک پرانتز بسته‌ی اضافه وجود دارد که هیچ پرانتز بازی برایش نیست. در خطای `SyntaxError` پایتون حتی شروع به اجرای کد نمی‌کند؛ علامت `^` در پیام خطا جای مشکل را نشان می‌دهد.

⚠️ اشتباه رایج

- فراموش کردن نقل‌قول دور متن: `print(Hello)` ← پایتون `Hello` را اسم یک متغیر فرض می‌کند و `NameError` می‌دهد.
- بزرگ و کوچک بودن حروف مهم است: `Print` با `print` فرق دارد.
- علامت نقل‌قول باید انگلیسی باشد (`"`) و نه علامت‌هایی مثل `»` یا `“` که از کیبورد فارسی یا Word کپی می‌شوند.

📌 آنچه باید بلد باشید

- پایتون یک زبان **مفسری** است و مفسر، کد را خط به خط اجرا می‌کند.
- پایتون را روی ویندوز نصب کنید و با `python --version` آن را بررسی کنید؛ نقش گزینه‌ی **Add to PATH** را بدانید.
- تفاوت سلول **Code** و **Markdown** را بدانید.
- بدانید **کرنل** متغیرها را بین سلول‌ها نگه می‌دارد و **ترتیب اجرا** (عدد داخل `[ ]`) تعیین‌کننده است، نه ترتیب سلول‌ها در صفحه.
- در نوت‌بوک، مقدار **آخرین عبارت** هر سلول خودکار نمایش داده می‌شود.
- خروجی `print` را با چند مقدار و با `sep` پیش‌بینی کنید.
- پیام خطا را از پایین بخوانید و `NameError` و `SyntaxError` را تشخیص دهید.
- تفاوت Jupyter و VS Code و تفاوت نوت‌بوک `.ipynb` و اسکریپت `.py` را بدانید.

📝 سؤالات مرور / نمونه سؤال امتحان

> این جلسه مقدماتی است و سؤال‌هایش ساده‌اند؛ اما **نوع** سؤال‌ها (پیش‌بینی خروجی، پیدا کردن خطا، توضیح کد) همان نوعی است که در تمام دوره و در امتحان می‌بینید. اول خودتان جواب دهید، بعد پاسخ را باز کنید.

سؤال ۱ — پیش‌بینی خروجی

خروجی کد زیر را دقیقاً بنویسید.

```
print("Data", "Science")
print("Data", "Science", sep="")
print(10, 20, 30, sep=", ")
```

▼ پاسخ

```
Data Science
DataScience
10, 20, 30
```

پیش‌فرض جداکننده یک فاصله است. با `sep=""` هیچ چیزی بین مقدارها گذاشته نمی‌شود و با `sep=", "` یک ویرگول و یک فاصله.

سؤال ۲ — ترتیب اجرای سلول‌ها

یک نوت‌بوک سه سلول دارد:

سلول	کد
A	`a = 1`
B	`a = a + 10`
C	`print(a)`

دانشجویی سلول‌ها را به این ترتیب اجرا کرده است: **A، B، B، C**. چه عددی چاپ می‌شود؟ اگر بعد از آن فقط **Restart & Run All** را بزند، چه عددی چاپ می‌شود؟

▼ پاسخ

بار اول **21** چاپ می‌شود: بعد از A مقدار `a` برابر 1 است، هر بار اجرای B ده واحد اضافه می‌کند (اول 1، بعد 11، بعد 21).

بعد از Restart & Run All حافظه پاک می‌شود و سلول‌ها یک بار به ترتیب A، B، C اجرا می‌شوند؛ پس **11** چاپ می‌شود.

سؤال ۳ — پیدا کردن خطا

دانشجویی کد `Print("Hello")` را اجرا کرده و این پیام را دیده است:

```
NameError: name 'Print' is not defined
```

علت خطا چیست و کد درست چیست؟

▼ پاسخ

پایتون به بزرگی و کوچکی حروف حساس است. تابع چاپ `print` (با p کوچک) است و چیزی به نام `Print` وجود ندارد. کد درست: `print("Hello")` .

سؤال ۴ — توضیح کد

یک سلول نوت‌بوک شامل این دو خط است:

```
5 + 5
3 + 3
```

با اجرای سلول فقط عدد `6` نمایش داده می‌شود. چرا `10` نمایش داده نمی‌شود؟ چطور می‌توان هر دو را نمایش داد؟

▼ پاسخ

نوت‌بوک فقط مقدار **آخرین عبارت** سلول را خودکار نمایش می‌دهد. عبارت `5 + 5` حساب می‌شود، اما نمایش داده نمی‌شود. برای نمایش هر دو باید از `print` استفاده کرد:

```
print(5 + 5)
print(3 + 3)
```

سؤال ۵ — مفهومی

دانشجویی پایتون را روی ویندوز نصب کرده، اما با اجرای `python --version` در Command Prompt پیام `'python' is not recognized ...` را می‌بیند. محتمل‌ترین علت چیست و راه حل چیست؟

▼ پاسخ

هنگام نصب، تیک **Add python.exe to PATH** زده نشده است؛ بنابراین ویندوز مسیر پایتون را نمی‌شناسد. راه حل: اجرای دوباره‌ی فایل نصب، انتخاب **Modify** و فعال کردن **Add Python to environment variables** (یا نصب دوباره با زدن تیک PATH).

خلاصه‌ی جلسه

- پایتون زبان اصلی علوم داده است؛ به خاطر خوانایی و کتابخانه‌هایی مثل Pandas، NumPy و Matplotlib.
- پایتون را روی ویندوز از python.org نصب کردیم و با `python --version` بررسی کردیم.
- نوت‌بوک از سلول ساخته شده و کرنل حافظه‌ی مشترک سلول‌هاست؛ ترتیب اجرا مهم است.

- جزوه‌ها را از گیت‌هاب دریافت و در VS Code یا Jupyter اجرا می‌کنیم؛ VS Code ابزار اصلی این دوره است.
- پیام خطا را از پایین به بالا می‌خوانیم.

جلسه‌ی بعد: انواع داده در پایتون (اعداد، رشته‌ها، مقادیر بولی و تبدیل نوع).

جلسه ۲: انواع داده

Data Types

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- توضیح دهید **متغیر** در پایتون چیست و دستور انتساب `=` دقیقاً چه می‌کند،
- نام‌های مجاز و غیرمجاز برای متغیر را تشخیص دهید،
- نوع یک مقدار را با `type` پیدا کنید و نوع حاصل یک عبارت را **پیش‌بینی** کنید،
- با چهار نوع پایه‌ی `int` ، `float` ، `str` و `bool` (و مقدار ویژه‌ی `None`) کار کنید،
- مقدارها را از یک نوع به نوع دیگر **تبدیل** کنید و خطاهای رایج تبدیل نوع را بشناسید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. متغیرها و نام‌گذاری
2. نوع داده و تابع `type`
3. اعداد: `int` و `float`
4. رشته‌ها: `str`
5. مقادیر بولی: `bool` و مقدار `None`
6. تبدیل نوع

۱. متغیرها و نام‌گذاری

💡 مفهوم

متغیر (variable) یک **نام** است که به یک **مقدار** در حافظه اشاره می‌کند. با دستور **انتساب** (`=`) یک نام به یک مقدار وصل می‌شود:

`x = 5`

این خط را این‌طور بخوانید: «مقدار `5` را بساز و نام `x` را به آن وصل کن». علامت `=` در پایتون یعنی **انتساب**، نه «مساوی است با».

در پایتون لازم نیست نوع متغیر را از قبل اعلام کنید؛ نوع از روی **مقدار** مشخص می‌شود.

```
x = 5          # an integer (int)
pi = 3.14      # a decimal number (float)
name = "Ali"   # text (str)
flag = True    # a boolean value (bool)

print(x, pi, name, flag)
```

🔍 کد را بخوان

- هر خط یک **انتساب** است: سمت راست `=` یک مقدار است و سمت چپ نامی که به آن وصل می‌شود.
- چهار مقدار از چهار **نوع** مختلف ساخته شده‌اند: عدد صحیح، عدد اعشاری، متن و مقدار درست/نادرست.
- `print` هر چهار مقدار را با یک فاصله بینشان چاپ می‌کند.

ترتیب اجرای انتساب

پایتون همیشه **اول سمت راست** `=` را حساب می‌کند و **بعد** نام سمت چپ را به نتیجه وصل می‌کند. برای همین خط زیر کاملاً درست است، با اینکه از نظر ریاضی بی‌معنی است:

```
count = 10
count = count + 1
print(count)
```

🔍 خط دوم: اول `count + 1` حساب می‌شود (یعنی `11`)، بعد نام `count` به این مقدار جدید وصل می‌شود.

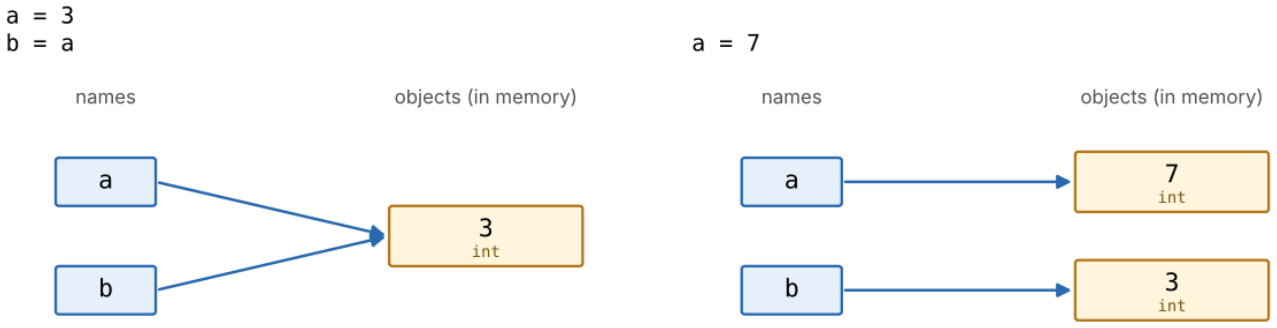
❓ پیش‌بینی کن

قبل از اجرای سلول زیر، روی کاغذ بنویسید چه چیزی چاپ می‌شود.

```
a = 3
b = a
a = 7
print(b)
```

🔍 توضیح: نام‌ها و مقدارها

خط `b = a` یعنی «`b` را به **همان مقداری** وصل کن که `a` الان به آن اشاره می‌کند». بعد از آن، `a = 7` فقط نام `a` را به مقدار جدید وصل می‌کند و **کاری به `b` ندارد**. پس `b` همچنان `3` است.



سمت چپ: بعد از `a = 3` *و* `b = a` *هر دو نام به یک مقدار اشاره می‌کنند. سمت راست: بعد از* `a = 7` *فقط پیکان* `a` *جابه‌جا می‌شود.*

این تصویر «نام‌ها و مقدارها» را به خاطر بسپارید؛ در جلسه‌ی لیست‌ها دوباره به آن برمی‌گردیم و آنجا نتیجه‌ی غافلگیرکننده‌تری می‌بینیم.

انتساب چندگانه و جابه‌جایی

در پایتون می‌توانید در یک خط به چند نام مقدار بدهید. یک کاربرد جالب، جابه‌جا کردن مقدار دو متغیر است:

```
x, y = 5, 10
print(x, y)

x, y = y, x    # swap
print(x, y)
```

🔍 در خط `x, y = y, x` باز هم **اول سمت راست** حساب می‌شود: مقدارهای فعلی `y` و `x` (یعنی `10` و `5`) برداشته می‌شوند و بعد به ترتیب به `x` و `y` داده می‌شوند.

قواعد نام‌گذاری

قاعده	مثال درست	مثال نادرست
فقط حروف انگلیسی، رقم و `_`	`total_sales`	`total-sales`
نباید با رقم شروع شود	`value2`	`2value`
نباید یکی از **کلمات کلیدی** پایتون باشد	`class_name`	`class`
حروف بزرگ و کوچک فرق دارند	`Name` و `name` دو متغیر متفاوت‌اند	

قرارداد رایج در پایتون: نام متغیرها با حروف کوچک و کلمات با `_` جدا شوند (به این سبک **snake_case** می‌گویند)؛ مثل `student_count` . نام باید **معنی‌دار** باشد: `average_score` خیلی بهتر از `a2` است.

کلمات کلیدی کلمه‌هایی هستند که پایتون برای خودش رزرو کرده است. فهرست کامل آن‌ها را این‌طور ببینید:

```
import keyword
print(keyword.kwlist)
```

⚠️ اشتباه رایج

- **استفاده از متغیری که هنوز ساخته نشده:** اگر قبل از انتساب از یک نام استفاده کنید، خطای `NameError` می‌گیرید (همان خطایی که در جلسه‌ی ۱ دیدید).
- **گذاشتن اسم توابع آماده روی متغیر:** نام‌هایی مثل `print` ، `type` ، `sum` ، `list` یا `str` کلمه‌ی کلیدی نیستند و پایتون اجازه می‌دهد از آن‌ها استفاده کنید؛ اما بعد از `sum = 10` دیگر تابع `sum` کار نمی‌کند! این اسم‌ها را برای متغیرها به کار نبرید.

۲. نوع داده و تابع `type`

💡 مفهوم

هر مقدار در پایتون یک **نوع (type)** دارد. نوع تعیین می‌کند آن مقدار **چه اطلاعاتی** نگه می‌دارد و **چه کارهایی** روی آن مجاز است. مثلاً دو عدد را می‌شود تقسیم کرد، ولی دو متن را نه.

تابع `type` نوع هر مقدار را نشان می‌دهد:

```
print(type(5))
print(type(3.14))
print(type("Ali"))
print(type(True))
```

چهار نوع پایه‌ی این جلسه

نوع	نام کامل	مثال	کاربرد در علوم داده
`int`	integer (عدد صحیح)	`42` ، `-7` ، `0`	تعداد، شمارنده، سال
`float`	floating point (عدد اعشاری)	`3.14` ، `-0.5` ، `2.0`	اندازه‌گیری، میانگین، احتمال
`str`	string (رشته، متن)	`"Ali"` ، `'Tehran'`	نام، دسته، متن
`bool`	boolean (منطقی)	`True` ، `False`	نتیجه‌ی مقایسه، فیلتر کردن داده

نوع پویا (dynamic typing)

نوع به **مقدار** تعلق دارد، نه به **نام**. پس یک نام می‌تواند در طول برنامه به مقدارهایی از نوع‌های مختلف اشاره کند:

```
x = 42
print(type(x))

x = "Now I am a string!"
print(type(x))
```

🔍 خط اول `x` را به یک `int` وصل می‌کند. بعد همان نام به یک `str` وصل می‌شود. پایتون اعتراضی نمی‌کند؛ اما در برنامه‌های واقعی این کار باعث سردرگمی می‌شود. **یک نام را برای یک نوع داده نگه دارید.**

۳. اعداد: `int` و `float`

💡 مفهوم

- **`int`** عدد صحیح است و در پایتون **محدودیت اندازه ندارد**؛ هر قدر حافظه اجازه دهد می‌تواند بزرگ باشد.
- **`float`** عدد اعشاری است. هر عددی که **نقطه‌ی اعشار** داشته باشد `float` است، حتی `2.0` .
- اعداد خیلی بزرگ یا خیلی کوچک را می‌توان با **نماد علمی** نوشت: `1.5e3` یعنی 1.5×10^3 و `2e-4` یعنی 0.0002 (هر دو `float` هستند).
- پایتون نوع سومی هم برای اعداد مختلط دارد (`complex` ، مثل `1 + 2j`) که در این دوره کمتر به آن نیاز داریم.

```
population = 88_550_570      # underscores make big numbers readable
print(population, type(population))

big = 2 ** 100               # int has no size limit
print(big)

distance = 1.5e3
tiny = 2e-4
print(distance, tiny, type(distance))
```

🔍 کد را بخوان

- `_` داخل عدد فقط برای خوانایی است و پایتون آن را نادیده می‌گیرد.
- `**` یعنی **توان**؛ `2 ** 100` یک عدد ۳۱ رقمی است و پایتون بدون مشکل آن را نگه می‌دارد.
- `1.5e3` با اینکه مقدارش `1500` است، از نوع `float` است.

❓ پیش‌بینی کن

نوع حاصل هر عبارت را پیش‌بینی کنید. (راهنمایی: به خروجی تقسیم `/` دقت کنید.)

```
print(type(3 + 4))
print(type(3 + 4.0))
print(type(8 / 2))
print(8 / 2)
```

🔍 دو قاعده‌ی مهم:

1. اگر در یک عبارت `int` و `float` با هم باشند، نتیجه **`float`** است.
2. تقسیم با `/` **همیشه** `float` می‌دهد، حتی وقتی تقسیم بدون باقی‌مانده باشد (`8 / 2` برابر `4.0` است، نه `4`).

⚠️ اشتباه رایج: دقت اعداد اعشاری

کامپیوتر اعداد اعشاری را در **مبنای ۲** ذخیره می‌کند و بعضی اعداد ساده مثل `0.1` در مبنای ۲ دقیقاً قابل نمایش نیستند. نتیجه را ببینید:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

🔍 حاصل `0.1 + 0.2` عددی **بسیار نزدیک** به `0.3` است، ولی دقیقاً خودش نیست؛ برای همین مقایسه‌ی `==` نتیجه‌ی `False` می‌دهد. این باگ پایتون نیست؛ در همه‌ی زبان‌های برنامه‌نویسی همین‌طور است.

درس مهم برای علوم داده: هیچ‌وقت دو `float` را مستقیم با `==` مقایسه نکنید. به جای آن بررسی کنید که **خیلی به هم نزدیک** باشند:

```
import math
print(math.isclose(0.1 + 0.2, 0.3))
print(round(0.1 + 0.2, 2))
```

۴. رشته‌ها: `str`

💡 مفهوم

رشته (string) یک دنباله از کاراکترهاست و برای نگهداری متن استفاده می‌شود. رشته را می‌توان بین `'...'` یا `"..."` نوشت؛ هر دو یکسان‌اند. برای متن چندخطی از سه علامت نقل‌قول (`"""..."""`) استفاده می‌شود.

در این جلسه فقط با اصول رشته آشنا می‌شویم؛ جلسه‌ی ۱۱ کاملاً به رشته‌ها اختصاص دارد.

```
text = "Hello"
print(text)
print(len(text))      # number of characters
print(text[0])        # first character (counting starts from 0)
print(text.upper())   # a NEW string in upper case
print(text)           # the original is unchanged

message = """This is
a multi-line
string."""
print(message)
```

🔍 کد را بخوان

- `len(text)` تعداد کاراکترها را می‌دهد: `5` .
- `text[0]` **اولین** کاراکتر است؛ در پایتون شماره‌گذاری از **صفر** شروع می‌شود.
- `text.upper()` یک رشته‌ی **جدید** با حروف بزرگ می‌سازد. خود `text` تغییر نمی‌کند؛ این را در خط بعد می‌بینید.

❓ پیش‌بینی کن

خروجی هر خط چیست؟ (به تفاوت عدد و رشته‌ای که شبیه عدد است دقت کنید.)

```
print("5" + "3")
print(5 + 3)
print("ab" * 3)
print(len("Data Science"))
```

🔍 عملگر `+` روی رشته‌ها یعنی **چسباندن** (concatenation)، نه جمع. پس `"5" + "3"` می‌شود `"53"` . عملگر `*` بین رشته و عدد صحیح، رشته را **تکرار** می‌کند. فاصله هم یک کاراکتر است؛ پس طول `"Data Science"` برابر `12` است.

⚠️ اشتباه رایج: ترکیب رشته و عدد

پایتون رشته و عدد را خودبه‌خود به هم تبدیل نمی‌کند:

```
age = 25
print("Age: " + age)
```

🔍 خط آخر پیام خطا `TypeError: can only concatenate str (not "int") to str` است؛ یعنی «فقط می‌توانم رشته را به رشته بچسبانم، نه عدد صحیح را». راه حل‌ها: `print("Age: " + str(age))` یا `print("Age:", age)` . (روش بهتر، یعنی **f-string**، را در جلسه‌ی ۴ می‌بینید.)

۵. مقادیر بولی: `bool` و مقدار `None`

💡 مفهوم

نوع `bool` فقط دو مقدار دارد: `True` (درست) و `False` (نادرست). حرف اول هر دو **بزرگ** است. نتیجه‌ی هر **مقایسه** یک مقدار بولی است و در جلسه‌ی شرط‌ها (جلسه‌ی ۵) از آن برای تصمیم‌گیری استفاده می‌کنیم.

```
a = 10
b = 5
print(a > b)
print(a == b)
print(type(a > b))
```

🔍 `a > b` یک سؤال است: «آیا `a` از `b` بزرگ‌تر است؟» و جوابش `True` است. دقت کنید `==` (دو علامت) یعنی **مقایسه**، ولی `=` (یک علامت) یعنی **انتساب**.

💡 بولی‌ها در واقع عدد هستند

در پایتون `True` مثل عدد `1` و `False` مثل عدد `0` رفتار می‌کند. این نکته در علوم داده بسیار کاربردی است: **جمع چند مقدار بولی، تعداد `True` ها را می‌دهد.**

❓ پیش‌بینی کن

```
passed_1 = True
passed_2 = False
passed_3 = True
print(passed_1 + passed_2 + passed_3)
```

🔍 حاصل `1 + 0 + 1` یعنی `2` : تعداد دانشجویانی که قبول شده‌اند. بعدها در Pandas دقیقاً با همین ایده می‌شماریم چند ردیف از داده یک شرط را دارند.

💡 مقدار ویژه‌ی `None`

`None` یعنی «**هیچ مقداری وجود ندارد**». نوع آن `NoneType` است. مثلاً وقتی مقدار یک متغیر هنوز معلوم نیست، می‌توان آن را `None` گذاشت. این مفهوم با **داده‌ی گمشده (missing data)** در علوم داده ارتباط نزدیک دارد که در جلسه‌ی ۲۶ مفصل به آن می‌پردازیم.

```
result = None
print(result)
print(type(result))
```

⚠️ اشتباه رایج

- نوشتن `true` یا `false` با حرف کوچک: پایتون آن‌ها را اسم متغیر فرض می‌کند و `NameError` می‌دهد.
- `None` با `0`، با رشته‌ی خالی `""` و با `False` **یکی نیست**؛ هر کدام معنی خودش را دارد.

۶. تبدیل نوع

💡 مفهوم

گاهی داده با نوع نامناسب به دست ما می‌رسد. مثلاً عددی که از یک فایل متنی یا از ورودی کاربر خوانده می‌شود، **همیشه رشته** است (`"25"` و نه `25`). برای تبدیل، از **نام نوع** مثل یک تابع استفاده می‌کنیم:

تابع	کار	مثال	نتیجه
`int()`	تبدیل به عدد صحیح	`int("25")`	`25`
`float()`	تبدیل به عدد اعشاری	`float("3.5")`	`3.5`
`str()`	تبدیل به رشته	`str(10)`	`"10"`
`bool()`	تبدیل به بولی	`bool(0)`	`False`

```
score_text = "17.5"            # e.g. read from a file
score = float(score_text)
print(score + 1, type(score))

print(int(3.99))               # int() cuts the decimal part, it does NOT round
print(round(3.99))             # use round() for rounding
```

🔍 کد را بخوان

- `float("17.5")` رشته را به عدد اعشاری `17.5` تبدیل می‌کند و حالا می‌شود با آن حساب کرد.
- `int(3.99)` برابر `3` است: `int` قسمت اعشاری را **دور می‌اندازد** و گرد نمی‌کند. برای گرد کردن از `round` استفاده کنید.

تبدیل به `bool`: چه چیزی «خالی» است؟

قاعده ساده است: **صفر و مقدارهای خالی** به `False` تبدیل می‌شوند و **هر چیز دیگری** به `True`.

❓ پیش‌بینی کن

قبل از اجرا، جلوی هر خط بنویسید `True` یا `False`. (به خط آخر خیلی دقت کنید!)

```
print(bool(0))
print(bool(42))
print(bool(0.0))
print(bool(""))
print(bool(" "))
print(bool("False"))
```

🔍 `" "` یک فاصله دارد، پس خالی نیست و `True` است. `"False"` هم یک رشته‌ی **پنج‌حرفی** است، پس خالی نیست و به `True` تبدیل می‌شود. `bool` به **محتوای** رشته نگاه نمی‌کند، فقط به خالی بودن یا نبودن آن.

⚠️ اشتباه رایج: تبدیل غیرممکن

هر رشته‌ای را نمی‌توان به عدد تبدیل کرد. سلول زیر را اجرا کنید و پیام خطا را بخوانید:

```
int("3.5")
```

🔍 خطای `ValueError: invalid literal for int() with base 10: '3.5'` یعنی «رشته‌ی `'3.5'` شکل یک عدد صحیح را ندارد». `int` فقط رشته‌هایی مثل `"35"` را می‌پذیرد. راه حل: اول به `float` و بعد به `int` تبدیل کنید: `int(float("3.5"))`.

دو نوع خطایی که در این جلسه دیدید را با هم مقایسه کنید:

خطا	معنی	مثال
`TypeError`	عملیات برای این **نوع** مجاز نیست	`"Age: " + 25`
`ValueError`	نوع درست است، اما **مقدار** مناسب نیست	`int("3.5")`

📌 آنچه باید بلد باشید

- `=` یعنی **انتساب**: اول سمت راست حساب می‌شود، بعد نام به نتیجه وصل می‌شود؛ `==` یعنی **مقایسه**.
- بعد از `b = a`، تغییر دادن `a` با یک انتساب جدید روی `b` اثری ندارد.
- نام‌های مجاز و غیرمجاز متغیر را تشخیص دهید.
- نوع هر مقدار را بدانید و نوع **حاصل یک عبارت** را پیش‌بینی کنید (مخصوصاً: ترکیب `int` و `float`، و تقسیم `/`).
- چرا `0.1 + 0.2 == 0.3` نادرست است و به جای آن چه باید کرد.
- رفتار `+` و `*` روی رشته‌ها و خطای ترکیب رشته با عدد.
- `True` و `False` مثل `1` و `0` رفتار می‌کنند.
- نتیجه‌ی `int()`، `float()`، `str()` و `bool()` را برای ورودی‌های مختلف پیش‌بینی کنید.
- تفاوت `TypeError` و `ValueError`.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی خروجی

خروجی کد زیر را دقیقاً بنویسید.

```
x = 5
y = x
x = x + 1
y = "x"
print(x, y)
```

▼ پاسخ

```
6 x
```

خط سوم `x` را به `6` وصل می‌کند. در خط چهارم `y` به رشته‌ی `"x"` (یک حرف) وصل می‌شود، نه به متغیر `x`؛ چون داخل نقل‌قول است.

سؤال ۲ — پیش‌بینی نوع

نوع (`type`) هر یک از عبارت‌های زیر چیست؟

عبارت	نوع؟
`7 + 3`	
`7 + 3.0`	
`12 / 4`	
`"7" + "3"`	
`7 > 3`	
`True + True`	

▼ پاسخ

عبارت	نوع	مقدار
`7 + 3`	`int`	`10`
`7 + 3.0`	`float`	`10.0`
`12 / 4`	`float`	`3.0`
`"7" + "3"`	`str`	`"73"`

عبارت	نوع	مقدار
`7 > 3`	`bool`	`True`
`True + True`	`int`	`2`

سؤال ۳ — تبدیل نوع

حاصل هر عبارت را بنویسید. اگر خطا می‌دهد، **نوع خطا** را بنویسید.

`str(7) + bool("0")` — `int("7.0")` — `float("7")` — `int("7")` — `int(7.9)` — `bool(None)` — `str(7)`

▼ پاسخ

عبارت	نتیجه
`int(7.9)`	`7` (قسمت اعشاری حذف می‌شود)
`int("7")`	`7`
`float("7")`	`7.0`
`int("7.0")`	`ValueError`
`bool("0")`	`True` (رشته‌ی غیرخالی)
`str(7) + str(7)`	`"77"`
`bool(None)`	`False`

سؤال ۴ — پیدا کردن خطا

کد زیر قرار است قیمت کل را چاپ کند، اما خطا می‌دهد. نوع خطا چیست و دو راه برای درست کردن آن بنویسید.

```
price = 120000
count = 3
print("Total price: " + price * count)
```

▼ پاسخ

خطا از نوع `TypeError` است: حاصل `price * count` یک `int` است و نمی‌شود آن را با `+` به رشته چسباند.

راه حل ۱: `print("Total price: " + str(price * count))`

راه حل ۲: `print("Total price:", price * count)`

سؤال ۵ — توضیح کد

دانشجویی برای بررسی اینکه مجموع دو احتمال برابر `0.3` است، کد زیر را نوشته و خروجی `False` گرفته است. توضیح دهید چرا، و کد را اصلاح کنید.

```
p1 = 0.1
p2 = 0.2
print(p1 + p2 == 0.3)
```

▼ **پاسخ**

اعداد اعشاری در کامپیوتر در مبنای ۲ ذخیره می‌شوند و `0.1` و `0.2` دقیقاً قابل نمایش نیستند. حاصل جمع `0.30000000000000004` است، نه دقیقاً `0.3`، پس `==` نتیجه‌ی `False` می‌دهد.

اصلاح:

```
import math
print(math.isclose(p1 + p2, 0.3))
```

سؤال ۶ — نام‌گذاری

کدام نام‌ها برای متغیر **مجاز** هستند؟ برای نام‌های غیرمجاز دلیل بیاورید. کدام نام مجاز است اما **استفاده از آن توصیه نمی‌شود**؟

`sum` — `Class` — `class` — `total_sales` — `my-var` — `_count` — `2nd_value`

▼ **پاسخ**

- `2nd_value` : غیرمجاز (با رقم شروع شده)
- `_count` : مجاز
- `my-var` : غیرمجاز (`-` مجاز نیست؛ پایتون آن را تفریق می‌فهمد)
- `total_sales` : مجاز (و به سبک snake_case)
- `class` : غیرمجاز (کلمه‌ی کلیدی)
- `Class` : مجاز (با `class` فرق دارد، چون حروف بزرگ و کوچک فرق دارند)
- `sum` : مجاز، اما **توصیه نمی‌شود**؛ چون تابع آماده‌ی `sum` را از کار می‌اندازد.

سؤال ۷ — تکمیل کد

داده‌های زیر از یک فایل متنی خوانده شده‌اند و همه **رشته** هستند. جای خالی را طوری کامل کنید که میانگین نمره‌ی کلاس چاپ شود.

```
n_students = "25"
total_score = "412.5"
```

```
average = _______
print(average)
```

▼ پاسخ

```
average = float(total_score) / int(n_students)
```

خروجی `16.5` است. اگر تبدیل نوع را فراموش کنید، `"412.5" / "25"` خطای `TypeError` می‌دهد، چون تقسیم برای رشته‌ها تعریف نشده است.

خلاصه‌ی جلسه

- متغیر یک **نام** است که به یک **مقدار** اشاره می‌کند؛ `=` انتساب است و اول سمت راست حساب می‌شود.
- نوع به مقدار تعلق دارد و با `type` دیده می‌شود. چهار نوع پایه: `int` ، `float` ، `str` ، `bool` ؛ و مقدار ویژه‌ی `None` .
- ترکیب `int` و `float` و همچنین تقسیم `/` همیشه `float` می‌دهد. اعداد اعشاری را با `==` مقایسه نکنید.
- `+` روی رشته‌ها یعنی چسباندن؛ رشته و عدد خودبه‌خود ترکیب نمی‌شوند.
- `True` و `False` مثل `1` و `0` رفتار می‌کنند.
- `int()` ، `float()` ، `str()` و `bool()` نوع را تبدیل می‌کنند؛ `TypeError` و `ValueError` را از هم تشخیص دهید.

جلسه‌ی بعد: عملگرها (ریاضی، مقایسه‌ای، منطقی و انتساب) و اولویت آن‌ها.

جلسه ۳: عملگرها

Operators

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- حاصل عبارت‌های ریاضی، به‌ویژه تقسیم صحیح `//` و باقی‌مانده `%`، را **بدون اجرا** پیش‌بینی کنید،
- با عملگرهای مقایسه‌ای و منطقی شرط‌های ترکیبی بسازید و مقدار آن‌ها را تعیین کنید،
- با عملگرهای انتساب ترکیبی (`+=`، `-=` و ...) کار کنید و مقدار متغیر را خط به خط **دنبال (trace)** کنید،
- عملگر عضویت `in` را به کار ببرید،
- **اولویت عملگرها** را بدانید و ترتیب محاسبه‌ی یک عبارت طولانی را مشخص کنید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. عملگرهای ریاضی
2. عملگرهای مقایسه‌ای
3. عملگرهای منطقی
4. عملگرهای انتساب
5. عملگر عضویت
6. اولویت عملگرها

۱. عملگرهای ریاضی

💡 مفهوم

عملگر (operator) نمادی است که روی یک یا چند مقدار (**عملوند**) کاری انجام می‌دهد و یک مقدار جدید می‌سازد. در `3 + 4`، علامت `+` عملگر و `3` و `4` عملوندها هستند.

عملگر	نام	مثال	نتیجه
`+`	جمع	`5 + 2`	`7`
`-`	تفریق	`5 - 2`	`3`
`*`	ضرب	`5 * 2`	`10`
`/`	تقسیم	`5 / 2`	`2.5`
`//`	تقسیم صحیح (خارج‌قسمت)	`5 // 2`	`2`
`%`	باقی‌مانده	`5 % 2`	`1`
`**`	توان	`5 ** 2`	`25`

```
a = 10
b = 3
print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a // b =", a // b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
```

🔍 کد را بخوان: `//` و `%` با هم

`//` و `%` دو نیمه‌ی یک تقسیم با باقی‌مانده هستند. وقتی `10` را بر `3` تقسیم می‌کنیم: خارج‌قسمت `3` و باقی‌مانده `1` است، چون

`10 = 3 × 3 + 1`

همیشه این رابطه برقرار است: `a == (a // b) * b + (a % b)`

این دو عملگر در عمل بسیار پرکاربردند. کد زیر را بخوانید و قبل از اجرا حدس بزنید هر خط چه می‌کند:

```
total_minutes = 135
hours = total_minutes // 60
minutes = total_minutes % 60
print(hours, "h", minutes, "min")

n = 2026
print(n % 2 == 0)      # is n even?
print(n % 10)          # last digit of n
print(n // 10)         # n without its last digit
```

- `135 // 60` تعداد ساعت‌های **کامل** است (`2`) و `135 % 60` دقیقه‌های **باقی‌مانده** (`15`).

- عددی زوج است که باقی‌مانده‌اش بر ۲ صفر باشد: `n % 2 == 0` .
- `n % 10` رقم یکان عدد را می‌دهد و `n // 10` رقم یکان را حذف می‌کند.

❓ پیش‌بینی کن

حاصل هر خط چیست؟ (خط‌های سوم و چهارم را با دقت فکر کنید.)

```
print(17 // 5, 17 % 5)
print(17.0 // 5)
print(-7 // 2)
print(-7 % 2)
```

🔍 نکته‌ها:

- اگر یکی از عملوندها `float` باشد، `//` هم `float` برمی‌گرداند: `17.0 // 5` برابر `3.0` است.
- `//` همیشه به سمت **پایین** (منفی بی‌نهایت) گرد می‌کند، نه به سمت صفر. حاصل `-7 / 2` برابر `-3.5` است و پایین‌ترین عدد صحیح بعد از آن `-4` است، نه `-3` .
- پس باقی‌مانده باید طوری باشد که رابطه‌ی بالا برقرار بماند: `-7 == (-4) * 2 + 1` ؛ یعنی `-7 % 2` برابر `1` است.

⚠️ اشتباه رایج

- `^` **توان نیست!** در پایتون توان با `**` نوشته می‌شود. علامت `^` عملگر دیگری است (XOR بیتی) و `2 ^ 3` برابر `1` است، نه `8` . این خطا پیام خطا نمی‌دهد و فقط جواب غلط می‌دهد؛ پس خطرناک‌تر است.
- **تقسیم بر صفر:** `5 / 0` ، `5 // 0` و `5 % 0` همه خطای `ZeroDivisionError` می‌دهند.

```
print(2 ^ 3)
print(2 ** 3)
```

۲. عملگرهای مقایسه‌ای

💡 مفهوم

عملگرهای مقایسه‌ای دو مقدار را مقایسه می‌کنند و نتیجه‌ی آن‌ها **همیشه** یک `bool` است (`True` یا `False`).

عملگر	معنی	مثال	نتیجه
`==`	مساوی است	`5 == 5`	`True`
`!=`	مساوی نیست	`5 != 3`	`True`
`>`	بزرگ‌تر	`5 > 3`	`True`

عملگر	معنی	مثال	نتیجه
`<`	کوچک‌تر	`5 < 3`	`False`
`>=`	بزرگ‌تر یا مساوی	`5 >= 5`	`True`
`<=`	کوچک‌تر یا مساوی	`3 <= 2`	`False`

```
x = 5
y = 10
print(x == y)
print(x != y)
print(x < y)
```

💡 مقایسه‌ی زنجیره‌ای

در پایتون می‌توان چند مقایسه را مثل ریاضی پشت سر هم نوشت. `0 <= score <= 20` یعنی **و** `0 <= score` و `score <= 20`:

```
score = 17.5
print(0 <= score <= 20)     # is it a valid grade?

score = 21
print(0 <= score <= 20)
```

💡 مقایسه‌ی رشته‌ها

رشته‌ها را هم می‌توان مقایسه کرد. مقایسه **حرف به حرف** و بر اساس ترتیب الفبایی (دقیق‌تر: کد کاراکترها) انجام می‌شود؛ مثل ترتیب کلمات در فرهنگ لغت. حروف بزرگ انگلیسی **قبل از** حروف کوچک می‌آیند.

❓ پیش‌بینی کن

جلوی هر خط بنویسید `True` یا `False`. (خط آخر را با دقت فکر کنید.)

```
print("apple" < "banana")
print("Ali" == "ali")
print("Z" < "a")
print(10 < 9)
print("10" < "9")
```

🔍 در خط آخر، دو **رشته** مقایسه می‌شوند، نه دو عدد. مقایسه از حرف اول شروع می‌شود: `"1"` کوچک‌تر از `"9"` است، پس `"10" < "9"` درست است؛ بقیه‌ی حروف اصلاً بررسی نمی‌شوند.

چرا برای علوم داده مهم است؟ اگر عددها به شکل رشته در داده ذخیره شده باشند (مثلاً از یک فایل CSV)، مرتب‌سازی آن‌ها نتیجه‌ی غلط می‌دهد: `"10"` قبل از `"9"` قرار می‌گیرد. قبل از مقایسه، نوع را تبدیل کنید (جلسه‌ی ۲).

⚠️ اشتباه رایج

- **`=` به جای `==`:** `x = 5` مقدار می‌دهد؛ `x == 5` سؤال می‌پرسد.
- **مقایسه‌ی `float` با `==`:** همان‌طور که در جلسه‌ی ۲ دیدید، `0.1 * 3 == 0.3` نادرست است. از `math.isclose` استفاده کنید.

۳. عملگرهای منطقی

💡 مفهوم

عملگرهای منطقی چند شرط را با هم ترکیب می‌کنند:

عملگر	معنی	وقتی `True` است که ...
`and`	و	**هر دو** طرف `True` باشند
`or`	یا	**حداقل یکی** از دو طرف `True` باشد
`not`	نقیض	عملوند `False` باشد (مقدار را برعکس می‌کند)

جدول درستی:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```python
score = 14
absences = 4
has_excuse = True

passed = score >= 10 and (absences <= 3 or has_excuse)
print(passed)
```

🔍 کد را بخوان

قانون قبولی: «نمره حداقل ۱۰ باشد **و** (غیبت حداکثر ۳ جلسه باشد **یا** دانشجو عذر موجه داشته باشد)». مرحله به مرحله:

1. `score >= 10` برابر `True` است.
2. `absences <= 3` برابر `False` است (چهار غیبت).

3. داخل پرانتز: `False or has_excuse` یعنی `False or True` که برابر `True` است.
4. در آخر: `True and True` که برابر `True` است.

پرانتز اینجا ضروری است. بدون آن، معنی شرط عوض می‌شود (در بخش ۶ می‌بینید چرا).

💡 ارزیابی کوتاه (short-circuit)

پایتون عبارت منطقی را از چپ به راست حساب می‌کند و **به محض اینکه جواب معلوم شد، متوقف می‌شود**:

- در `A and B` اگر `A` نادرست باشد، `B` اصلاً حساب نمی‌شود (جواب حتماً `False` است).
- در `A or B` اگر `A` درست باشد، `B` اصلاً حساب نمی‌شود (جواب حتماً `True` است).

این رفتار برای نوشتن شرط‌های **امن** کاربرد دارد:

```
count = 0
total = 50
print(count != 0 and total / count > 10)
```

🔍 چون `count != 0` نادرست است، پایتون به `total / count` نمی‌رسد و خطای تقسیم بر صفر رخ نمی‌دهد. اگر ترتیب دو شرط را عوض کنید، `ZeroDivisionError` می‌گیرید. امتحان کنید!

❓ پیش‌بینی کن

کد زیر قرار است بررسی کند که آیا `x` برابر `1` یا `2` است. با `x = 5` چه چیزی چاپ می‌شود؟

```
x = 5
print(x == 1 or 2)
```

⚠️ اشتباه رایج: `x == 1 or 2`

این یکی از رایج‌ترین باگ‌هاست. پایتون این عبارت را این‌طور می‌خواند: `(x == 1) or 2`، نه «x برابر 1 یا 2». سمت چپ `False` است، پس `or` مقدار سمت راست یعنی `2` را برمی‌گرداند؛ و `2` (مثل هر عدد غیرصفر) در شرط‌ها «درست» حساب می‌شود. نتیجه: این شرط **همیشه** برقرار است!

شکل درست:

```
x == 1 or x == 2
```

> **نکته برای آینده:** در NumPy و Pandas برای ترکیب شرط‌ها روی ستون‌های داده از `&` (به جای `and`)، `|` (به جای `or`) و `~` (به جای `not`) استفاده می‌شود. این را در بخش ۳ دوره می‌بینید.

۴. عملگرهای انتساب

💡 مفهوم

خیلی وقت‌ها می‌خواهیم مقدار یک متغیر را بر اساس مقدار قبلی‌اش تغییر دهیم. پایتون برای این کار شکل کوتاه‌تری دارد:

عملگر	مثال	معادل
`=`	`x = 5`	—
`+=`	`x += 3`	`x = x + 3`
`-=`	`x -= 2`	`x = x - 2`
`*=`	`x *= 2`	`x = x * 2`
`/=`	`x /= 2`	`x = x / 2`
`//=`	`x //= 2`	`x = x // 2`
`%=`	`x %= 3`	`x = x % 3`
`**=`	`x **= 2`	`x = x ** 2`

```
balance = 1000
balance += 250     # deposit
balance -= 400     # withdraw
balance *= 1.1     # 10 percent interest
print(balance)
```

🔍 کد را بخوان: دنبال کردن مقدار (trace)

یکی از مهم‌ترین مهارت‌های خواندن کد، نوشتن **جدول دنبال کردن** است: بعد از هر خط، مقدار متغیر را بنویسید.

خط	مقدار `balance`	نوع
`balance = 1000`	`1000`	`int`
`balance += 250`	`1250`	`int`
`balance -= 400`	`850`	`int`
`balance *= 1.1`	`935.0000000000001`	`float`

دقت کنید که بعد از ضرب در `1.1` نوع به `float` تغییر کرد، و چرا آخر عدد یک `1` اضافه دیده می‌شود؟ (همان مسئله‌ی دقت اعشاری از جلسه‌ی ۲.)

❓ پیش‌بینی کن

جدول دنبال کردن را برای کد زیر روی کاغذ بکشید و خروجی را پیش‌بینی کنید.

```
x = 10
x += 5
x *= 2
x //= 4
x %= 3
print(x)
```

⚠️ اشتباه رایج

- `x =+ 3` با `x += 3` فرق دارد! اولی یعنی `x = +3` (مقدار ۳ را بده) و خطا هم نمی‌دهد.
- `+=` روی رشته‌ها هم کار می‌کند و یعنی **چسباندن**: بعد از `s = "ab"` و `s += "c"`، مقدار `s` برابر `"abc"` است.

۵. عملگر عضویت

💡 مفهوم

عملگر `in` بررسی می‌کند که آیا یک مقدار **داخل** یک مجموعه از مقدارها هست یا نه؛ `not in` برعکس آن است. نتیجه یک `bool` است. فعلاً آن را روی رشته‌ها می‌بینیم؛ در جلسه‌های ۸ تا ۱۰ روی لیست، تاپل، مجموعه و دیکشنری هم استفاده می‌شود.

```
course = "Python for Data Science"
print("Data" in course)
print("data" in course)
print("R" not in course)
print("" in course)
```

🔍 `in` روی رشته‌ها دنبال یک **زیررشته** می‌گردد و به حروف بزرگ و کوچک حساس است؛ برای همین `"data"` پیدا نمی‌شود. رشته‌ی خالی `""` جزو هر رشته‌ای حساب می‌شود.

۶. اولویت عملگرها

💡 مفهوم

وقتی یک عبارت چند عملگر دارد، پایتون آن‌ها را با یک **ترتیب ثابت** حساب می‌کند؛ مثل ریاضی که ضرب قبل از جمع است.

جدول زیر از **بالاترین** اولویت به **پایین‌ترین** است:

اولویت	عملگر	توضیح
۱	`( )`	پرانتز
۲	`**`	توان (از **راست** به چپ)
۳	`-x`	منفی کردن
۴	`*` ، `/` ، `//` ، `%`	ضرب و تقسیم (از چپ به راست)
۵	`+` ، `-`	جمع و تفریق (از چپ به راست)
۶	`==` ، `!=` ، `<` ، `>` ، `<=` ، `>=` ، `in` ، `not in`	مقایسه
۷	`not`	نقیض
۸	`and`	و
۹	`or`	یا

عملگرهای هم‌اولویت از **چپ به راست** حساب می‌شوند؛ تنها استثنای مهم `**` است که از **راست به چپ** حساب می‌شود.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(10 / 2 * 5)
print(2 ** 3 ** 2)
print(-3 ** 2)
```

🔍 کد را بخوان

عبارت	ترتیب محاسبه	نتیجه
`2 + 3 * 4`	اول `3 * 4`	`14`
`(2 + 3) * 4`	اول پرانتز	`20`
`10 / 2 * 5`	هم‌اولویت، از چپ: `(10 / 2) * 5`	`25.0`
`2 ** 3 ** 2`	از راست: `2 ** (3 ** 2)` یعنی `2 ** 9`	`512`
`-3 ** 2`	توان قبل از منفی: `-(3 ** 2)`	`-9`

❓ پیش‌بینی کن

اولویت `and` از `or` بیشتر است. حاصل این دو خط چیست؟

```
print(True or False and False)
print((True or False) and False)
```

🔍 در خط اول، `and` زودتر حساب می‌شود: `True or (False and False)` یعنی `True or False` که `True` است. در خط دوم پرانتز ترتیب را عوض کرده و نتیجه `False` است.

⚠️ توصیه

لازم نیست کل جدول اولویت را حفظ کنید؛ **در کد خودتان از پرانتز استفاده کنید** تا منظورتان برای هر خواننده‌ای روشن باشد. اما برای **خواندن** کد دیگران (و سؤال‌های امتحان!) باید بدانید پایتون یک عبارت بدون پرانتز را چطور حساب می‌کند.

📌 آنچه باید بلد باشید

- حاصل `/`، `//`، `%` و `**` را برای اعداد مثبت و **منفی** پیش‌بینی کنید؛ رابطه‌ی `a == (a // b) * b + a % b` را بدانید.
- کاربردهای `//` و `%`: زوج و فرد بودن، رقم یکان، تبدیل دقیقه به ساعت و دقیقه.
- `^` توان نیست؛ توان `**` است.
- نتیجه‌ی مقایسه‌ها `bool` است؛ مقایسه‌ی زنجیره‌ای و مقایسه‌ی رشته‌ها (از جمله `"10" < "9"`) را بفهمید.
- مقدار عبارت‌های ترکیبی با `and`، `or` و `not` را تعیین کنید؛ ارزیابی کوتاه را توضیح دهید.
- باگ `x == 1 or 2` را تشخیص دهید و درست کنید.
- برای یک دنباله از انتساب‌های ترکیبی (`+=`، `*=` و ...) جدول دنبال کردن بنویسید.
- `in` و `not in` را روی رشته‌ها به کار ببرید.
- ترتیب محاسبه‌ی یک عبارت بدون پرانتز را بر اساس اولویت عملگرها مشخص کنید.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی حاصل

حاصل هر عبارت را بنویسید (به نوع نتیجه هم دقت کنید).

عبارت	حاصل؟
`17 // 5`	
`17 % 5`	
`17 / 5`	

عبارت	حاصل؟
`-17 // 5`	
`-17 % 5`	
`2 ** -1`	
`9 % 3`	
`7.5 // 2`	

▼ پاسخ

عبارت	حاصل
`17 // 5`	`3`
`17 % 5`	`2`
`17 / 5`	`3.4`
`-17 // 5`	`-4` (گرد کردن به پایین)
`-17 % 5`	`3` (چون `( -17 == (-4) * 5 + 3 )`)
`2 ** -1`	`0.5`
`9 % 3`	`0`
`7.5 // 2`	`3.0` (چون یک عملوند `float` است)

سؤال ۲ – دنبال کردن مقدار

جدول دنبال کردن را کامل کنید و خروجی را بنویسید.

```
total = 7
total *= 3
total -= 1
total //= 3
total **= 2
total %= 10
print(total)
```

▼ پاسخ

خط	`total`
`total = 7`	`7`
`total *= 3`	`21`

خط	total
`total -= 1`	20
`total //= 3`	6
`total **= 2`	36
`total %= 10`	6

خروجی: `6`

سؤال ۳ — پیدا کردن خطا

کد زیر قرار است بررسی کند که آیا `day` روز تعطیل (پنجشنبه یا جمعه) است. با `day = "Mon"` خروجی `Fri` چاپ می‌شود! علت را توضیح دهید و کد را درست کنید.

```
day = "Mon"
is_weekend = day == "Thu" or "Fri"
print(is_weekend)
```

▼ پاسخ

عبارت به شکل `(day == "Thu") or "Fri"` حساب می‌شود. سمت چپ `False` است، پس `or` مقدار سمت راست یعنی رشته‌ی `"Fri"` را برمی‌گرداند. این رشته غیرخالی است و در شرط‌ها «درست» حساب می‌شود؛ پس این شرط برای هر روزی برقرار است.

اصلاح:

```
is_weekend = day == "Thu" or day == "Fri"
```
یا (بعد از یادگیری لیست‌ها): `day in ["Thu", "Fri"]`

سؤال ۴ — توضیح کد و اولویت

عبارت زیر بررسی می‌کند که سال `year` کبیسه (در تقویم میلادی) است یا نه:

```
year % 4 == 0 and year % 100 != 0 or year % 400 == 0
```
الف) با توجه به اولویت عملگرها، جای پرانتزهایی را که پایتون در نظر می‌گیرد مشخص کنید.

ب) مقدار عبارت را برای سال‌های `2024`، `2026`، `1900` و `2000` تعیین کنید.

▼ پاسخ

الف) عملگرهای ریاضی اول، بعد مقایسه‌ها، بعد `and` و در آخر `or`:

```
((year % 4 == 0) and (year % 100 != 0)) or (year % 400 == 0)
```
یعنی: «بر ۴ بخش‌پذیر باشد و بر ۱۰۰ نباشد، **یا** بر ۴۰۰ بخش‌پذیر باشد».

ب)

سال	بر ۴	بر ۱۰۰ نه	بر ۴۰۰	نتیجه
2024	True	True	False	True
2026	False	—	False	False
1900	True	False	False	False
2000	True	False	True	True

سؤال ۵ — اولویت عملگرها

حاصل هر عبارت را بنویسید.

`not 3 > 5` — `20 / 4 * 2` — `20 - 6 - 4` — `2 ** 2 ** 3` — `(-2) ** 2` — `-2 ** 2` — `and 2 < 1`

▼ پاسخ

عبارت	حاصل	توضیح
`-2 ** 2`	`-4`	توان قبل از منفی
`(-2) ** 2`	`4`	
`2 ** 2 ** 3`	`256`	از راست: `2 ** 8`
`20 - 6 - 4`	`10`	از چپ: `(20 - 6) - 4`
`20 / 4 * 2`	`10.0`	از چپ: `(20 / 4) * 2`
`not 3 > 5 and 2 < 1`	`False`	`(not False) and False`

سؤال ۶ — مقایسه

جلوی هر عبارت `True` یا `False` بنویسید و برای موارد غیرمنتظره دلیل بیاورید.

`0.1 * 3` — `"sci" in "Data Science"` — `3 < 5 < 4` — `"Ali" < "ali"` — `"10" < "9"` — `10 == 10.0` — `== 0.3`

▼ پاسخ

عبارت	نتیجه	دلیل
`"10" < "9"`	`True`	مقایسه‌ی رشته، حرف به حرف: `"1" < "9"`
`"Ali" < "ali"`	`True`	حروف بزرگ قبل از حروف کوچک‌اند

عبارت	نتیجه	دلیل
`3 < 5 < 4`	`False`	یعنی `3 < 5 and 5 < 4`
`"sci" in "Data Science"`	`False`	حساس به حروف بزرگ و کوچک: `"Sci"` هست، `"sci"` نیست
`0.1 * 3 == 0.3`	`False`	دقت محدود اعداد اعشاری
`10 == 10.0`	`True`	مقدار عددی برابر است، نوع مهم نیست

سؤال ۷ — تکمیل کد

الف) `seconds` تعداد ثانیه‌های یک ویدیو است. جاهای خالی را با `//` و `%` کامل کنید تا مدت ویدیو به شکل دقیقه و ثانیه چاپ شود (برای `seconds = 754` باید `12 min 34 s` چاپ شود).

```
seconds = 754
m = _______
s = _______
print(m, "min", s, "s")
```

ب) یک عبارت بولی بنویسید که بررسی کند `age` بین ۱۸ تا ۳۰ (هر دو شامل) است **و** `city` برابر `"Bushehr"` نیست.

▼ پاسخ

الف)

```
m = seconds // 60
s = seconds % 60
```

ب)

```
18 <= age <= 30 and city != "Bushehr"
```

خلاصه‌ی جلسه

- عملگرهای ریاضی: `+ - * / // % **`. تقسیم `/` همیشه `float` می‌دهد؛ `//` به پایین گرد می‌کند و `%` باقی‌مانده‌ی متناظر با آن را می‌دهد.
- مقایسه‌ها `bool` برمی‌گردانند؛ رشته‌ها حرف به حرف مقایسه می‌شوند.
- `and`، `or` و `not` شرط‌ها را ترکیب می‌کنند؛ ارزیابی از چپ و کوتاه است. مراقب باگ `x == 1 or 2` باشید.
- `x += 1` شکل کوتاه `x = x + 1` است؛ برای خواندن کد، جدول دنبال کردن بنویسید.
- `in` عضویت را بررسی می‌کند.
- اولویت: پرانتز، توان، ضرب و تقسیم، جمع و تفریق، مقایسه، `not`، `and`، `or`. در کد خودتان پرانتز بگذارید.

جلسه‌ی بعد: ورودی و خروجی (`input`، `print` و قالب‌بندی با f-string).

جلسه ۴: ورودی و خروجی

Input / Output

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- خروجی دقیق `print` را با پارامترهای `sep` و `end` و کاراکترهای ویژه (`\n` ، `\t`) پیش‌بینی کنید،
- تفاوت **چاپ کردن** یک مقدار و **نمایش** آن در نوت‌بوک را توضیح دهید،
- با `input` از کاربر داده بگیرید و بدانید چرا نتیجه‌ی آن **همیشه رشته** است،
- با **f-string** متن و مقدار را ترکیب کنید و اعداد را قالب‌بندی کنید (تعداد رقم اعشار، جداکننده‌ی هزارگان، درصد، عرض ستون)،
- کدهای قدیمی‌تری را که با `format()` یا `%` نوشته شده‌اند بخوانید و به f-string تبدیل کنید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. تابع `print()`
2. تابع `input()`
3. f-string
4. متد `format()` و روش‌های قدیمی‌تر

۱. تابع `print()`

💡 مفهوم

`print` مقدارهایی را که به آن می‌دهید به **متن** تبدیل و روی خروجی نمایش می‌دهد. در جلسه‌ی ۱ دیدید که می‌تواند چند مقدار بگیرد و با `sep` جداکننده‌ی آن‌ها را عوض کرد. پارامتر مهم دیگر `end` است: چیزی که **بعد از** آخرین مقدار

چاپ می‌شود. مقدار پیش‌فرض `end` یک **خط جدید** (`"\n"`) است؛ برای همین هر `print` در یک خط جدا چاپ می‌شود.

پارامتر	معنی	مقدار پیش‌فرض
`sep`	جداکننده‌ی بین مقدارها	`" "` (یک فاصله)
`end`	آنچه در انتها چاپ می‌شود	`"\n"` (خط جدید)

```
a = 5
b = 10
print("a + b =", a + b)

print("Loading", end="")
print("...", end="")
print("done")

print("2026", "10", "30", sep="/")
print()                        # an empty line
print("end of output")
```

🔍 کد را بخوان

- دو `print` با `end=""` خط جدید چاپ نمی‌کنند؛ پس `Loading` ، `...` و `done` در **یک خط** کنار هم قرار می‌گیرند.
- `print()` بدون ورودی فقط همان `end` را چاپ می‌کند؛ یعنی یک خط خالی.

💡 کاراکترهای ویژه

داخل رشته، علامت `\` (backslash) شروع یک **کاراکتر ویژه** است:

نوشتن	معنی
`\n`	خط جدید
`\t`	فاصله‌ی جدول (tab)
`\\`	خود علامت `\`
`\"`	علامت `"` داخل رشته‌ای که با `"` شروع شده

```
print("Name:\tAli\nScore:\t17")
print("He said \"hello\"")
print("C:\\Users\\Ali")
```

❓ پیش‌بینی کن

خروجی دقیق سلول زیر چند خط است و هر خط چیست؟

```
print("A", "B", sep="-", end="|")
print("C")
print("D", end="")
print("E")
```

🔍 اولین `print` چاپ می‌کند `A-B|` و به خط بعد **نمی‌رود**؛ پس `C` کنار آن می‌آید و بعد خط عوض می‌شود. سومی `D` را بدون خط جدید چاپ می‌کند و `E` کنار آن قرار می‌گیرد. نتیجه دو خط است: `A-B|C` و `DE`.

💡 چاپ کردن یا نمایش دادن؟

در نوت‌بوک دو راه برای دیدن یک مقدار دارید و نتیجه‌ی آن‌ها کمی فرق دارد:

```
"Hello"
```

```
print("Hello")
```

🔍 در سلول اول، نوت‌بوک **مقدار** آخرین عبارت را نشان می‌دهد؛ برای رشته این نمایش **با نقل‌قول** است (`'Hello'`) تا معلوم باشد این مقدار یک رشته است. `print` اما **متن** را برای کاربر چاپ می‌کند، بدون نقل‌قول. هنگام بررسی داده، این فرق کمک می‌کند بفهمید با عدد `5` سروکار دارید یا با رشته‌ی `'5'`.

⚠️ اشتباه رایج: `print` مقداری برنمی‌گرداند

`print` فقط **نمایش** می‌دهد و حاصلش `None` است. اگر نتیجه‌ی آن را در متغیر بگذارید، چیزی جز `None` به دست نمی‌آورید:

```
result = print("Hi")
print(result)
```

این اشتباه در جلسه‌ی توابع (جلسه‌ی ۷) بسیار مهم می‌شود: **چاپ کردن یک مقدار با برگرداندن آن یکی نیست.**

۲. تابع `input()`

💡 مفهوم

`input` برنامه را **متوقف** می‌کند، متنی را (اگر داده باشید) به‌عنوان **پیام راهنما** نشان می‌دهد و منتظر می‌ماند تا کاربر چیزی تایپ کند و Enter بزند. نکته‌ی کلیدی:

> **`input` همیشه یک رشته (`str`) برمی‌گرداند؛ حتی اگر کاربر عدد تایپ کند.**

💡 اجرای `input` در نوت‌بوک

- در **VS Code** یک کادر ورود متن در **بالای پنجره** باز می‌شود.
- در **Jupyter** یک کادر ورود متن **زیر سلول** ظاهر می‌شود.

تا زمانی که چیزی وارد نکنید، سلول در حال اجرا می‌ماند (علامت `[*]`) و **هیچ سلول دیگری اجرا نمی‌شود**. اگر گیر کردید، Enter بزنید یا اجرای کرنل را متوقف کنید (Interrupt).

سلول زیر را اجرا کنید و نام خود را (به انگلیسی) وارد کنید:

```
name = input("Please enter your name: ")
print("Hello,", name)
print(type(name))
```

🔍 کد را بخوان: عدد گرفتن از کاربر

اگر عدد لازم دارید، باید رشته را خودتان **تبدیل** کنید (جلسه‌ی ۲). کد زیر را بخوانید و بعد اجرا کنید:

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)
```

خط اول را از **داخل به بیرون** بخوانید:

1. `input(...)` متن تایپ‌شده را به‌عنوان رشته برمی‌گرداند؛ مثلاً `"25"` .
2. `int(...)` این رشته را به عدد صحیح `25` تبدیل می‌کند.
3. نام `age` به عدد `25` وصل می‌شود.

❓ پیش‌بینی کن

سلول زیر را اجرا کنید و دو بار عدد `5` و `3` را وارد کنید. **قبل از اجرا** بنویسید چه چیزی چاپ می‌شود.

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
```

🔍 خروجی `53` است، نه `8` ! چون `a` و `b` رشته‌اند و `+` روی رشته‌ها یعنی چسباندن. برای جمع عددی باید `int(a) + int(b)` بنویسید.

⚠️ اشتباه رایج

- **فراموش کردن تبدیل نوع:** همین مثال بالا. اگر به جای `+` از `-` یا `/` استفاده کنید، این بار خطای `TypeError` می‌گیرید.
- **ورودی نامعتبر:** اگر کاربر برای `int(input(...))` عبارت `abc` یا حتی `3.5` را وارد کند، خطای `ValueError` رخ می‌دهد. در جلسه‌ی ۱۵ (مدیریت خطا) یاد می‌گیرید چطور این خطا را کنترل کنید.

💡 `input` در علوم داده

در تحلیل داده به‌ندرت داده را با `input` می‌گیریم؛ داده معمولاً از **فایل** خوانده می‌شود (جلسه‌ی ۱۶ و بخش ۳). اما یک درس `input` همیشه معتبر است: **داده‌ای که از بیرون برنامه می‌آید معمولاً متن است** و باید نوع آن را بررسی و تبدیل کنید.

۳. f-string

💡 مفهوم

f-string راحت‌ترین و خواناترین روش برای ساختن متن از ترکیب متن ثابت و مقدار متغیرهاست. کافی است قبل از نقل‌قول حرف `f` بگذارید و هر **عبارت** پایتون را داخل `{ }` بنویسید؛ پایتون آن را حساب می‌کند و مقدارش را جایش می‌گذارد.

```
name = "Ali"
age = 25
print(f"Hello {name}! You are {age} years old.")
print(f"In 5 years you will be {age + 5}.")
print(f"Your name in capitals: {name.upper()}")
```

🔍 کد را بخوان

- `{name}` با مقدار متغیر جایگزین می‌شود.
- داخل `{ }` می‌توان **هر عبارتی** نوشت: `{age + 5}` اول حساب می‌شود (`30`) و بعد در متن قرار می‌گیرد.
- f-string برخلاف `+` لازم ندارد عدد را با `str()` تبدیل کنید.

💡 قالب‌بندی اعداد

بعد از عبارت داخل `{ }` می‌توانید با `:` یک **قالب (format spec)** بنویسید. پرکاربردترین‌ها در علوم داده:

قالب	معنی	مثال	خروجی
`.2f`	دو رقم اعشار	`f"{3.14159:.2f}"`	`3.14`

قالب	معنی	مثال	خروجی
`,`	جداکننده‌ی هزارگان	`f"{1250000:,}"`	`1,250,000`
`,.2f`	هر دو با هم	`f"{1234.5:,.2f}"`	`1,234.50`
`.1%`	درصد با یک رقم اعشار	`f"{0.256:.1%}"`	`25.6%`
`03d`	عدد صحیح با سه رقم، با صفرهای ابتدایی	`f"{7:03d}"`	`007`
`>8` / `<8` / `^8`	عرض ۸، راست‌چین / چپ‌چین / وسط‌چین	`f"[{'ab':>8}]"`	`[ ab]`

```
mean_income = 48753.6789
accuracy = 0.9134
n_samples = 1250000

print(f"Mean income: {mean_income:,.2f}")
print(f"Accuracy:    {accuracy:.1%}")
print(f"Samples:     {n_samples:,}")
```

🔍 کد را بخوان: ساختن یک جدول مرتب

با قالب‌های عرض و جهت می‌توان خروجی را به شکل جدول درآورد؛ کاری که هنگام گزارش نتایج تحلیل زیاد انجام می‌دهید:

```
print(f"{'Name':<8}|{'Score':>7}|{'Result':^8}")
print("-" * 25)
print(f"{'Ali':<8}|{17.5:>7.2f}|{'PASS':^8}")
print(f"{'Sara':<8}|{9.25:>7.2f}|{'FAIL':^8}")
print(f"{'Reza':<8}|{12:>7.2f}|{'PASS':^8}")
```

- `<8` : متن در ستونی به عرض ۸ کاراکتر، **چپ‌چین**.
- `>7.2f` : عدد با دو رقم اعشار، در ستونی به عرض ۷، **راست‌چین** (اعداد را معمولاً راست‌چین می‌کنند تا ممیزها زیر هم باشند).
- `^8` : وسط‌چین.
- `"-" * 25` یعنی ۲۵ بار تکرار `-` (جلسه‌ی ۲).

💡 ترفند اشکال‌زدایی: `{x=}`

اگر بعد از عبارت علامت `=` بگذارید، f-string هم **خود عبارت** و هم **مقدارش** را چاپ می‌کند. برای بررسی سریع مقدار متغیرها بسیار مفید است:

```
x = 7
y = 3
print(f"{x=}, {y=}, {x // y=}")
```

❓ پیش‌بینی کن

خروجی هر خط را دقیقاً بنویسید.

```
price = 1234567.891
print(f"{price:,.2f}")
print(f"{10 / 3:.3f}")
print(f"{0.05:.0%}")
print(f"[{42:>6}]")
print(f"{2 / 3:.0f}")
```

🔍 دقت کنید `.0f` یعنی **بدون رقم اعشار**؛ پس `0.666...` گرد می‌شود و `1` نمایش داده می‌شود.

⚠️ اشتباه رایج

- **فراموش کردن `f`:** `print("Hello {name}")` همان متن را با آکولادها چاپ می‌کند و خطا هم نمی‌دهد.
- **قالب‌بندی، مقدار را تغییر نمی‌دهد:** `f"{x:.2f}"` فقط یک **رشته** برای نمایش می‌سازد. خود `x` با همه‌ی رقم‌های اعشارش باقی می‌ماند. اگر واقعاً می‌خواهید عدد را گرد کنید، از `round()` استفاده کنید.

```
x = 3.14159
s = f"{x:.2f}"
print(s, type(s))
print(x)
```

- **نقل‌قول داخل f-string:** اگر f-string را با `"` شروع کرده‌اید، داخل `{ }` از `'` استفاده کنید؛ مثل `f"{'Name':<8}"` در مثال جدول.

۴. متد `format()` و روش‌های قدیمی‌تر

💡 مفهوم

f-string از پایتون ۳.۶ اضافه شده است. در کدهای قدیمی‌تر، کتاب‌ها و پاسخ‌های اینترنتی دو روش دیگر را زیاد می‌بینید. لازم نیست با آن‌ها کد بنویسید، اما باید بتوانید آن‌ها را **بخوانید**:

```python
name = "Zahra"
age = 30

# 1) the format() method: {} are filled in order
print("Hello {}! You are {} years old.".format(name, age))

# 2) the same, with names and a format spec
print("Hello {n}! Score: {s:.1f}".format(n=name, s=17.5))

# 3) the oldest style: % placeholders
print("Hello %s! You are %d years old." % (name, age))

# 4) the modern way
print(f"Hello {name}! You are {age} years old.")
```

🔍 کد را بخوان

روش	جای خالی	مقدارها کجا می‌آیند؟
`format()`	`{}` یا `{name}`	داخل `.format(...)` بعد از رشته
`%`	`%s` (رشته)، `%d` (عدد صحیح)، `%.1f` (اعشاری)	بعد از علامت `%` در انتهای رشته
f-string	`{expression}`	مستقیم داخل خود رشته

قالب‌هایی که برای f-string یاد گرفتید (مثل `:.2f` یا `:,`) در `format()` هم به همان شکل کار می‌کنند.

توصیه: در کد خودتان از f-string استفاده کنید؛ خواناتر است و احتمال اشتباه در ترتیب مقدارها کمتر است.

📌 آنچه باید بلد باشید

- خروجی دقیق `print` با `sep`، `end`، `\n` و `\t` را پیش‌بینی کنید.
- تفاوت نمایش مقدار در نوت‌بوک (`'Hello'`) و چاپ آن (`Hello`) را بدانید.
- `print` مقدار `None` برمی‌گرداند؛ چاپ کردن با برگرداندن یکی نیست.
- `input` همیشه `str` برمی‌گرداند؛ نتیجه‌ی `input() + input()` را پیش‌بینی کنید و آن را با تبدیل نوع درست کنید.
- عبارت داخل `{ }` در f-string حساب می‌شود؛ قالب‌های `.2f`، `,`، `%`، `03d` و عرض/جهت (`<`، `>`، `^`) را بخوانید و بنویسید.
- قالب‌بندی فقط نمایش را تغییر می‌دهد، نه مقدار متغیر را.
- کدی را که با `format()` یا `%` نوشته شده بخوانید و به f-string تبدیل کنید.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی خروجی

خروجی کد زیر را **دقیقاً** (با رعایت خط‌ها و فاصله‌ها) بنویسید.

```
print("x", "y", "z", sep="")
print("1", "2", end=" ")
print("3")
print("a\tb")
print("Data", "Science", sep="\n")
```

▼ پاسخ

```
xyz
1 2 3
a	b
Data
Science
```

- `sep=""` یعنی بدون فاصله.
- `end=" "` به جای خط جدید یک فاصله می‌گذارد؛ پس `3` در همان خط می‌آید.
- `\t` یک فاصله‌ی جدول است.
- `sep="\n"` بین دو مقدار خط جدید می‌گذارد.

سؤال ۲ — توضیح کد

خروجی کد زیر چیست؟ توضیح دهید چرا خط دوم این مقدار را چاپ می‌کند.

```
x = print("Python")
print(x)
```

▼ پاسخ

```
Python
None
```

`print` متن را نمایش می‌دهد، اما **مقداری برنمی‌گرداند** (حاصلش `None` است). پس `x` به `None` وصل می‌شود.

سؤال ۳ — پیدا کردن خطا

برنامه‌ی زیر قرار است میانگین دو نمره را حساب کند. کاربر `18` و `15` را وارد کرده، اما برنامه خطا می‌دهد. نوع خطا چیست؟ کد را درست کنید. اگر در خط آخر به جای `/ 2` نوشته بودیم `print(a + b)` ، چه چیزی چاپ می‌شد؟

```
a = input("Score 1: ")
b = input("Score 2: ")
average = (a + b) / 2
print(average)
```

▼ پاسخ

`a + b` دو رشته را به هم می‌چسباند (`"1815"`) و تقسیم رشته بر عدد ممکن نیست؛ خطا از نوع `TypeError` است.

اصلاح:

```
a = float(input("Score 1: "))
b = float(input("Score 2: "))
average = (a + b) / 2
print(average)
```

با `print(a + b)` (در کد اولیه) خروجی `1815` می‌شد.

سؤال ۴ — قالب‌بندی

خروجی هر خط را بنویسید.

کد	خروجی؟
`f"{1234567.891:,.2f}"`	
`f"{0.4567:.1%}"`	
`f"{7:03d}"`	
`f"[{'ab':>5}]"`	
`f"[{'ab':<5}]"`	
`f"{10 / 4:.1f}"`	
`f"{3 * 4 = }"`	

▼ پاسخ

کد	خروجی
`f"{1234567.891:,.2f}"`	`1,234,567.89`
`f"{0.4567:.1%}"`	`45.7%`
`f"{7:03d}"`	`007`
`f"[{'ab':>5}]"`	`[   ab]`
`f"[{'ab':<5}]"`	`[ab   ]`

کد	خروجی
`f"{10 / 4:.1f}"`	`2.5`
`f"{3 * 4 = }"`	`3 * 4 = 12`

در مورد آخر، فاصله‌های اطراف `=` هم عیناً در خروجی می‌آیند.

سؤال ۵ — مفهومی

دانشجویی برای گرد کردن میانگین نوشته است:

```
mean = 16.4871
mean_text = f"{mean:.1f}"
total = mean_text * 2
```

الف) نوع `mean_text` چیست و مقدار `mean` بعد از خط دوم چقدر است؟

ب) مقدار `total` چیست؟ دانشجو انتظار `33.0` را داشت؛ کد را طوری اصلاح کنید که این نتیجه به دست آید.

▼ پاسخ

الف) `mean_text` یک **رشته** است: `"16.5"` . مقدار `mean` تغییر نکرده و همان `16.4871` است.

ب) `"16.5" * 2` یعنی تکرار رشته: `"16.516.5"` . برای گرد کردن **عدد** باید از `round` استفاده کرد:

```
mean_rounded = round(mean, 1)   # 16.5 (float)
total = mean_rounded * 2        # 33.0
```

سؤال ۶ — خواندن کد قدیمی

دو خط زیر را به f-string تبدیل کنید و خروجی هر کدام را بنویسید.

```
name, score = "Sara", 17.5
print("%s scored %.1f points" % (name, score))
print("{} has {:,} followers".format(name, 15300))
```

▼ پاسخ

```
print(f"{name} scored {score:.1f} points")
print(f"{name} has {15300:,} followers")
```

خروجی:

```
Sara scored 17.5 points
Sara has 15,300 followers
```

سؤال ۷ — تکمیل کد

جای خالی را با یک f-string پر کنید تا خروجی **دقیقاً** به شکل زیر باشد (نام در ستونی به عرض ۶ و چپ‌چین؛ درصد با یک رقم اعشار):

```
Reza  : 87.5%
```

```
name = "Reza"
ratio = 0.875
print(______)
```

▼ پاسخ

```
print(f"{name:<6}: {ratio:.1%}")
```

`"Reza"` چهار حرف دارد و با `<6` دو فاصله بعد از آن اضافه می‌شود.

خلاصه‌ی جلسه

- `print` با `sep` و `end` کنترل می‌شود؛ `\n` و `\t` کاراکترهای ویژه‌اند؛ مقدار `print` مقدار `None` برمی‌گرداند.
- نوت‌بوک مقدار آخرین عبارت را **نمایش** می‌دهد (رشته‌ها با نقل‌قول)؛ `print` آن را **چاپ** می‌کند.
- `input` همیشه رشته برمی‌گرداند؛ برای عدد، تبدیل نوع لازم است.
- f-string: `f"... {expression:format} ..."`؛ قالب‌های `.2f` ، `,` ، `%` ، `03d` ، `<` ، `>` ، `^` و ترفند `{x=}` .
- قالب‌بندی فقط نمایش است؛ برای گرد کردن مقدار از `round` استفاده کنید.
- `format()` و `%` روش‌های قدیمی‌ترند؛ آن‌ها را بخوانید، ولی با f-string بنویسید.

جلسه‌ی بعد: شرط‌ها (`if` ، `elif` ، `else`).

جلسه ۵: شرط‌ها

Conditionals

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- ساختار `if` را بنویسید و نقش **دونقطه** و **تورفتگی (indentation)** را توضیح دهید،
- برای یک زنجیره‌ی `if` / `elif` / `else` مشخص کنید **کدام شاخه** اجرا می‌شود و چرا **ترتیب شرط‌ها** مهم است،
- تفاوت چند `if` مستقل با یک زنجیره‌ی `elif` را تشخیص دهید،
- شرط‌های تودرتو را بخوانید و در صورت امکان با `and` / `or` ساده کنید،
- عبارت شرطی یک‌خطی (`... if ... else ...`) را بخوانید و بنویسید،
- رفتار `if` با مقدارهای غیربولی (عدد، رشته‌ی خالی، `None`) را پیش‌بینی کنید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. دستور `if`
2. `else` و `elif`
3. شرط‌های تودرتو
4. عبارت شرطی یک‌خطی

۱. دستور `if`

💡 مفهوم

تا اینجا همه‌ی برنامه‌ها خط به خط، از بالا به پایین اجرا می‌شدند. **کنترل جریان (control flow)** یعنی برنامه بتواند **تصمیم بگیرد** کدام خط‌ها اجرا شوند. ساده‌ترین ابزار آن `if` است:

```
if condition:
    statement_1
    statement_2
next_statement
```

- بعد از `if` یک **شرط** می‌آید (معمولاً یک مقایسه از جلسه‌ی ۳) و بعد از آن **دونقطه** (`:`).
- خط‌هایی که **تورفتگی** دارند (معمولاً ۴ فاصله) **بدنه‌ی** `if` هستند و فقط وقتی شرط درست باشد اجرا می‌شوند.
- اولین خط **بدون تورفتگی** (`next_statement`) دیگر جزو `if` نیست و در هر حالت اجرا می‌شود.

در پایتون، تورفتگی **بخشی از دستور زبان** است، نه فقط برای زیبایی. پایتون از روی تورفتگی می‌فهمد یک **بلوک** کد کجا شروع و کجا تمام می‌شود.

```
temperature = 38.2

if temperature > 37.5:
    print("Fever detected")
    print("Please see a doctor")
print("Check finished")
```

🔍 کد را بخوان

- شرط `temperature > 37.5` برابر `True` است، پس **هر دو** خط تورفته اجرا می‌شوند.
- `print("Check finished")` تورفتگی ندارد؛ پس بیرون از `if` است و همیشه اجرا می‌شود.

مقدار `temperature` را به `36.6` تغییر دهید و دوباره اجرا کنید. کدام خط‌ها چاپ می‌شوند؟

❓ پیش‌بینی کن

تنها فرق دو سلول زیر **تورفتگی خط آخر** است. خروجی هر کدام چیست؟

```
x = 3
if x > 5:
    print("big")
    print("done")
```

```
x = 3
if x > 5:
    print("big")
print("done")
```

🔍 در سلول اول، `print("done")` جزو بدنه‌ی `if` است و چون شرط نادرست است، **هیچ چیزی** چاپ نمی‌شود. در سلول دوم، `done` بیرون از `if` است و چاپ می‌شود. یک تورفتگی، رفتار برنامه را عوض کرد!

💡 شرط‌هایی که بولی نیستند

بعد از `if` لازم نیست حتماً یک مقایسه بیاید. پایتون هر مقداری را مثل `bool()` (جلسه‌ی ۲) به درست یا نادرست تبدیل می‌کند: **صفر، رشته‌ی خالی و `None`** نادرست‌اند و بقیه درست.

❓ پیش‌بینی کن

کدام خط‌ها چاپ می‌شوند؟

```
name = ""
count = 0
city = "0"

if name:
    print("name is set")
if count:
    print("count is non-zero")
if city:
    print("city is set")
```

🔍 فقط `city is set` چاپ می‌شود: `""` و `0` نادرست‌اند، اما `"0"` یک رشته‌ی **غیرخالی** است و درست حساب می‌شود.

⚠️ اشتباه رایج

- **فراموش کردن دونقطه** یا **استفاده از `=` به جای `==`** در شرط: هر دو `SyntaxError` می‌دهند.
- **تورفتگی ناهماهنگ:** همه‌ی خط‌های یک بلوک باید دقیقاً یک اندازه تورفتگی داشته باشند. سلول زیر را اجرا کنید و پیام خطا را بخوانید:

```
score = 18
if score >= 17:
    print("Excellent")
      print("Well done")
```

🔍 خطای `IndentationError: unexpected indent` یعنی خط دوم بدنه، تورفتگی بیشتری از خط قبلی دارد. **همیشه از ۴ فاصله استفاده کنید** (VS Code با زدن کلید Tab به‌طور خودکار ۴ فاصله می‌گذارد).

۲. `else` و `elif`

💡 مفهوم

- **`else`** («در غیر این صورت») بلوکی است که وقتی شرط `if` **نادرست** باشد اجرا می‌شود. `else` شرط ندارد.

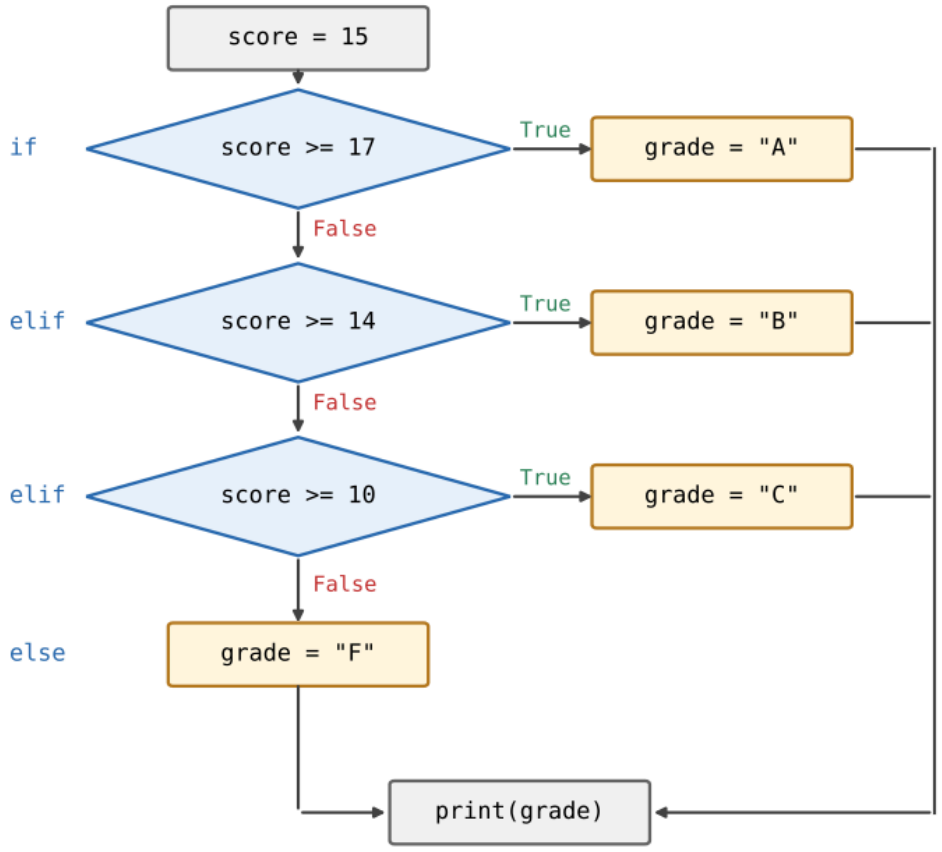
- `elif` (مخفف «else if») یک شرط **دیگر** را بررسی می‌کند؛ فقط اگر همه‌ی شرط‌های قبلی نادرست بوده باشند.

در یک زنجیره‌ی `if` / `elif` / ... / `else` ، پایتون شرط‌ها را **از بالا به پایین** بررسی می‌کند و **فقط اولین** شاخه‌ای را که شرطش درست باشد اجرا می‌کند؛ بقیه‌ی شرط‌ها اصلاً بررسی نمی‌شوند. پس **دقیقاً یک** شاخه اجرا می‌شود (اگر `else` وجود داشته باشد).

```
x = 15
if x > 20:
    print("x is greater than 20")
elif x == 15:
    print("x is exactly 15")
else:
    print("x is less than 15")
```

🔍 کد را بخوان: تبدیل نمره به رتبه

نمودار زیر مسیر اجرای یک زنجیره‌ی `elif` را برای `score = 15` نشان می‌دهد. کد متناظر آن پایین آمده است. مسیر را با انگشت دنبال کنید:



زنجیره‌ی `if` / `elif` / `else` : شرط‌ها از بالا بررسی می‌شوند و اولین شرط درست، مسیر را تعیین می‌کند.

```python
score = 15

if score >= 17:
    grade = "A"
elif score >= 14:
    grade = "B"
elif score >= 10:
    grade = "C"
else:
    grade = "F"

print(grade)
```

- `15 >= 17` نادرست است، پس می‌رویم سراغ `elif` بعدی.
- `15 >= 14` درست است؛ `grade = "B"` اجرا می‌شود و **بقیه‌ی زنجیره نادیده گرفته می‌شود**؛ با اینکه `15 >= 10` هم درست است.
- دقت کنید در شاخه‌ی دوم لازم نیست بنویسیم `14 <= score < 17`؛ چون اگر به این شاخه رسیده‌ایم، **حتماً** شرط قبلی (`score >= 17`) نادرست بوده است.

⚠️ اشتباه رایج: ترتیب نادرست شرط‌ها

اگر شرط **عمومی‌تر** را بالاتر بگذارید، شرط‌های خاص‌تر هیچ‌وقت به نوبت نمی‌رسند:

```python
score = 19

if score >= 10:
    grade = "C"
elif score >= 14:
    grade = "B"
elif score >= 17:
    grade = "A"
else:
    grade = "F"

print(grade)
```

🔍 نمره‌ی `19` رتبه‌ی `C` گرفت! چون `19 >= 10` درست است و زنجیره همان‌جا تمام می‌شود. در زنجیره‌ی `elif` همیشه از **سخت‌گیرانه‌ترین** شرط شروع کنید.

❓ پیش‌بینی کن: چند `if` یا یک زنجیره؟

دو سلول زیر فقط در کلمه‌ی `elif` فرق دارند. خروجی هر کدام چیست؟

```python
x = 15
if x > 10:
    print("A")
if x > 5:
    print("B")
if x > 0:
    print("C")
```

```python
x = 15
if x > 10:
    print("A")
elif x > 5:
    print("B")
elif x > 0:
    print("C")
```

🔍 در سلول اول سه `if` **مستقل** داریم و هر کدام جداگانه بررسی می‌شود؛ هر سه درست‌اند و `A` ، `B` و `C` چاپ می‌شوند. در سلول دوم یک **زنجیره** داریم و فقط اولین شاخه‌ی درست اجرا می‌شود: فقط `A` .

قاعده: اگر حالت‌ها **ناسازگار** هستند (فقط یکی از آن‌ها باید رخ دهد)، از `elif` استفاده کنید. اگر هر شرط باید **جداگانه** بررسی شود، از `if` های مستقل.

۳. شرط‌های تودرتو

💡 مفهوم

بدنه‌ی یک `if` می‌تواند خودش یک `if` دیگر داشته باشد. به این حالت **شرط تودرتو (nested)** می‌گویند. هر سطح، یک تورفتگی بیشتر دارد.

```python
age = 22
has_id = False

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Please show your ID")
else:
    print("Entry not allowed")
```

🔍 کد را بخوان

تورفتگی نشان می‌دهد هر `else` مال کدام `if` است:

- `else` اول (با ۴ فاصله) مال `if has_id` است.
- `else` دوم (بدون تورفتگی) مال `if age >= 18` است.

برای `age = 22` و `has_id = False` : شرط بیرونی درست است، وارد آن می‌شویم؛ شرط درونی نادرست است، پس `Please show your ID` چاپ می‌شود.

💡 ساده کردن با `and`

وقتی شاخه‌ی `else` درونی لازم نیست، دو `if` تودرتو را می‌توان با `and` یکی کرد. این دو کد **هم‌ارز** هستند:

```
# nested
if age >= 18:
    if has_id:
        print("Entry allowed")

# flat
if age >= 18 and has_id:
    print("Entry allowed")
```

کد **کم‌عمق‌تر** معمولاً خواناتر است. سعی کنید بیشتر از دو سطح تودرتو ننویسید.

❓ پیش‌بینی کن

برای هر یک از چهار حالت زیر، خروجی کد را بنویسید. بعد مقدارها را در سلول تغییر دهید و جواب‌هایتان را بررسی کنید.

is_member	total	خروجی؟
True	120	
True	80	
False	120	
False	80	

```
is_member = True
total = 80

if is_member:
    if total >= 100:
        discount = 0.20
    else:
        discount = 0.10
else:
    if total >= 100:
        discount = 0.05
    else:
        discount = 0.0

print(f"discount = {discount:.0%}")
```

▼ پاسخ جدول

is_member	total	خروجی
True	120	discount = 20%
True	80	discount = 10%
False	120	discount = 5%
False	80	discount = 0%

نوشتن جدول برای همه‌ی حالت‌ها بهترین راه فهمیدن (و آزمودن) یک کد شرطی است. در امتحان هم زیاد از شما خواسته می‌شود.

۴. عبارت شرطی یک‌خطی

💡 مفهوم

وقتی می‌خواهید بر اساس یک شرط، **یکی از دو مقدار** را انتخاب کنید، پایتون شکل کوتاه‌تری دارد (به آن **عبارت شرطی** یا **ternary** هم می‌گویند):

```
value = A if condition else B
```

یعنی: «اگر `condition` درست است، مقدار `A`؛ وگرنه مقدار `B`». این دو کد هم‌ارزند:

```
x = 7

# full form
if x % 2 == 0:
    y = x ** 2
else:
    y = x ** 3
print(y)

# one-line form
y = x ** 2 if x % 2 == 0 else x ** 3
print(y)
```

🔍 کد را بخوان

عبارت یک‌خطی را این‌طور بخوانید: «`y` برابر است با `x ** 2`، **اگر** `x` زوج باشد، **وگرنه** `x ** 3`». چون `7` فرد است، هر دو روش `343` می‌دهند.

یک مثال دیگر از برچسب زدن به داده، که در علوم داده بسیار رایج است:

```
age = 27
group = "young" if age < 30 else "old"
print("You are", group)
```

> **نکته برای آینده:** در NumPy و Pandas، برچسب زدن به **همه‌ی ردیف‌های یک ستون** با یک شرط، با تابعی به نام `np.where` انجام می‌شود که دقیقاً همین منطق را دارد: `np.where(condition, A, B)`.

⚠️ اشتباه رایج

- عبارت یک‌خطی **همیشه** به `else` نیاز دارد؛ `y = 1 if x > 0` خطای `SyntaxError` می‌دهد.
- برای منطق پیچیده (بیش از دو حالت) از عبارت یک‌خطی استفاده نکنید؛ `if` / `elif` معمولی خواناتر است.

⚠️ درباره‌ی `eval(input(...))`

در بعضی کتاب‌ها و کدهای قدیمی برای گرفتن عدد از کاربر این را می‌بینید:

```
x = eval(input("Enter x: "))
```

`eval` **هر کد پایتونی** را که کاربر تایپ کند اجرا می‌کند؛ پس کاربر می‌تواند به جای عدد، دستوری خطرناک وارد کند. **از `eval` استفاده نکنید**؛ به جای آن `int(input(...))` یا `float(input(...))` بنویسید (جلسه‌ی ۴).

📌 آنچه باید بلد باشید

- ساختار `if` / `elif` / `else`، نقش دونقطه، و اینکه تورفتگی مرز بلوک‌ها را تعیین می‌کند.
- برای یک مقدار داده‌شده، مشخص کنید کدام شاخه‌ی یک زنجیره اجرا می‌شود؛ در زنجیره فقط **اولین** شرط درست اجرا می‌شود.
- اهمیت **ترتیب** شرط‌ها در زنجیره‌ی `elif` (از سخت‌گیرانه‌ترین به عمومی‌ترین).
- تفاوت خروجی چند `if` مستقل و یک زنجیره‌ی `elif`.
- خواندن شرط‌های تودرتو، تشخیص اینکه هر `else` مال کدام `if` است، و ساده کردن آن‌ها با `and`.
- کامل کردن جدول خروجی برای همه‌ی حالت‌های ورودی.
- عبارت شرطی یک‌خطی `A if condition else B`.
- `""`، `0` و `None` در شرط نادرست‌اند؛ `"0"` درست است.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی خروجی

خروجی کد زیر را برای `n = 12` و برای `n = 7` بنویسید.

```
if n % 2 == 0:
    print("even")
if n % 3 == 0:
    print("divisible by 3")
elif n > 10:
    print("big")
else:
    print("other")
print("end")
```

▼ پاسخ

برای `n = 12`:

```
even
divisible by 3
end
```

برای `n = 7`:

```
other
end
```

نکته‌ی کلیدی: `if` اول **مستقل** است. زنجیره‌ی دوم از `if n % 3 == 0` شروع می‌شود. برای `12` شرط اول زنجیره درست است و `elif n > 10` (با اینکه درست است) بررسی نمی‌شود.

سؤال ۲ — جدول حالت‌ها

خروجی کد زیر را برای هر ردیف جدول بنویسید.

```
if temp > 30:
    if humid:
        print("muggy")
    else:
        print("hot")
elif temp < 10:
    print("cold")
else:
    print("mild")
```

temp	humid	خروجی؟
35	True	
35	False	

temp	humid	خروجی؟
5	True	
20	True	
30	False	

▼ پاسخ

temp	humid		خروجی
35	True	muggy	
35	False	hot	
5	True	cold	
20	True	mild	
30	False	mild	(چون `30 > 30` نادرست است)

سؤال ۳ – پیدا کردن خطا

کد زیر قرار است بر اساس شاخص توده‌ی بدنی (BMI) دسته‌بندی کند، اما برای `bmi = 32` خروجی `normal` می‌دهد (انتظار: `obese`). علت را توضیح دهید و کد را درست کنید.

```
if bmi >= 18.5:
    category = "normal"
elif bmi >= 25:
    category = "overweight"
elif bmi >= 30:
    category = "obese"
else:
    category = "underweight"
```

▼ پاسخ

چون `32 >= 18.5` درست است، زنجیره در همان شاخه‌ی اول تمام می‌شود و شرط‌های بعدی هرگز بررسی نمی‌شوند. مشکل، **ترتیب شرط‌ها** است: شرط عمومی‌تر بالاتر آمده است. با این کد، هیچ مقداری هرگز `overweight` یا `obese` نمی‌گیرد.

اصلاح (از سخت‌گیرانه‌ترین شرط):

```
if bmi >= 30:
    category = "obese"
elif bmi >= 25:
    category = "overweight"
elif bmi >= 18.5:
```

```
    category = "normal"
else:
    category = "underweight"
```

سؤال ۴ — هم‌ارزی

آیا کد **الف** و کد **ب** برای همه‌ی مقدارهای `x` (عدد) خروجی یکسانی دارند؟ اگر نه، یک مقدار برای `x` مثال بزنید که خروجی‌ها فرق کنند.

```
# A
if x > 0:
    if x < 10:
        print("small positive")
    else:
        print("other")

# B
if x > 0 and x < 10:
    print("small positive")
else:
    print("other")
```

▼ **پاسخ**

خیر. در کد الف، برای `x <= 0` هیچ چیزی چاپ نمی‌شود (چون `else` درونی فقط برای `x > 0` اجرا می‌شود). در کد ب، برای `x <= 0` عبارت `other` چاپ می‌شود. مثال: `x = -5`. برای `x` بین ۰ تا ۱۰ و برای `x >= 10` خروجی دو کد یکسان است.

سؤال ۵ — عبارت شرطی یک‌خطی

الف) مقدار `result` بعد از اجرای هر خط چیست؟

```
a, b = 4, 9
result = a if a > b else b
result = "pass" if result >= 10 else "fail"
```

ب) خط اول این کد در واقع چه چیزی را حساب می‌کند؟

ج) کد زیر را به یک خط تبدیل کنید:

```
if n >= 0:
    sign = "+"
else:
    sign = "-"
```

▼ **پاسخ**

الف) بعد از خط دوم `result` برابر `9` است و بعد از خط سوم برابر `"fail"`.

ب) **بزرگ‌ترین** مقدار بین `a` و `b` (همان کاری که تابع `max(a, b)` انجام می‌دهد).

ج) `sign = "+" if n >= 0 else "-"`

سؤال ۶ — پیدا کردن خطاهای نحوی

کد زیر چند خطای **نحوی (syntax)** دارد. همه را پیدا کنید و نسخه‌ی درست را بنویسید.

```
x = 10
if x = 10
    print("ten")
else if x > 10:
    print("more")
else:
print("less")
```

▼ پاسخ

1. `if x = 10` باید `if x == 10` باشد (مقایسه، نه انتساب).
2. بعد از شرط `if` دونقطه (`:`) نیامده است.
3. `else if` در پایتون وجود ندارد؛ باید `elif` باشد.
4. خط آخر تورفتگی ندارد؛ بدنه‌ی `else` باید تورفتگی داشته باشد.

```
x = 10
if x == 10:
    print("ten")
elif x > 10:
    print("more")
else:
    print("less")
```

سؤال ۷ — تکمیل کد

یک فروشگاه هزینه‌ی ارسال را این‌طور حساب می‌کند: سفارش‌های **۵۰۰ هزار تومان یا بیشتر** ارسال رایگان دارند؛ برای بقیه، اگر مقصد **همان شهر** باشد هزینه‌ی ارسال ۳۰ هزار تومان و در غیر این صورت ۶۰ هزار تومان است. جاهای خالی را کامل کنید.

```
total = 420        # thousand tomans
same_city = False

if ______:
    shipping = 0
elif ______:
    shipping = 30
else:
```

```
    shipping = _______

print(f"Shipping: {shipping}")
```

▼ پاسخ

```
if total >= 500:
    shipping = 0
elif same_city:
    shipping = 30
else:
    shipping = 60
```

برای `total = 420` و `same_city = False` خروجی `Shipping: 60` است. دقت کنید `elif` کافی است؛ لازم نیست بنویسید `same_city == True` .

خلاصه‌ی جلسه

- `if` بر اساس یک شرط تصمیم می‌گیرد؛ دونقطه و تورفتگی (۴ فاصله) بلوک را مشخص می‌کنند.
- در زنجیره‌ی `if` / `elif` / `else` فقط **اولین** شاخه‌ی درست اجرا می‌شود؛ پس ترتیب شرط‌ها مهم است.
- چند `if` مستقل ممکن است چند شاخه را اجرا کنند؛ زنجیره‌ی `elif` حداکثر یکی.
- شرط‌های تودرتو را با تورفتگی بخوانید و در صورت امکان با `and` ساده کنید؛ برای فهم کد، جدول همه‌ی حالت‌ها را بنویسید.
- `A if condition else B` برای انتخاب بین دو مقدار است.
- از `eval(input())` استفاده نکنید.

جلسه‌ی بعد: حلقه‌ها (`for` ، `while` ، `range` ، `break` و `continue`).

جلسه ۶: حلقه‌ها

Loops

بخش ۱: مبانی زبان پایتون – دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- با حلقه‌ی `for` روی حروف یک رشته یا روی اعداد یک `range` تکرار کنید،
- اعدادی را که یک `range` تولید می‌کند (با `start` ، `stop` و `step`) دقیقاً پیش‌بینی کنید،
- الگوی **جمع‌کننده (accumulator)** را بشناسید: جمع، شمارش و ضرب در حلقه،
- با حلقه‌ی `while` تکرار وابسته به شرط بنویسید و **حلقه‌ی بی‌پایان** را تشخیص دهید،
- اثر `break` و `continue` را روی خروجی پیش‌بینی کنید،
- برای یک حلقه **جدول دنبال کردن** بنویسید و تعداد اجرای حلقه‌های تودرتو را بشمارید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. حلقه‌ی `for`
2. تابع `range()`
3. حلقه‌ی `while`
4. `break` و `continue`
5. حلقه‌های تودرتو

۱. حلقه‌ی `for`

💡 مفهوم

حلقه (loop) یعنی اجرای چندباره‌ی یک بلوک کد. حلقه‌ی `for` روی **اعضای یک دنباله** حرکت می‌کند و بدنه را **یک بار برای هر عضو** اجرا می‌کند:

```
for item in sequence:
    body
```

- در هر دور، `item` به عضو بعدی دنباله وصل می‌شود (به آن **متغیر حلقه** می‌گوییم).
- بدنه مثل `if` با دونقطه و تورفتگی مشخص می‌شود.
- وقتی اعضا تمام شدند، اجرا از اولین خط بعد از حلقه ادامه پیدا می‌کند.

یک رشته دنباله‌ای از کاراکترهاست؛ پس می‌توان روی آن حلقه زد:

```
for ch in "Data":
    print(ch)
print("loop finished")
```

🔍 کد را بخوان

حلقه ۴ بار اجرا می‌شود؛ در هر دور `ch` یکی از حروف `"D"` ، `"a"` ، `"t"` و `"a"` است. `print("loop finished")` تورفتگی ندارد، پس **یک بار** و بعد از پایان حلقه اجرا می‌شود.

حلقه روی **لیست** هم کار می‌کند (لیست‌ها را در جلسه‌ی ۸ مفصل می‌خوانیم؛ فعلاً بدانید لیست مجموعه‌ای از مقدارها داخل `[ ]` است):

```
my_list = [1, 2, 3, 4, 5]
for i in my_list:
    print(i)
```

💡 الگوی جمع‌کننده (accumulator)

مهم‌ترین کاربرد حلقه در علوم داده، **خلاصه کردن** مجموعه‌ای از مقدارهاست: جمع، شمارش، میانگین، بیشینه و ... . الگوی آن همیشه سه قدم دارد:

1. **قبل از حلقه:** یک متغیر با مقدار اولیه بسازید (برای جمع `0`).
2. **داخل حلقه:** آن را با هر عضو به‌روز کنید (`total += x`).
3. **بعد از حلقه:** از نتیجه استفاده کنید.

```
scores = [17, 12.5, 19, 15]

total = 0
count = 0
for s in scores:
    total += s
    count += 1

print("sum =", total)
print("mean =", total / count)
```

🔍 کد را بخوان: جدول دنبال کردن

مثل جلسه‌ی ۳، مقدار متغیرها را **بعد از هر دور** حلقه بنویسید:

دور	s	total	count
قبل از حلقه	—	0	0
۱	17	17	1
۲	12.5	29.5	2
۳	19	48.5	3
۴	15	63.5	4

بعد از حلقه: میانگین `63.5 / 4` یعنی `15.875` ؛ همان عددی که در جلسه‌ی ۱ با `sum` و `len` به دست آوردیم.
(`sum` و `len` در واقع همین حلقه را برای شما اجرا می‌کنند.)

⚠️ اشتباه رایج: مقدار اولیه داخل حلقه

اگر `total = 0` را **داخل** حلقه بنویسید، در هر دور دوباره صفر می‌شود و در آخر فقط آخرین عضو را نگه می‌دارد. اگر `print` را داخل حلقه بگذارید، به جای یک نتیجه، در هر دور یک بار چاپ می‌شود. **همیشه بررسی کنید هر خط داخل حلقه است یا بیرون آن.**

۲. تابع `range()`

💡 مفهوم

برای تکرار روی **اعداد**، از `range` استفاده می‌کنیم. `range` دنباله‌ای از اعداد صحیح می‌سازد:

شکل	اعداد تولیدشده	مثال	نتیجه
`range(stop)`	از `0` تا `stop - 1`	`range(5)`	`0 1 2 3 4`
`range(start, stop)`	از `start` تا `stop - 1`	`range(2, 6)`	`2 3 4 5`
`range(start, stop, step)`	از `start` با گام `step`، قبل از رسیدن به `stop`	`range(0, 10, 2)`	`0 2 4 6 8`

قاعده‌ی طلایی: `stop` **هیچ‌وقت** جزو نتیجه نیست. `range(5)` پنج عدد دارد، اما عدد `5` جزو آن‌ها نیست.

```
for i in range(5):
    print("Iteration:", i)
```

```
for i in range(5, 0, -1):     # counting down
    print(i, end=" ")
print("Go!")
```

🔍 کد را بخوان

- در سلول اول، `i` از `0` تا `4` تغییر می‌کند: ۵ دور.
- در سلول دوم گام **منفی** است؛ پس شمارش رو به پایین است و قبل از رسیدن به `0` متوقف می‌شود: `5 4 3 2 1`.
- `end=" "` (جلسه‌ی ۴) باعث می‌شود اعداد در یک خط چاپ شوند.

❓ پیش‌بینی کن

جمع اعداد ۱ تا ۱۰۰ را با یک حلقه حساب می‌کنیم. **قبل از اجرا** دو سؤال را جواب دهید: چرا `stop` برابر `101` است؟ اگر `range(100)` می‌نوشتیم، جواب چقدر می‌شد؟

```
total = 0
for k in range(1, 101):
    total += k
print(total)
```

🔍 برای اینکه خود `100` هم جمع شود، `stop` باید `101` باشد. `range(100)` اعداد `0` تا `99` را می‌دهد و جمع آن‌ها `4950` است؛ یعنی `100` کمتر از جواب درست.

⚠️ اشتباه رایج: خطای «یکی کم یا زیاد» (off-by-one)

رایج‌ترین خطای حلقه‌ها همین است: یک دور کمتر یا بیشتر از مقدار لازم. هر وقت `range` می‌نویسید، از خودتان بپرسید: **اولین** عدد چیست؟ **آخرین** عدد چیست؟ **چند** عدد؟ (تعداد اعداد `range(a, b)` برابر `b - a` است.)

۳. حلقه‌ی `while`

💡 مفهوم

حلقه‌ی `while` تا وقتی **یک شرط درست باشد** تکرار می‌شود:

```
while condition:
    body
```

قبل از **هر** دور، شرط بررسی می‌شود. اگر درست بود بدنه اجرا می‌شود و دوباره سراغ شرط می‌رود؛ اگر نادرست بود حلقه تمام می‌شود.

```
i = 0
while i < 5:
    print("Iteration:", i)
    i += 1
```

🔍 کد را بخوان

این حلقه دقیقاً همان کار `for i in range(5)` را می‌کند، اما سه بخش آن جداگانه نوشته شده است: **مقدار اولیه** (`i = 0`)، **شرط** (`i < 5`) و **به‌روزرسانی** (`i += 1`). اگر هر کدام از این سه را اشتباه بنویسید، حلقه درست کار نمی‌کند.

💡 چه وقت `while`؟

`while` وقتی مناسب است که **از قبل نمی‌دانید** حلقه چند بار باید تکرار شود. مثال: پس‌اندازی با سود سالانه‌ی ۱۸٪ چند سال طول می‌کشد تا دو برابر شود؟

```
money = 1000
years = 0
while money < 2000:
    money *= 1.18
    years += 1

print(f"{years} years, final amount = {money:.0f}")
```

🔍 کد را بخوان: ارقام یک عدد

کد زیر از `%` و `//` (جلسه‌ی ۳) استفاده می‌کند تا ارقام یک عدد را یکی‌یکی جدا کند. اول جدول دنبال کردن را روی کاغذ کامل کنید، بعد اجرا کنید:

```python
n = 2026
digit_sum = 0
while n > 0:
    digit_sum += n % 10    # take the last digit
    n //= 10               # remove the last digit
print(digit_sum)
```

دور	`n % 10`	`digit_sum`	`n` (بعد از دور)
قبل از حلقه	—	0	2026
۱	6	6	202
۲	2	8	20
۳	0	8	2
۴	2	10	0

بعد از دور چهارم `n` صفر می‌شود، شرط `n > 0` نادرست است و حلقه تمام می‌شود. جواب: `10` .

⚠️ اشتباه رایج: حلقه‌ی بی‌پایان

اگر شرط `while` هیچ‌وقت نادرست نشود، حلقه **تا ابد** اجرا می‌شود. معمولاً علت این است که فراموش کرده‌اید متغیر شرط را داخل حلقه تغییر دهید:

```python
i = 0
while i < 5:
    print(i)        # i never changes: infinite loop!
```

اگر سلولی بی‌پایان اجرا شد (علامت `[*]` کنار آن ماند)، اجرای کرنل را متوقف کنید: در Jupyter دکمه‌ی ■ (Interrupt) و در VS Code دکمه‌ی مربعی کنار سلول.

مقایسه‌ی `for` و `while`

	`for`	`while`
تکرار بر اساس	اعضای یک دنباله	یک شرط
تعداد دورها	از قبل معلوم	ممکن است از قبل معلوم نباشد
خطر حلقه‌ی بی‌پایان	تقریباً ندارد	دارد
مثال	جمع همه‌ی نمره‌ها	تکرار تا رسیدن به یک هدف

۴. `break` و `continue`

💡 مفهوم

- `break` : حلقه را **همین الان** کاملاً متوقف می‌کند و اجرا از بعد از حلقه ادامه پیدا می‌کند.
- `continue` : **بقیه‌ی همین دور** را رد می‌کند و به **دور بعد** می‌رود.

هر دو معمولاً داخل یک `if` می‌آیند.

```
for i in range(10):
    if i == 5:
        break
    print("Value:", i)
```

```
for i in range(5):
    if i == 2:
        continue
    print("Number:", i)
```

🔍 کد را بخوان

- در سلول اول، وقتی `i` به `5` می‌رسد `break` اجرا می‌شود؛ پس فقط `0` تا `4` چاپ می‌شوند و دورهای `5` تا `9` اصلاً اجرا نمی‌شوند.
- در سلول دوم، وقتی `i` برابر `2` است `continue` اجرا می‌شود؛ `print` آن دور رد می‌شود ولی حلقه ادامه پیدا می‌کند: `0 1 3 4` .

💡 جست‌وجو با `break` : آیا عدد اول است؟

یک کاربرد رایج `break` ، **جست‌وجو** است: به محض پیدا شدن جواب، دیگر لازم نیست ادامه دهیم. کد زیر بررسی می‌کند که `n` عدد اول است یا نه:

```
n = 91
is_prime = True
for d in range(2, n):
    if n % d == 0:
        is_prime = False
        print(n, "is divisible by", d)
        break

print("prime?", is_prime)
```

🔍 متغیر `is_prime` یک **پرچم (flag)** است: فرض می‌کنیم عدد اول است و اگر مقسوم‌علیهی پیدا شد، پرچم را `False` می‌کنیم. برای `91` در دور `d = 7` مقسوم‌علیه پیدا می‌شود و `break` بقیه‌ی دورها (۸ تا ۹۰) را حذف

می‌کند. مقدار `n` را به `97` تغییر دهید و نتیجه را ببینید.

> **برای خواندن کد دیگران:** پایتون ساختاری به نام `for ... else` هم دارد: بلوک `else` حلقه فقط وقتی اجرا می‌شود که حلقه **بدون** `break` تمام شده باشد. با آن می‌توان همین برنامه را بدون پرچم نوشت. لازم نیست از آن استفاده کنید، ولی اگر در کدی دیدید، معنی‌اش را بدانید.

❓ پیش‌بینی کن

خروجی سلول زیر را دقیقاً بنویسید. (برای هر `i` بپرسید: اول کدام `if` بررسی می‌شود؟)

```
for i in range(10):
    if i % 3 == 0:
        continue
    if i > 7:
        break
    print(i, end=" ")
```

🔍 مضرب‌های ۳ (`0` ، `3` ، `6` ، `9`) با `continue` رد می‌شوند. برای `i = 8` شرط `i > 7` درست است و حلقه با `break` تمام می‌شود. خروجی: `1 2 4 5 7` .

⚠️ اشتباه رایج: `continue` در `while`

در حلقه‌ی `while` ، اگر `continue` **قبل از** به‌روزرسانی شمارنده بیاید، به‌روزرسانی رد می‌شود و حلقه بی‌پایان می‌شود:

```
i = 0
while i < 5:
    if i == 3:
        continue     # i stays 3 forever!
    print(i)
    i += 1
```

راه حل: شمارنده را **قبل از** `continue` به‌روز کنید (یا از `for` استفاده کنید).

۵. حلقه‌های تودرتو

💡 مفهوم

بدنه‌ی یک حلقه می‌تواند خودش یک حلقه باشد. حلقه‌ی **درونی** برای **هر** دور حلقه‌ی **بیرونی**، از اول تا آخر اجرا می‌شود.

```
for i in [1, 2]:
    for j in [10, 20, 30]:
        print(i, j)
```

🔍 کد را بخوان

حلقه‌ی بیرونی ۲ دور و حلقه‌ی درونی در هر دور ۳ دور دارد؛ پس `print` در مجموع **۲ × ۳ = ۶** بار اجرا می‌شود. ترتیب: اول `i = 1` با همه‌ی `j` ها، بعد `i = 2` با همه‌ی `j` ها.

یک مثال کاربردی‌تر: جدول ضرب. به نقش f-string و `end` (جلسه‌ی ۴) در مرتب کردن خروجی دقت کنید:

```
for row in range(1, 6):
    for col in range(1, 6):
        print(f"{row * col:4}", end="")
    print()          # new line after each row
```

- `{row * col:4}` هر عدد را در ستونی به عرض ۴ قرار می‌دهد (اعداد به‌طور پیش‌فرض راست‌چین‌اند).
- `print()` بعد از حلقه‌ی درونی (اما داخل حلقه‌ی بیرونی) در پایان هر سطر خط را عوض می‌کند. اگر تورفتگی آن را یک سطح بیشتر کنید، چه اتفاقی می‌افتد؟ امتحان کنید.

❓ پیش‌بینی کن

در کد زیر، حلقه‌ی درونی به متغیر حلقه‌ی **بیرونی** وابسته است. چه شکلی چاپ می‌شود؟ و `print` داخلی در مجموع چند بار اجرا می‌شود؟

```
for i in range(1, 5):
    for j in range(i):
        print("*", end="")
    print()
```

🔍 برای `i = 1` حلقه‌ی درونی ۱ بار اجرا می‌شود، برای `i = 2` دو بار و ... . پس یک مثلث چهارسطری چاپ می‌شود و `print("*")` در مجموع `1 + 2 + 3 + 4 = 10` بار اجرا می‌شود.

چرا شمردن تعداد دورها مهم است؟ در علوم داده با داده‌های بزرگ کار می‌کنید. دو حلقه‌ی تودرتو روی یک میلیون ردیف یعنی **یک میلیون میلیون** دور، که عملاً تمام نمی‌شود. یکی از دلایل اصلی استفاده از NumPy و Pandas (بخش ۳) همین است: آن‌ها حلقه‌ها را با سرعت بسیار بیشتری برای شما اجرا می‌کنند.

📌 آنچه باید بلد باشید

- ساختار `for` روی رشته، لیست و `range` ؛ اینکه کدام خط‌ها داخل حلقه‌اند و کدام بیرون.
- الگوی جمع‌کننده: مقدار اولیه **قبل از** حلقه، به‌روزرسانی **داخل** حلقه، استفاده **بعد از** حلقه.
- اعداد تولیدشده توسط `range(start, stop, step)` را دقیقاً بنویسید (`stop` جزو نتیجه نیست؛ گام منفی؛ `range` خالی).
- سه بخش حلقه‌ی `while` (مقدار اولیه، شرط، به‌روزرسانی) و علت حلقه‌ی بی‌پایان.

- برای یک حلقه‌ی `for` یا `while` جدول دنبال کردن بنویسید.
- اثر `break` و `continue` بر خروجی؛ خطای `continue` قبل از به‌روزرسانی در `while` .
- الگوی جست‌وجو با پرچم و `break` .
- تعداد اجرای بدنه در حلقه‌های تودرتو (حتی وقتی حلقه‌ی درونی به حلقه‌ی بیرونی وابسته است).

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ – `range`

اعدادی را که هر `range` تولید می‌کند بنویسید (اگر هیچ عددی تولید نمی‌کند، بنویسید «خالی»).

عبارت	اعداد؟
`range(3)`	
`range(1, 10, 3)`	
`range(10, 0, -3)`	
`range(5, 5)`	
`range(0, 5, -1)`	
`range(-2, 2)`	

▼ پاسخ

عبارت	اعداد
`range(3)`	`0 1 2`
`range(1, 10, 3)`	`1 4 7`
`range(10, 0, -3)`	`10 7 4 1`
`range(5, 5)`	خالی
`range(0, 5, -1)`	خالی (با گام منفی نمی‌توان از `0` به `5` رسید)
`range(-2, 2)`	`-2 -1 0 1`

سؤال ۲ – جدول دنبال کردن

جدول دنبال کردن کد زیر را کامل کنید و خروجی را بنویسید. این کد در کل چه کاری انجام می‌دهد؟

```
n = 1234
result = 0
while n > 0:
    result = result * 10 + n % 10
    n //= 10
print(result)
```

▼ پاسخ

دور	`n % 10`	`result`	`n` (بعد از دور)
قبل از حلقه	–	`0`	`1234`
۱	`4`	`4`	`123`
۲	`3`	`43`	`12`
۳	`2`	`432`	`1`
۴	`1`	`4321`	`0`

خروجی: `4321` . این کد ارقام عدد را **برعکس** می‌کند.

سؤال ۳ – `break` و `continue`

خروجی کد زیر را دقیقاً بنویسید.

```
total = 0
for k in range(1, 20):
    if k % 2 == 0:
        continue
    total += k
    if total > 20:
        break
print(k, total)
```

▼ پاسخ

9 25

فقط اعداد فرد جمع می‌شوند: `1` ، `4` (=1+3)، `9` ، `16` ، `25` . بعد از اضافه شدن `9` ، مقدار `total` برابر `25` و بیشتر از `20` است؛ پس حلقه در `k = 9` با `break` تمام می‌شود. دقت کنید متغیر حلقه (`k`) بعد از حلقه هم آخرین مقدارش را نگه می‌دارد.

سؤال ۴ – پیدا کردن خطا

هدف: چاپ اعداد ۰ تا ۵ **به‌جز ۳**. هر دو کد زیر اشتباه‌اند. مشکل هر کدام را توضیح دهید و درستش کنید.

```
# A
i = 0
```

```
while i <= 5:
    if i == 3:
        continue
    print(i)
    i += 1

# B
for i in range(5):
    if i == 3:
        break
    print(i)
```

▼ **پاسخ**

الف) وقتی `i` برابر `3` می‌شود، `continue` قبل از `i += 1` اجرا می‌شود؛ `i` هیچ‌وقت تغییر نمی‌کند و حلقه بی‌پایان می‌شود. اصلاح:

```
i = 0
while i <= 5:
    if i == 3:
        i += 1
        continue
    print(i)
    i += 1
```

ب) دو اشکال دارد: `break` حلقه را در `3` کاملاً متوقف می‌کند (باید `continue` باشد)، و `range(5)` عدد `5` را شامل نمی‌شود (باید `range(6)` باشد). اصلاح:

```
for i in range(6):
    if i == 3:
        continue
    print(i)
```

سؤال ۵ — شمارش دورها

بعد از اجرای کد زیر، مقدار `count` چیست؟

```
count = 0
for i in range(4):
    for j in range(i, 4):
        count += 1
print(count)
```

▼ **پاسخ**

برای `i = 0, 1, 2, 3` حلقه‌ی درونی به ترتیب `4، 3، 2، 1` بار اجرا می‌شود. پس `count = 4 + 3 + 2 + 1 = 10`.

سؤال ۶ — توضیح کد

کد زیر را بخوانید:

```
text = "data science is fun"
count = 0
for ch in text:
    if ch in "aeiou":
        count += 1
print(count)
```

الف) این کد در کل چه چیزی را حساب می‌کند؟ خروجی آن چیست؟

ب) اگر خط `count = 0` را به داخل حلقه (بالای `if`) منتقل کنیم، خروجی چه می‌شود؟ چرا؟

▼ پاسخ

الف) تعداد **حروف صدادار** انگلیسی متن را می‌شمارد. حروف صدادار: `a, a` (data)، `i, e, e` (science)، `i` (is)، `u` (fun) یعنی `7`.

ب) `count` در ابتدای هر دور صفر می‌شود و در پایان فقط مقدار مربوط به **آخرین حرف** را دارد. آخرین حرف `n` صدادار نیست، پس خروجی `0` است.

سؤال ۷ — تکمیل کد

جمعیت یک شهر `500` هزار نفر است و سالی `3٪` رشد می‌کند. جاهای خالی را کامل کنید تا برنامه چاپ کند **چند سال** طول می‌کشد جمعیت به **حداقل** `750` **هزار** نفر برسد.

```
population = 500
years = 0
while ______:
    population ______
    years += 1
print(years)
```

▼ پاسخ

```
while population < 750:
    population *= 1.03
    years += 1
```

خروجی `14` است. دقت کنید شرط باید `population < 750` باشد (نه `<=`)، چون وقتی جمعیت **به** ۷۵۰ رسید باید متوقف شویم.

خلاصه‌ی جلسه

- `for` روی اعضای یک دنباله (رشته، لیست، `range`) تکرار می‌کند؛ `while` تا وقتی شرط درست است.

- `range(start, stop, step)` : `stop` جزو نتیجه نیست؛ مراقب خطای یکی کم یا زیاد باشید.
- الگوی جمع‌کننده: مقدار اولیه قبل از حلقه، به‌روزرسانی داخل حلقه.
- در `while` حتماً متغیر شرط را تغییر دهید؛ وگرنه حلقه بی‌پایان می‌شود.
- `break` حلقه را تمام می‌کند؛ `continue` به دور بعد می‌رود.
- در حلقه‌های تودرتو، حلقه‌ی درونی برای هر دور بیرونی کامل اجرا می‌شود؛ تعداد دورها را بشمارید.

جلسه‌ی بعد: توابع (تعریف تابع، پارامتر، مقدار بازگشتی).

جلسه ۷: توابع

Functions

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- توضیح دهید چرا کد را در قالب **تابع** می‌نویسیم و یک تابع تعریف و فراخوانی کنید،
- تفاوت `print` و `return` را دقیقاً بدانید و مقداری را که یک فراخوانی برمی‌گرداند پیش‌بینی کنید،
- بگویید در هر فراخوانی، کدام **آرگومان** به کدام **پارامتر** می‌رسد (موقعیتی، نام‌دار، پیش‌فرض)،
- از تابع‌های آماده‌ی ماژول‌های `math` و `random` استفاده کنید و نقش `random.seed` را در **تکرارپذیری** نتایج توضیح دهید،
- تابع‌هایی با تعداد ورودی متغیر (`*args` و `**kwargs`) را بخوانید،
- **دامنه‌ی متغیرها** (محلی و سراسری) را بشناسید و خروجی کدهایی را که نام‌های یکسان دارند پیش‌بینی کنید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. چرا تابع؟ تعریف و فراخوانی
2. پارامتر و مقدار بازگشتی
3. توابع آماده: `math` و `random`
4. آرگومان‌های موقعیتی، نام‌دار و پیش‌فرض
5. `*args` و `**kwargs`
6. دامنه‌ی متغیرها

۱. چرا تابع؟ تعریف و فراخوانی

💡 مفهوم

فرض کنید می‌خواهید جمع اعداد ۱ تا ۲۵، بعد ۵۰ تا ۷۶ و بعد ۹۰ تا ۱۰۱ را حساب کنید. با چیزهایی که تا اینجا یاد گرفته‌اید، باید یک حلقه را سه بار با تغییرات کوچک تکرار کنید:

```
total = 0
for i in range(1, 26):
    total += i
print("Sum of integers from 1 to 25 is:", total)

total = 0
for i in range(50, 77):
    total += i
print("Sum of integers from 50 to 76 is:", total)

total = 0
for i in range(90, 102):
    total += i
print("Sum of integers from 90 to 101 is:", total)
```

این کد سه ایراد دارد: **طولانی** است، اگر بخواهید روش محاسبه را عوض کنید باید **سه جا** را درست کنید، و احتمال **اشتباه کپی** زیاد است. **تابع (function)** یک بلوک کد **نام‌دار** است که یک بار تعریف می‌شود و هر چند بار لازم باشد **فراخوانی (call)** می‌شود:

```
def range_sum(x, y):
    total = 0
    for i in range(x, y + 1):
        total += i
    return total

print("Sum of integers from 1 to 25 is:", range_sum(1, 25))
print("Sum of integers from 50 to 76 is:", range_sum(50, 76))
print("Sum of integers from 90 to 101 is:", range_sum(90, 101))
```

🔍 کد را بخوان

- `def` یعنی «تعریف تابع». بعد از آن **نام** تابع، داخل پرانتز **پارامترها** (`x` و `y`) و در آخر دونقطه می‌آید.
- بدنه با تورفتگی مشخص می‌شود (مثل `if` و `for`).
- `return total` نتیجه را به جایی که تابع فراخوانی شده **برمی‌گرداند**.
- `range_sum(1, 25)` یعنی «تابع را با `x = 1` و `y = 25` اجرا کن و حاصلش را بده».
- چون `range` عدد `stop` را شامل نمی‌شود (جلسه‌ی ۶)، داخل تابع `y + 1` نوشته‌ایم تا خود `y` هم جمع شود. این جزئیات حالا **فقط یک جا** نوشته شده است.

⚠️ اشتباه رایج

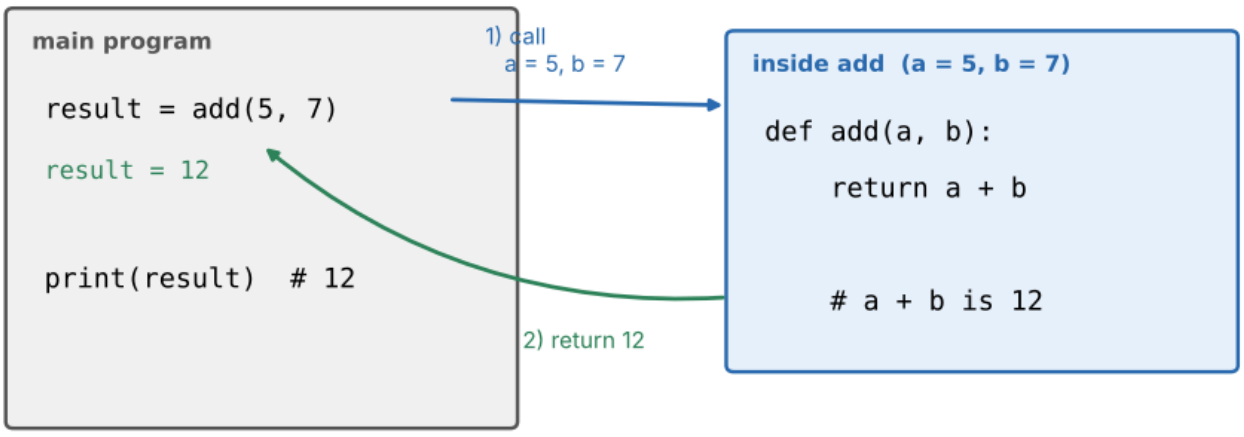
- **تعریف کردن، اجرا کردن نیست:** اجرای سلولی که فقط `def` دارد هیچ خروجی‌ای ندارد؛ تابع فقط وقتی اجرا می‌شود که **فراخوانی** شود.
- **فراخوانی قبل از تعریف:** پایتون کد را از بالا به پایین اجرا می‌کند؛ اگر تابعی را قبل از اجرای `def` آن صدا بزنید، `NameError` می‌گیرید. در نوت‌بوک یعنی: اول سلول تعریف را اجرا کنید.
- **فراموش کردن پرانتز:** `range_sum` (بدون پرانتز) خود تابع است، نه نتیجه‌ی اجرای آن.

۲. پارامتر و مقدار بازگشتی

💡 مفهوم

- **پارامتر (parameter):** نامی که در تعریف تابع داخل پرانتز می‌آید (`def add(a, b)`).
- **آرگومان (argument):** مقداری که هنگام فراخوانی به تابع داده می‌شود (`add(5, 7)`).
- **`return`** : اجرای تابع را **تمام می‌کند** و یک مقدار را به فراخوانی‌کننده برمی‌گرداند.

نمودار زیر اتفاقی را که هنگام اجرای `result = add(5, 7)` می‌افتد نشان می‌دهد:



۱) فراخوانی: آرگومان‌ها به پارامترها داده می‌شوند. ۲) `return` : مقدار حاصل به جای فراخوانی قرار می‌گیرد.

```
def add(a, b):
    return a + b

result = add(5, 7)
print("Sum is:", result)
print(add(2, 3) * 10)
```

🔍 یک فراخوانی مثل `add(2, 3)` را می‌توانید مثل یک **مقدار** در نظر بگیرید: پایتون اول تابع را اجرا می‌کند و بعد **مقدار برگشتی** (`5`) را به جای آن می‌گذارد. پس `add(2, 3) * 10` یعنی `5 * 10` .

💡 `print` یا `return`؟

این مهم‌ترین نکته‌ی این جلسه است. در جلسه‌ی ۴ دیدید که `print` فقط **نمایش** می‌دهد و حاصلش `None` است. تابعی که به جای `print` از `return` استفاده کند، چیزی **برنمی‌گرداند**:

❓ پیش‌بینی کن

قبل از اجرا بنویسید هر سلول چه خروجی‌ای دارد.

```
def double_print(x):
    print(x * 2)

z = double_print(3)
print("z =", z)
```

```
def double_return(x):
    return x * 2

z = double_return(3)
print("z =", z)
```

🔍 در سلول اول، `6` **داخل تابع** چاپ می‌شود، اما تابع `return` ندارد؛ پس مقدار برگشتی‌اش `None` است و `z =` `None` چاپ می‌شود. در سلول دوم، تابع چیزی چاپ نمی‌کند ولی `6` را **برمی‌گرداند**؛ پس `z = 6` .

قاعده: تابعی که **محاسبه** می‌کند باید نتیجه را `return` کند تا بتوان از آن در ادامه استفاده کرد (جمع کرد، مقایسه کرد، در جدول گذاشت). `print` را برای **گزارش به کاربر** نگه دارید.

💡 `return` اجرای تابع را تمام می‌کند

تابع می‌تواند چند `return` داشته باشد؛ **اولین** `return` که اجرا شود، تابع را تمام می‌کند و خط‌های بعدی اجرا نمی‌شوند:

```python
def grade(score):
    """Return the letter grade for a score between 0 and 20."""
    if score >= 17:
        return "A"
    if score >= 14:
        return "B"
    if score >= 10:
        return "C"
    return "F"

print(grade(18), grade(15), grade(11), grade(4))
```

🔍 کد را بخوان

- این همان منطق رتبه‌بندی جلسه‌ی ۵ است، اما حالا یک **ابزار قابل استفاده‌ی مجدد** شده است.
- چون هر `return` تابع را تمام می‌کند، به `elif` نیازی نیست: اگر `score >= 17` باشد، تابع همان‌جا `"A"` را برمی‌گرداند و به خط‌های بعدی نمی‌رسد.
- رشته‌ی سه‌نقل‌قولی زیر `def` یک **docstring** است: توضیح کوتاهی درباره‌ی کار تابع. با `help(grade)` می‌توانید آن را ببینید. عادت کنید برای تابع‌هایتان docstring بنویسید.

💡 برگرداندن چند مقدار

تابع می‌تواند چند مقدار را با کاما برگرداند. هنگام فراخوانی، می‌توان آن‌ها را مستقیم در چند متغیر گرفت (همان انتساب چندگانه‌ی جلسه‌ی ۲):

```python
import math

def circle(r):
    area = round(math.pi * r ** 2, 4)
    perimeter = round(2 * math.pi * r, 4)
    return area, perimeter

a, p = circle(3)
print(f"Area = {a}, Perimeter = {p}")

both = circle(3)
print(both, type(both))
```

🔍 `return area, perimeter` در واقع یک **تاپل** (جلسه‌ی ۹) برمی‌گرداند. اگر آن را در یک متغیر بگیرید (`both`)، کل تاپل را دارید؛ اگر در دو متغیر بگیرید، باز (unpack) می‌شود.

در این مثال از ثابت `math.pi` در ماژول `math` استفاده کردیم. این ماژول را در بخش بعد معرفی می‌کنیم.

⚠️ اشتباه رایج

- **`return` داخل حلقه:** اگر `return` را با تورفتگی اشتباه داخل حلقه بگذارید، تابع در **اولین دور** تمام می‌شود.
- **برگرداندن نوع‌های مختلف:** تابعی که گاهی عدد و گاهی پیام متنی برمی‌گرداند (مثلاً `"Both numbers are equal"`) استفاده از نتیجه را سخت می‌کند. سعی کنید هر تابع همیشه یک **نوع** را برگرداند.

💡 تابع هم یک مقدار است

نام یک تابع (بدون پرانتز) به خود تابع اشاره می‌کند؛ پس می‌توان تابع را مثل هر مقدار دیگری به یک تابع دیگر **داد**:

```
def square(x):
    return x * x

def apply_twice(f, x):
    return f(f(x))

print(apply_twice(square, 3))
```

🔍 `apply_twice` تابع `square` را به‌عنوان پارامتر `f` می‌گیرد و دو بار روی `3` اعمال می‌کند: `square(square(3))` یعنی `square(9)` که `81` است. دقت کنید هنگام دادن تابع **پرانتز نمی‌گذاریم**: `square` خود تابع است، ولی `square(3)` نتیجه‌ی اجرای آن (`9`).

این ایده در یادگیری ماشین همه‌جا هست: الگوریتم‌های آموزش، **تابع خطا** یا **تابع فعال‌سازی** را به‌عنوان ورودی می‌گیرند. در جلسه‌ی ۱۳ بیشتر به آن می‌پردازیم.

۳. توابع آماده: `math` و `random`

💡 مفهوم

لازم نیست همه‌ی تابع‌ها را خودتان بنویسید. پایتون دو منبع تابع آماده دارد:

- **تابع‌های داخلی (built-in):** همیشه در دسترس‌اند و `import` لازم ندارند. تا اینجا خیلی از آن‌ها را دیده‌اید: `print` ، `len` ، `type` ، `int` ، `round` ، `abs` ، `min` ، `max` ، `sum` .
- **کتابخانه‌ی استاندارد:** مجموعه‌ای از **ماژول**‌ها که همراه پایتون نصب می‌شوند. برای استفاده از هر ماژول، اول آن را با `import` وارد می‌کنید و بعد تابع‌هایش را با نام ماژول و یک نقطه صدا می‌زنید: `math.sqrt(16)` .

سازوکار کامل ماژول‌ها (و ساختن ماژول خودتان) را در جلسه‌ی ۱۷ می‌بینیم. در این جلسه با دو ماژول پرکاربرد آشنا می‌شوید. هر تابع آماده هم دقیقاً مثل تابع‌های خودتان است: **آرگومان** می‌گیرد، کاری انجام می‌دهد و **مقدار برمی‌گرداند**.

ماژول `math`

نام	کار	مثال	نتیجه
`math.sqrt(x)`	جذر	`math.sqrt(16)`	`4.0`
`math.exp(x)`	`e` به توان `x`	`math.exp(1)`	`2.718...`
`math.log(x)`	لگاریتم طبیعی	`math.log(math.e)`	`1.0`
`math.log(x, base)`	لگاریتم در مبنای دلخواه	`math.log(8, 2)`	`3.0`
`math.floor(x)`	بزرگ‌ترین عدد صحیح کوچک‌تر یا مساوی	`math.floor(-2.5)`	`-3`
`math.ceil(x)`	کوچک‌ترین عدد صحیح بزرگ‌تر یا مساوی	`math.ceil(-2.5)`	`-2`
`math.isclose(a, b)`	مقایسه‌ی امن دو `float`	`math.isclose(0.1 + 0.2, 0.3)`	`True`
`math.pi` ، `math.e`	ثابت‌ها (تابع نیستند، پرانتز ندارند)	`math.pi`	`3.14159...`

`exp` و `log` در یادگیری ماشین همه‌جا هستند: در تابع سیگموید، softmax و تابع خطای cross-entropy.

```
import math

print(math.sqrt(2))
print(math.exp(0), math.log(1))
print(math.log(1024, 2))
print(math.floor(2.7), math.ceil(2.1), int(-2.7), math.floor(-2.7))
```

🔍 به خط آخر دقت کنید: برای اعداد منفی، `int` (حذف قسمت اعشاری، جلسه‌ی ۲) با `math.floor` (گرد کردن به پایین) فرق دارد: `int(-2.7)` برابر `-2` و `math.floor(-2.7)` برابر `-3` است.

ماژول `random`

تابع	کار
`random.random()`	عدد اعشاری تصادفی در بازه‌ی `[0, 1)`
`random.uniform(a, b)`	عدد اعشاری تصادفی بین `a` و `b`
`random.randint(a, b)`	عدد صحیح تصادفی بین `a` و `b` ، **هر دو شامل**
`random.gauss(mu, sigma)`	عدد تصادفی از توزیع نرمال با میانگین `mu` و انحراف معیار `sigma`
`random.choice(seq)`	یک عضو تصادفی از یک دنباله

تابع	کار
`random.sample(seq, k)`	`k` عضو تصادفی **بدون تکرار** (لیست جدید)
`random.shuffle(lst)`	ترتیب اعضای لیست را **در جا** به هم می‌ریزد
`random.seed(n)`	نقطه‌ی شروع تولید اعداد تصادفی را ثابت می‌کند

```python
import random

print(random.random())
print(random.uniform(-1, 1))
print(random.randint(1, 6))          # a dice roll
print(random.gauss(0, 1))            # a standard normal sample
print(random.choice(["rock", "paper", "scissors"]))
```

این سلول را چند بار اجرا کنید؛ هر بار عددهای متفاوتی می‌بینید.

💡 `seed` : تصادفی، اما تکرارپذیر

کامپیوتر در واقع اعداد «شبه‌تصادفی» تولید می‌کند: دنباله‌ای که از یک **نقطه‌ی شروع (seed)** با یک فرمول ثابت ساخته می‌شود. اگر نقطه‌ی شروع را با `random.seed` ثابت کنید، **همان دنباله** دوباره تولید می‌شود.

❓ پیش‌بینی کن

آیا دو خط چاپ‌شده یکسان‌اند؟ اگر خط `random.seed(42)` دوم را حذف کنید چطور؟

```python
random.seed(42)
first = [random.randint(1, 100), random.randint(1, 100), random.random()]

random.seed(42)
second = [random.randint(1, 100), random.randint(1, 100), random.random()]

print(first)
print(second)
print(first == second)
```

🔍 با seed یکسان، دنباله‌ی یکسانی تولید می‌شود؛ پس `first == second` برابر `True` است. بدون seed دوم، دنباله از جایی که قبلاً رسیده بود ادامه پیدا می‌کند و اعداد فرق می‌کنند.

چرا در علوم داده مهم است؟ تقسیم تصادفی داده، مقداردهی اولیه‌ی وزن‌های یک شبکه‌ی عصبی و بسیاری از الگوریتم‌ها به اعداد تصادفی وابسته‌اند. بدون seed ثابت، هر بار اجرا نتیجه‌ی کمی متفاوت می‌دهد و نمی‌توانید نتایجتان را تکرار یا با دیگران مقایسه کنید. در scikit-learn همین مفهوم با پارامتری به نام `random_state` دیده می‌شود.

⚠️ اشتباه رایج

- **`randint` با `range` فرق دارد:** `random.randint(1, 6)` می‌تواند خود `6` را هم برگرداند، اما `range(1, 6)` شامل `6` نیست.
- **`random.shuffle` چیزی برنمی‌گرداند:** لیست را **در جا** تغییر می‌دهد و حاصلش `None` است (مثل `sort` در جلسه‌ی ۸). `x = random.shuffle(x)` لیست را از دست می‌دهد.
- **نام فایل خودتان را `random.py` یا `math.py` نگذارید:** در این صورت `import random` به جای ماژول پایتون، فایل شما را وارد می‌کند و خطاهای عجیبی مثل `AttributeError: module 'random' has no attribute 'randint'` می‌گیرید.

۴. آرگومان‌های موقعیتی، نام‌دار و پیش‌فرض

💡 مفهوم

هنگام فراخوانی، پایتون باید تصمیم بگیرد هر آرگومان به کدام پارامتر برسد:

نوع	شکل فراخوانی	قاعده
موقعیتی (positional)	`display("John", 25)`	به ترتیب: اولی به پارامتر اول، دومی به دوم
نام‌دار (keyword)	`display(age=25, name="John")`	با نام پارامتر؛ ترتیب مهم نیست
پیش‌فرض (default)	`def greet(name, msg="Hi")`	اگر آرگومانی داده نشود، مقدار پیش‌فرض استفاده می‌شود

```python
def display(name, age):
    print("Name =", name, ", age =", age)

display("John", 25)
display(25, "John")
display(age=25, name="John")
```

🔍 کد را بخوان

- فراخوانی اول درست است.
- فراخوانی دوم **خطا نمی‌دهد**، اما نتیجه بی‌معنی است: `name = 25` و `age = "John"`. پایتون فقط **ترتیب** را می‌بیند، نه معنی را.
- فراخوانی سوم با آرگومان‌های نام‌دار، با وجود ترتیب برعکس، درست کار می‌کند. آرگومان نام‌دار کد را **خواناتر** و **امن‌تر** می‌کند.

اگر تعداد آرگومان‌ها با پارامترها جور نباشد، خطا می‌گیرید:

```
display("John")
```

🔍 خطای `TypeError: display() missing 1 required positional argument: 'age'` دقیقاً می‌گوید کدام پارامتر مقدار نگرفته است.

💡 مقدار پیش‌فرض

```
def greet(name, msg="Welcome to Python!"):
    print("Hello", name, msg)

greet("Hossein")
greet("Hossein", "How are you?")
greet(msg="Good morning!", name="Sara")
```

🔍 پارامتر `msg` مقدار پیش‌فرض دارد؛ پس دادن آن **اختیاری** است. در فراخوانی اول از پیش‌فرض استفاده می‌شود و در دومی مقدار جدید جای آن را می‌گیرد.

در علوم داده بسیار رایج است: بیشتر تابع‌های NumPy، Pandas و scikit-learn ده‌ها پارامتر با مقدار پیش‌فرض دارند و شما فقط آن‌هایی را که لازم دارید، **با نام** مشخص می‌کنید؛ مثلاً `round(3.14159, ndigits=2)` .

⚠️ اشتباه رایج

- در **تعریف** تابع، پارامترهای پیش‌فرض باید **بعد از** پارامترهای بدون پیش‌فرض بیایند: `def f(a=1, b)` خطای `SyntaxError` می‌دهد.
- در **فراخوانی**، آرگومان‌های موقعیتی باید **قبل از** آرگومان‌های نام‌دار بیایند: `greet(name="Ali", "Hi")` خطا می‌دهد.
- مقدار پیش‌فرض **یک بار**، در لحظه‌ی اجرای `def` حساب می‌شود؛ نه در هر فراخوانی. اگر پیش‌فرض را از یک متغیر بگیرید و بعداً آن متغیر را تغییر دهید، پیش‌فرض عوض نمی‌شود.

۵. `*args` و `**kwargs`

💡 مفهوم

گاهی تعداد ورودی‌ها از قبل معلوم نیست. مثلاً `print` هر تعداد مقداری را که بدهید می‌پذیرد. پایتون برای این حالت دو شکل ویژه دارد:

شکل	چه چیزی را جمع می‌کند	داخل تابع از چه نوعی است
`*args`	همه‌ی آرگومان‌های **موقعیتی** اضافه	تاپل

شکل	چه چیزی را جمع می‌کند	داخل تابع از چه نوعی است
`**kwargs`	همه‌ی آرگومان‌های **نام‌دار** اضافه	دیکشنری

اسم‌های `args` و `kwargs` فقط قرارداد هستند؛ چیزی که اهمیت دارد علامت `*` و `**` است.

```python
def mean(*values):
    print("values =", values)
    return sum(values) / len(values)

print(mean(17, 12.5, 19, 15))
print(mean(10, 20))
```

```python
def describe(name, **info):
    print("name:", name)
    for key in info:
        print(" ", key, "=", info[key])

describe("Ali", age=21, city="Bushehr", major="Statistics")
```

🔍 کد را بخوان

- در `mean`، همه‌ی آرگومان‌ها داخل یک **تاپل** به نام `values` جمع می‌شوند؛ خروجی خط اول این را نشان می‌دهد.
- در `describe`، آرگومان اول به `name` می‌رسد و بقیه‌ی آرگومان‌های نام‌دار داخل **دیکشنری** `info` جمع می‌شوند. (دیکشنری‌ها را در جلسه‌ی ۱۰ می‌خوانیم؛ فعلاً بدانید `info[key]` مقدار مربوط به هر نام را می‌دهد.)

لازم نیست این دو شکل را زیاد بنویسید، اما در **مستندات کتابخانه‌ها** مدام آن‌ها را می‌بینید؛ مثلاً امضای بسیاری از تابع‌های Pandas با `**kwargs` تمام می‌شود، یعنی «پارامترهای اضافه را به تابع دیگری منتقل می‌کنم».

۶. دامنه‌ی متغیرها

💡 مفهوم

دامنه (scope) یعنی یک نام در کدام قسمت برنامه **شناخته می‌شود**:

- **متغیر محلی (local):** متغیری که **داخل** تابع ساخته می‌شود (از جمله پارامترها). فقط داخل همان تابع و فقط هنگام اجرای آن وجود دارد.
- **متغیر سراسری (global):** متغیری که **بیرون** از همه‌ی تابع‌ها ساخته می‌شود. داخل تابع‌ها هم قابل **خواندن** است.

اگر داخل تابع نامی را **انتساب** کنید، یک متغیر **محلی جدید** ساخته می‌شود؛ حتی اگر متغیری سراسری با همان نام وجود داشته باشد.

```
def f(x):
    y = 3          # local variable
    return x * y

y = 1              # global variable
z = f(2)
print("z =", z)
print("y =", y)
```

🔍 کد را بخوان

- داخل `f` ، انتساب `y = 3` یک `y` **محلی** می‌سازد. `x * y` از همین `y` محلی استفاده می‌کند: `2 * 3 = 6`.
- `y` سراسری دست‌نخورده باقی می‌ماند: `y = 1` .
- بعد از پایان تابع، `x` و `y` محلی از بین می‌روند؛ اگر بیرون از تابع `print(x)` بنویسید، `NameError` می‌گیرید.

❓ پیش‌بینی کن

خروجی هر سلول چیست؟

```
def f(x):
    return x * y      # y is not defined inside f

y = 1
print(f(2))
y = 10
print(f(2))
```

```
p = 20

def demo():
    p = 10
    print("inside: p =", p)

demo()
print("outside: p =", p)
```

🔍 در سلول اول، `f` خودش `y` ندارد؛ پس `y` **سراسری** را در **لحظه‌ی اجرا** می‌خواند: بار اول `2` و بار دوم `20` . در سلول دوم، `p = 10` یک `p` **محلی** می‌سازد و `p` سراسری همان `20` می‌ماند.

⚠️ اشتباه رایج: `UnboundLocalError`

سلول زیر را اجرا کنید و پیام خطا را بخوانید:

```
counter = 0

def increment():
    counter = counter + 1
    return counter

increment()
```

🔍 چون داخل تابع به `counter` **انتساب** شده، پایتون آن را در **کل تابع** محلی فرض می‌کند. پس در سمت راست `counter + 1`، `counter` محلی هنوز مقداری ندارد و خطای `UnboundLocalError: cannot access local variable 'counter'` رخ می‌دهد.

با کلمه‌ی `global` می‌توان این را حل کرد، ولی **راه بهتر** این است که مقدار را به‌عنوان **پارامتر** بگیرید و نتیجه را **برگردانید**:

```
def increment(counter):
    return counter + 1

counter = increment(counter)
```

قاعده‌ی طلایی: یک تابع خوب همه‌ی چیزی را که لازم دارد از **پارامترها** می‌گیرد و نتیجه را با `return` برمی‌گرداند. چنین تابعی مستقل است و به سادگی خوانده و آزمایش می‌شود.

> **نکته برای آینده:** اگر یک **لیست** یا **دیکشنری** سراسری را داخل تابع **تغییر** دهید (مثلاً با `append`)، بدون `global` هم تغییر بیرون از تابع دیده می‌شود؛ چون به همان شیء اشاره می‌کنید (یادتان هست شکل «نام‌ها و مقدارها» در جلسه‌ی ۲؟). در جلسه‌ی ۸ این موضوع را مفصل می‌بینیم.

📌 آنچه باید بلد باشید

- تعریف تابع با `def`، فراخوانی آن، و اینکه تعریف کردن تابع آن را اجرا نمی‌کند.
- تفاوت پارامتر و آرگومان.
- **تفاوت `print` و `return`**؛ مقدار برگشتی تابعی که `return` ندارد `None` است.
- `return` اجرای تابع را فوراً تمام می‌کند؛ اثر `return` با تورفتگی اشتباه داخل حلقه.
- برگرداندن چند مقدار و باز کردن آن‌ها در چند متغیر.
- برای هر فراخوانی بگویید هر آرگومان به کدام پارامتر می‌رسد (موقعیتی، نام‌دار، پیش‌فرض) و کدام فراخوانی خطا می‌دهد.
- تابع‌های پرکاربرد `math` و `random`؛ `randint` هر دو سر بازه را شامل می‌شود؛ `shuffle` در جا تغییر می‌دهد؛ `seed` نتایج تصادفی را تکرارپذیر می‌کند.
- `*args` تاپل و `**kwargs` دیکشنری است.

- تابع را می‌توان بدون پرانتز به تابع دیگر داد؛ تفاوت `f` و `f(x)`.
- مقدار پیش‌فرض پارامتر در لحظه‌ی **تعریف** تابع حساب می‌شود.
- متغیر محلی و سراسری؛ انتساب داخل تابع یک متغیر محلی می‌سازد؛ علت `UnboundLocalError`.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — فراخوانی‌های تودرتو و مقدار پیش‌فرض

خروجی کد زیر را دقیقاً بنویسید. برای هر فراخوانی، مقدار هر پارامتر را مشخص کنید.

```
rate = 2

def scale(x, factor=rate):
    return x * factor

def shift(x, by=1):
    return scale(x) + by

rate = 10
print(shift(3), shift(3, by=scale(1, rate)), scale(shift(0),
factor=shift(1)))
```

▼ پاسخ

```
7 16 3
```

- **نکته‌ی کلیدی:** مقدار پیش‌فرض `factor=rate` در **لحظه‌ی تعریف** تابع حساب می‌شود، نه در لحظه‌ی فراخوانی. در آن لحظه `rate` برابر `2` بود؛ تغییر بعدی آن به `10` روی پیش‌فرض اثری ندارد.
- `shift(3)` یعنی `scale(3) + 1` که برابر است با `3 * 2 + 1 = 7`.
- `scale(1, rate)` یعنی `1 * 10 = 10` (این بار `rate` صریحاً و با مقدار فعلی‌اش داده شده).
  پس `shift(3, by=10)` برابر `6 + 10 = 16` است.
- `shift(0)` برابر `0 + 1 = 1` و `shift(1)` برابر `2 + 1 = 3` است؛ پس `scale(1, factor=3)` برابر `3` است.

سؤال ۲ — تابع‌های فعال‌سازی

در شبکه‌های عصبی، خروجی هر نورون از یک **تابع فعال‌سازی (activation function)** عبور می‌کند. سه تابع رایج را در نظر بگیرید:

```
import math
```

```python
def step(z):
    return 1 if z >= 0 else 0

def relu(z):
    return z if z > 0 else 0

def sigmoid(z):
    return 1 / (1 + math.exp(-z))
```

الف) جدول را کامل کنید (اعداد اعشاری را تا ۳ رقم گرد کنید؛ راهنمایی: `math.exp(-2)` تقریباً `0.135` است).

عبارت	مقدار؟
`step(0)`	
`relu(-2.5)`	
`relu(step(-1) - 3)`	
`sigmoid(0)`	
`sigmoid(2)`	
`sigmoid(-2) + sigmoid(2)`	

ب) چرا `sigmoid(-2) + sigmoid(2)` این مقدار را دارد؟ آیا برای هر عدد `z` همین نتیجه برقرار است؟

ج) اگر در تعریف `relu` به جای `return` از `print` استفاده شده بود، عبارت `relu(step(-1) - 3) + 1` چه نتیجه‌ای داشت؟

▼ پاسخ

الف)

عبارت	مقدار
`step(0)`	`1`
`relu(-2.5)`	`0`
`relu(step(-1) - 3)`	`relu(0 - 3)` یعنی `0`
`sigmoid(0)`	`0.5`
`sigmoid(2)`	`1 / 1.135` یعنی حدود `0.881`
`sigmoid(-2) + sigmoid(2)`	`1` (پایتون به خاطر دقت محدود اعشاری ممکن است `0.9999999999999999` چاپ کند)

ب) برای هر `z` داریم `sigmoid(-z) = 1 - sigmoid(z)` ، پس مجموع همیشه `1` است. اثبات: `1 / (1 + e^-z)` را در صورت و مخرج در `e^-z` ضرب کنید تا `e^-z / (1 + e^-z)` به دست آید که برابر `1 - sigmoid(z)` است. (این تقارن در رگرسیون لجستیک استفاده می‌شود: اگر احتمال کلاس ۱ برابر `sigmoid(z)` باشد، احتمال کلاس ۰ برابر `sigmoid(-z)` است.)

ج) `relu` مقدار `0` را **چاپ** می‌کرد ولی `None` برمی‌گرداند؛ در نتیجه `None + 1` خطای `TypeError` می‌داد. تابع‌هایی که قرار است با هم **ترکیب** شوند (مثل لایه‌های یک شبکه) حتماً باید `return` کنند.

سؤال ۳ – پرسپترون

ساده‌ترین نورون مصنوعی (**پرسپترون**) مجموع وزن‌دار ورودی‌ها را حساب می‌کند و آن را از تابع `step` عبور می‌دهد:

```
def neuron(x1, x2, w1, w2, b):
    z = w1 * x1 + w2 * x2 + b
    return 1 if z > 0 else 0
```

الف) برای `w1 = w2 = 1` و `b = -1.5` ، خروجی را برای هر چهار ترکیب `x1, x2` از مجموعه‌ی `{0, 1}` حساب کنید. این نورون کدام **دروازه‌ی منطقی** (`and` ، `or` ، ...) را پیاده‌سازی می‌کند؟

ب) فقط با تغییر `b` (و ثابت نگه داشتن وزن‌ها) نورون را به دروازه‌ی `or` تبدیل کنید. چه بازه‌ای از مقدارها برای `b` جواب می‌دهد؟

ج) وزن‌ها و بایاسی پیدا کنید که خروجی برابر `not x1` باشد (`x2` تأثیری نداشته باشد).

▼ پاسخ

الف)

x1	x2	z	خروجی
0	0	`-1.5`	0
0	1	`-0.5`	0
1	0	`-0.5`	0
1	1	`0.5`	1

این دروازه‌ی **`and`** است.

ب) برای `or` ، حالت `(0, 0)` باید `0` شود (`b <= 0` ، چون شرط `z > 0` است) و حالت‌های با یک ورودی `1` باید `1` شوند (`1 + b > 0` یعنی `b > -1`). پس هر `b` در بازه‌ی `-1 < b <= 0` ؛ مثلاً `b = -0.5` .

ج) مثلاً `w1 = -1` ، `w2 = 0` ، برای `b = 0.5` : برای `x1 = 0` مقدار `z = 0.5` و خروجی `1` ؛ برای `x1 = 1` مقدار `z = -0.5` و خروجی `0` .

(یادگیری در شبکه‌ی عصبی یعنی همین: پیدا کردن خودکار وزن‌ها و بایاس‌هایی که خروجی درست بدهند.)

سؤال ۴ — تابع خطا

برای سنجیدن کیفیت یک مدل، **خطای** هر پیش‌بینی (`y_true - y_pred`) را حساب می‌کنیم و آن‌ها را در یک عدد خلاصه می‌کنیم. دانشجویی تابع **میانگین مربعات خطا (MSE)** را این‌طور نوشته است:

```
def mse(errors):
    total = 0
    for e in errors:
        total += e
    return total ** 2 / len(errors)

print(mse([2, -2, 1, -1]))
```

الف) خروجی چیست؟ چرا این عدد نشان می‌دهد تابع اشتباه است؟

ب) تابع را درست کنید. خروجی درست برای همین ورودی چقدر است؟

ج) تابع `mae(errors)` (میانگین **قدر مطلق** خطاها) را بنویسید. برای ورودی `[2, -2, 1, -1]` و ورودی `[0, 0, 6]`، مقدار MSE و MAE را حساب کنید و توضیح دهید کدام معیار به **یک خطای بزرگ** حساس‌تر است.

▼ **پاسخ**

الف) خروجی `0.0` است: خطاهای مثبت و منفی همدیگر را خنثی کرده‌اند و مدلی که هیچ پیش‌بینی درستی ندارد «بی‌خطا» به نظر می‌رسد. تابع، **مربعِ مجموع** را حساب می‌کند، در حالی که باید **مجموعِ مربع‌ها** را حساب کند.

ب)

```
def mse(errors):
    total = 0
    for e in errors:
        total += e ** 2
    return total / len(errors)
```

خروجی: `(4 + 4 + 1 + 1) / 4 = 2.5` .

ج)

```
def mae(errors):
    total = 0
    for e in errors:
        total += abs(e)
    return total / len(errors)
```

ورودی	MSE	MAE
`[2, -2, 1, -1]`	`2.5`	`1.5`

ورودی	MSE	MAE
`[0, 0, 0, 6]`	`9.0`	`1.5`

MAE در هر دو حالت `1.5` است، اما MSE برای دومی خیلی بزرگ‌تر است؛ چون مربع کردن، خطای بزرگ را **بیشتر جریمه** می‌کند. MSE به داده‌های پرت حساس‌تر است.

سؤال ۵ — گرادیان کاهشی

گرادیان کاهشی (gradient descent) الگوریتم اصلی آموزش تقریباً همه‌ی مدل‌های یادگیری ماشین و شبکه‌های عصبی است: در هر قدم، `x` را کمی **خلاف جهت مشتق** حرکت می‌دهیم تا به کمینه‌ی تابع نزدیک شویم. کد زیر کمینه‌ی تابع `f(x) = (x - 3) ** 2` را پیدا می‌کند:

```
def grad(x):
    return 2 * (x - 3)      # derivative of (x - 3) ** 2

x = 0.0
lr = 0.25                    # learning rate
for step in range(3):
    x = x - lr * grad(x)
    print(step, x)
```

الف) جدول دنبال کردن را بنویسید (مقدار `grad(x)` و `x` بعد از هر قدم) و خروجی را بنویسید. اگر حلقه خیلی بیشتر تکرار شود، `x` به چه عددی نزدیک می‌شود؟

ب) با `lr = 1.0` سه مقدار اول `x` چیست؟ با `lr = 1.25` چطور؟ در هر حالت چه اتفاقی می‌افتد؟

ج) فاصله‌ی `x` تا جواب (`x - 3`) در هر قدم در چه عددی ضرب می‌شود؟ با استفاده از آن توضیح دهید برای چه بازه‌ای از `lr` الگوریتم همگرا می‌شود.

▼ **پاسخ**

الف)

قدم	`x` قبل	`grad(x)`	`x` بعد
0	`0.0`	`-6.0`	`1.5`
1	`1.5`	`-3.0`	`2.25`
2	`2.25`	`-1.5`	`2.625`

خروجی: `0 1.5` ، `1 2.25` ، `2 2.625` . با تکرار بیشتر، `x` به `3` (نقطه‌ی کمینه) نزدیک می‌شود.

ب) با `lr = 1.0` : `6.0` ، `0.0` ، `6.0` ؛ `x` بین ۰ و ۶ **نوسان** می‌کند و هرگز همگرا نمی‌شود. با `lr = 1.25` : `7.5` ، `-3.75` ، `13.125` ؛ `x` هر بار دورتر می‌شود و الگوریتم **واگرا** است.

ج) `x_new - 3 = (x - 3) - lr * 2 * (x - 3) = (1 - 2 * lr) * (x - 3)` . پس فاصله در هر قدم در `(1 - 2 * lr)` ضرب می‌شود. برای همگرایی باید قدر مطلق این عدد کمتر از ۱ باشد: `0 < lr < 1` . (برای `lr = 0.5` در یک قدم به جواب می‌رسیم.) این همان دلیلی است که در یادگیری عمیق، انتخاب **نرخ یادگیری** این‌قدر مهم است.

سؤال ۶ — تابع به‌عنوان ورودی: مشتق عددی

در عمل همیشه فرمول مشتق را نداریم. می‌توان مشتق را **به‌طور عددی** تقریب زد:

```
def numeric_grad(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

def loss(w):
    return (w - 3) ** 2

print(round(numeric_grad(loss, 0), 4))
print(round(numeric_grad(loss, 3), 4))
```

الف) خروجی دو خط آخر چیست؟ آن‌ها را با `grad` در سؤال ۵ مقایسه کنید.

ب) چرا در فراخوانی نوشته‌ایم `numeric_grad(loss, 0)` و نه `numeric_grad(loss(0), 0)` ؟ دومی چه خطایی می‌دهد؟

ج) با استفاده از `numeric_grad` ، مشتق `sigmoid` (از سؤال ۲) را در `z = 0` تقریب بزنید. جواب را با فرمول `sigmoid(z) * (1 - sigmoid(z))` مقایسه کنید.

▼ پاسخ

الف) `-6.0` و `0.0` ؛ دقیقاً برابر `grad(0)` و `grad(3)` در سؤال ۵. مشتق در نقطه‌ی کمینه صفر است.

ب) `numeric_grad` به خود **تابع** نیاز دارد تا بتواند آن را در دو نقطه‌ی `x + h` و `x - h` صدا بزند. `loss(0)` یک **عدد** است (`9`)، و داخل `numeric_grad` عبارت `f(x + h)` یعنی `9(...)` خطای `TypeError: 'int' object is not callable` می‌دهد.

ج) `numeric_grad(sigmoid, 0)` تقریباً `0.25` است و فرمول هم `0.5 * (1 - 0.5) = 0.25` می‌دهد. (چک کردن مشتق تحلیلی با مشتق عددی، روشی استاندارد برای پیدا کردن باگ در پیاده‌سازی شبکه‌های عصبی است و به آن **gradient checking** می‌گویند.)

سؤال ۷ — تکمیل کد: کمینه‌ساز عمومی

تابع زیر گرادیان کاهشی را برای **هر** تابعی که مشتقش داده شود اجرا می‌کند. اگر طول قدم از `tol` کوچک‌تر شود، زودتر متوقف می‌شود و مقدار نهایی `x` و **تعداد قدم‌ها** را برمی‌گرداند. جاهای خالی را کامل کنید.

```
def minimize(grad, x0, lr=0.1, tol=1e-6, max_iter=1000):
    x = x0
    for i in range(max_iter):
        step = ______
        x = ______
        if ______:
            return x, i + 1
    return x, max_iter

def grad_f(x):
    return 2 * (x - 3)

x_min, n_steps = minimize(grad_f, x0=0.0)
print(round(x_min, 4), n_steps)
```

بعد از کامل کردن:

الف) پارامترهای `lr` ، `tol` و `max_iter` در یادگیری ماشین به چه چیزی شبیه‌اند؟

ب) اگر بنویسیم `minimize(grad_f, 0.0, 1.5)` ، تابع چه چیزی برمی‌گرداند و چرا؟

▼ پاسخ

```
step = lr * grad(x)
x = x - step
if abs(step) < tol:
```

با پیش‌فرض‌ها، خروجی `3.0` و تعداد قدم‌هایی در حدود ۶۰ است (با این شرط توقف دقیقاً `61`).

الف) این‌ها **ابرپارامتر (hyperparameter)** هستند: مقدارهایی که الگوریتم یاد نمی‌گیرد و ما تعیین می‌کنیم. در کتابخانه‌هایی مثل scikit-learn دقیقاً با همین نام‌ها و به شکل پارامتر پیش‌فرض دیده می‌شوند (مثلاً `tol` و `max_iter`).

ب) آرگومان سوم (موقعیتی) به `lr` می‌رسد، پس `lr = 1.5` . ضریب `1 - 2 * 1.5 = -2` است و الگوریتم واگرا می‌شود؛ قدم‌ها هرگز کوچک نمی‌شوند و حلقه بعد از `1000` دور تمام می‌شود. تابع یک `x` با قدر مطلق بسیار بزرگ (در حد `10 ** 301`) و عدد `1000` را برمی‌گرداند. پس **همیشه** تعداد قدم‌ها را بررسی کنید: اگر برابر `max_iter` بود، یعنی الگوریتم همگرا نشده است.

سؤال ۸ — جست‌وجوی تصادفی وزن‌ها

ساده‌ترین روش «آموزش» یک مدل، **جست‌وجوی تصادفی (random search)** است: وزن‌های تصادفی امتحان می‌کنیم تا به وزن‌هایی برسیم که خروجی درست می‌دهند. کد زیر از تابع `neuron` سؤال ۳ استفاده می‌کند:

```
import random

def is_and(w1, w2, b):
```

```
    return (neuron(0, 0, w1, w2, b) == 0 and neuron(0, 1, w1, w2, b) == 0
            and neuron(1, 0, w1, w2, b) == 0 and neuron(1, 1, w1, w2, b) ==
1)

def random_search(seed, max_tries=1000):
    random.seed(seed)
    for t in range(max_tries):
        w1 = random.uniform(-1, 1)
        w2 = random.uniform(-1, 1)
        b = random.uniform(-1, 1)
        if is_and(w1, w2, b):
            return w1, w2, b, t + 1
    return None

print(random_search(1))
print(random_search(1))
print(random_search(2))
```

الف) آیا دو خط اول خروجی یکسان‌اند؟ خط سوم چطور؟ چرا؟

ب) اگر خط `random.seed(seed)` را حذف کنیم، چه چیزی تغییر می‌کند؟ چرا در گزارش یک آزمایش یادگیری ماشین باید seed را هم گزارش کرد؟

ج) هر جواب پیدا شده در چه شرایطی صدق می‌کند؟ با کمک جواب سؤال ۳، توضیح دهید چرا مثلاً `w1 = w2 = 0.6` و `b = -0.9` قابل قبول است. آیا جواب این مسئله یکتاست؟

د) تابع `is_xor` را مشابه `is_and` برای دروازه‌ی **XOR** بنویسید (خروجی ۱ فقط وقتی دقیقاً یکی از ورودی‌ها ۱ باشد) و `random_search` را با آن اجرا کنید. حتی با `max_tries=100000` هم `None` برمی‌گردد. با رسم چهار نقطه‌ی ورودی روی صفحه، توضیح دهید چرا **هیچ** وزنی برای یک نورون تنها وجود ندارد.

▼ پاسخ

الف) دو خط اول **دقیقاً یکسان‌اند**: هر بار `random_search(1)` اجرا می‌شود، seed دوباره روی `1` تنظیم می‌شود و همان دنباله‌ی اعداد تصادفی، همان وزن‌ها و همان تعداد تلاش را تولید می‌کند. خط سوم با seed دیگری شروع می‌کند؛ پس وزن‌ها و تعداد تلاش‌ها (به احتمال زیاد) فرق دارند.

ب) هر اجرا وزن‌ها و تعداد تلاش‌های متفاوتی می‌دهد و نتیجه **تکرارپذیر** نیست. اگر seed گزارش نشود، کس دیگری (یا خود شما در آینده) نمی‌تواند همان نتیجه را بازتولید کند.

ج) شرط‌ها (با `z = w1 * x1 + w2 * x2 + b` و خروجی ۱ وقتی `z > 0`): `b <= 0`، `w2 + b <= 0`، `w1 + b <= 0` و `w1 + w2 + b > 0`. برای `0.6، 0.6، -0.9`: `-0.9`، `-0.3`، `-0.3` همگی منفی‌اند و `0.3` مثبت است؛ پس قابل قبول است. جواب **یکتا نیست**: بی‌نهایت ترکیب این شرط‌ها را برآورده می‌کنند. (مدل‌های یادگیری ماشین هم معمولاً بی‌نهایت جواب «خوب» دارند و الگوریتم آموزش یکی از آن‌ها را پیدا می‌کند.)

د)

```
def is_xor(w1, w2, b):
    return (neuron(0, 0, w1, w2, b) == 0 and neuron(0, 1, w1, w2, b) == 1
            and neuron(1, 0, w1, w2, b) == 1 and neuron(1, 1, w1, w2, b) ==
0)
```

یک نورون تنها صفحه را با **یک خط راست** (`w1 * x1 + w2 * x2 + b = 0`) به دو نیم تقسیم می‌کند. در XOR، نقطه‌های `(0, 1)` و `(1, 0)` باید یک طرف خط و نقطه‌های `(0, 0)` و `(1, 1)` طرف دیگر باشند؛ اما این دو جفت روی دو قطر یک مربع‌اند و هیچ خط راستی آن‌ها را جدا نمی‌کند (داده **به‌طور خطی جدایی‌پذیر نیست**). جبری: از شرط‌ها `w1 + b > 0` و `w2 + b > 0` با هم جمع کنید: `w1 + w2 + 2b > 0` ؛ از `b <= 0` نتیجه می‌شود `w1 + w2 + b > -b >= 0` ، که با شرط `w1 + w2 + b <= 0` تناقض دارد. همین مشکل، تاریخی‌ترین دلیل نیاز به شبکه‌های عصبی **چندلایه** است.

خلاصه‌ی جلسه

- تابع یک بلوک کد نام‌دار است که با `def` تعریف و با `name(...)` فراخوانی می‌شود؛ کد تکراری را حذف می‌کند.
- **`return`** مقدار را برمی‌گرداند و تابع را تمام می‌کند؛ **`print`** فقط نمایش می‌دهد. تابع بدون `return` مقدار `None` برمی‌گرداند.
- آرگومان‌ها به ترتیب (موقعیتی) یا با نام (نام‌دار) به پارامترها می‌رسند؛ پارامترها می‌توانند مقدار پیش‌فرض داشته باشند.
- تابع‌های آماده‌ی `math` و `random` را با `import` وارد کنید؛ برای تکرارپذیری، `random.seed` را ثابت کنید.
- `*args` آرگومان‌های اضافه را در یک تاپل و `**kwargs` در یک دیکشنری جمع می‌کند.
- متغیرهای داخل تابع محلی‌اند؛ تابع خوب ورودی را از پارامتر می‌گیرد و خروجی را `return` می‌کند.

جلسه‌ی بعد: لیست‌ها، اولین ساختار داده‌ی پایتون.

جلسه ۸: لیست‌ها

Lists

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- لیست بسازید و با **اندیس** (مثبت و منفی) و **برش (slicing)** به اعضای آن دسترسی پیدا کنید،
- متدهای اصلی لیست را به کار ببرید و بدانید کدام **خود لیست را تغییر می‌دهند** و کدام **لیست جدید** می‌سازند،
- مفهوم **تغییرپذیری (mutability)** و **نام‌های هم‌اشاره (aliasing)** را توضیح دهید و باگ‌های ناشی از آن را پیدا کنید،
- روی لیست با `for`، `range(len(...))`، `enumerate` و `zip` حلقه بزنید و با الگوی `append` لیست جدید بسازید،
- با **لیست‌های تودرتو** مثل یک ماتریس کار کنید.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. ساخت لیست
2. اندیس‌گذاری و برش
3. متدهای لیست
4. تغییرپذیری و کپی
5. پیمایش لیست و ساختن لیست جدید
6. لیست‌های تودرتو

۱. ساخت لیست

💡 مفهوم

لیست (list) مجموعه‌ای **مرتب** از مقدارهاست که داخل `[ ]` و با کاما از هم جدا می‌شوند. اعضای لیست می‌توانند از هر نوعی باشند (حتی نوع‌های مختلف در یک لیست)، و لیست **تغییرپذیر** است: بعد از ساختن، می‌توان اعضای آن را عوض، اضافه یا حذف کرد.

در علوم داده، لیست ساده‌ترین شکل نگه‌داری یک **ستون داده** یا یک **بردار** است؛ مثلاً نمره‌های یک کلاس یا وزن‌های یک مدل.

```
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "apple", True, 3.14]
empty = []

print(numbers, fruits, mixed, empty)
print(len(numbers), len(empty))
print(type(numbers))
```

💡 راه‌های دیگر ساختن لیست و عملگرها

```
print(list("hello"))          # from a string
print(list(range(0, 10, 3)))  # from a range

A = [1, 4, 2]
B = [4, 9, 3]
print(A + B)                  # concatenation (a new list)
print([0] * 4)                # repetition
print(4 in A, 7 not in B)     # membership
```

🔍 کد را بخوان

- `list(...)` هر دنباله‌ای را به لیست تبدیل می‌کند؛ حالا می‌توانید اعداد یک `range` را ببینید.
- `+` و `*` روی لیست‌ها مثل رشته‌ها رفتار می‌کنند (جلسه‌ی ۲): چسباندن و تکرار. **جمع برداری نیستند!** `[1, 4, 2] + [4, 9, 3]` لیستی شش‌عضوی است، نه `[5, 13, 5]`. (جمع برداری را NumPy در بخش ۳ انجام می‌دهد.)

💡 تبدیل رشته و لیست

دو متد رشته‌ای بسیار پرکاربرد، پلی بین رشته و لیست هستند:

```
song = "The rain in Spain"
words = song.split()          # string -> list of words
print(words)

line = "17,12.5,19,15"
print(line.split(","))        # split on commas (like a CSV line)

print(" ".join(words))        # list of strings -> string
print("-".join(["2026", "10", "30"]))
```

🔍 `split` رشته را در محل فاصله‌ها (یا هر جداکننده‌ای که بدهید) می‌شکند. این دقیقاً کاری است که برای خواندن یک خط از فایل CSV لازم است. دقت کنید اعضای حاصل **رشته** هستند (`'17'`، نه `17`). `join` برعکس عمل می‌کند و فقط روی لیستی از **رشته‌ها** کار می‌کند.

۲. اندیس‌گذاری و برش

💡 اندیس

هر عضو لیست یک **اندیس (index)** دارد که از **صفر** شروع می‌شود. اندیس **منفی** از انتهای لیست می‌شمارد: `-1` آخرین عضو است.



اندیس‌های مثبت (بالا) و منفی (پایین) برای یک لیست پنج‌عضوی.

```
colors = ["red", "blue", "green", "black", "white"]
print(colors[0], colors[2], colors[-1], colors[-2])

colors[0] = "navy"            # lists are mutable
print(colors)
```

اگر اندیسی خارج از محدوده بدهید، خطای `IndexError` می‌گیرید. برای لیستی با `n` عضو، اندیس‌های مجاز `0` تا `n - 1` (و `-1` تا `-n`) هستند:

```
colors[5]
```

💡 برش (slicing)

`lst[start:stop:step]` یک **لیست جدید** شامل بخشی از لیست می‌سازد. قواعدش دقیقاً مثل `range` است (جلسه‌ی ۶): `start` شامل است، `stop` **شامل نیست**، و `step` گام است. هر کدام را می‌توان حذف کرد:

برش	معنی
`lst[2:5]`	اندیس ۲ تا ۴
`lst[:3]`	از ابتدا تا اندیس ۲ (سه عضو اول)
`lst[3:]`	از اندیس ۳ تا انتها
`lst[-2:]`	دو عضو آخر
`lst[::2]`	یکی در میان
`lst[::-1]`	کل لیست به ترتیب برعکس

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(numbers[2:6])
print(numbers[:5])
print(numbers[5:])
print(numbers[::2])
print(numbers[::-1])
print(numbers[-3:])
```

🔍 کد را بخوان

- `numbers[2:6]` اندیس‌های `2، 3، 4، 5` را برمی‌دارد (۴ عضو؛ تعداد = `stop - start`).
- **برش هیچ‌وقت `IndexError` نمی‌دهد:** `numbers[5:100]` بدون خطا تا انتهای لیست را برمی‌گرداند و `numbers[8:3]` یک لیست خالی است.

یک کاربرد واقعی در یادگیری ماشین: تقسیم داده به بخش **آموزش** و **آزمون**. داده‌ها معمولاً **مرتب** ذخیره شده‌اند (مثلاً بر اساس تاریخ یا کلاس)؛ پس قبل از برش، ترتیب آن‌ها را با `random.shuffle` (جلسه‌ی ۷) به هم می‌ریزیم:

```python
import random

data = [3, 4, 6, 7, 8, 9, 11, 12, 14, 15]    # sorted data

random.seed(0)                # reproducible split
shuffled = data.copy()        # shuffle works in place: keep the original
random.shuffle(shuffled)

n_train = int(0.8 * len(shuffled))
train = shuffled[:n_train]
test = shuffled[n_train:]
print("train:", train)
print("test: ", test)
print("data: ", data)
```

🔍 کد را بخوان

- اگر بدون به‌هم‌ریختن برش می‌زدیم، داده‌ی آزمون فقط شامل **بزرگ‌ترین** اعداد (`14` و `15`) می‌شد و مدل روی داده‌ای ارزیابی می‌شد که شبیه داده‌ی آموزش نیست.
- `random.shuffle` لیست را **در جا** تغییر می‌دهد؛ برای همین اول یک کپی گرفته‌ایم تا `data` اصلی دست‌نخورده بماند (بخش ۴ همین جلسه را ببینید).
- `random.seed(0)` باعث می‌شود هر بار اجرا، **همان** تقسیم به دست آید؛ تا نتایج آزمایش قابل تکرار باشد.

❓ پیش‌بینی کن

قبل از اجرا بنویسید هر خط چه چیزی چاپ می‌کند.

```python
colors = ["red", "blue", "green", "black", "white"]
print(colors[1:-1])
print(colors[-1][0])
print(colors[::-2])
colors[1:3] = ["gray"]
print(colors, len(colors))
```

🔍 خط دوم: `colors[-1]` رشته‌ی `"white"` است و `[0]` اولین حرف آن (`"w"`) را برمی‌دارد. خط سوم: از انتها، یکی در میان. در خط چهارم یک **برش** (دو عضو) با لیستی **یک‌عضوی** جایگزین می‌شود؛ پس طول لیست از ۵ به ۴ می‌رسد.

۳. متدهای لیست

💡 مفهوم

متد	کار	چه چیزی برمی‌گرداند؟
`lst.append(x)`	`x` را به انتها اضافه می‌کند	`None`
`lst.extend(other)`	همه‌ی اعضای `other` را به انتها اضافه می‌کند	`None`
`lst.insert(i, x)`	`x` را در اندیس `i` قرار می‌دهد	`None`
`lst.remove(x)`	**اولین** `x` را حذف می‌کند (اگر نباشد: `ValueError`)	`None`
`lst.pop()` / `lst.pop(i)`	آخرین عضو / عضو اندیس `i` را حذف می‌کند	**عضو حذف‌شده**
`lst.index(x)`	اندیس اولین `x`	عدد
`lst.count(x)`	تعداد دفعات تکرار `x`	عدد
`lst.sort()`	لیست را **در جا** مرتب می‌کند	`None`
`lst.reverse()`	لیست را **در جا** برعکس می‌کند	`None`
`lst.copy()`	یک کپی از لیست	لیست جدید

و چند **تابع** داخلی که روی لیست کار می‌کنند: `len`، `sum`، `min`، `max` و `sorted`.

```
fruits = ["apple", "banana"]
fruits.append("cherry")
fruits.extend(["orange", "kiwi"])
fruits.insert(1, "mango")
print(fruits)

fruits.remove("banana")
removed = fruits.pop(1)
last = fruits.pop()
print(fruits, "| removed:", removed, "| last:", last)
```

🔍 کد را بخوان: `append` یا `extend`؟

`append` **یک** عضو اضافه می‌کند، هر چه باشد؛ حتی اگر آن عضو خودش یک لیست باشد. `extend` اعضای لیست دیگر را **یکی‌یکی** اضافه می‌کند:

```
a = [1, 2, 3]
a.append([4, 5])
print(a, len(a))

b = [1, 2, 3]
b.extend([4, 5])
print(b, len(b))
```

💡 در جا (in-place) یا لیست جدید؟

این یکی از مهم‌ترین تمایزها در پایتون (و بعدها در Pandas) است:

- `lst.sort()` **همان** لیست را تغییر می‌دهد و `None` برمی‌گرداند.
- `sorted(lst)` به لیست اصلی دست نمی‌زند و یک **لیست جدید** مرتب برمی‌گرداند.

❓ پیش‌بینی کن

```
scores = [15, 9, 18, 12]
ranked = sorted(scores, reverse=True)
print(scores, ranked)

result = scores.sort()
print(result, scores)
```

🔍 `sorted` لیست جدید ساخت و `scores` دست‌نخورده ماند. اما `scores.sort()` خود `scores` را مرتب کرد و حاصلش `None` است؛ پس `result` برابر `None` است.

⚠️ اشتباه رایج: `x = x.sort()`

نوشتن `scores = scores.sort()` لیست را مرتب می‌کند و بلافاصله نام `scores` را به `None` وصل می‌کند؛ لیست مرتب‌شده از دست می‌رود! همین اشتباه با `append` هم رایج است: `lst = lst.append(5)` . قاعده: **متدهایی که در جا تغییر می‌دهند، `None` برمی‌گردانند.**

۴. تغییرپذیری و کپی

💡 مفهوم

در جلسه‌ی ۲ دیدید که متغیر یک **نام** است که به یک **مقدار** اشاره می‌کند، و `b = a` یعنی «`b` را به **همان** مقداری وصل کن که `a` به آن اشاره می‌کند». برای اعداد و رشته‌ها این موضوع مشکلی ایجاد نمی‌کرد، چون آن‌ها **تغییرناپذیر**ند. اما لیست **تغییرپذیر** است؛ پس اگر دو نام به یک لیست اشاره کنند، تغییر از طریق یکی، از طریق دیگری هم دیده می‌شود. به این حالت **هم‌اشاره‌سازی (aliasing)** می‌گویند.

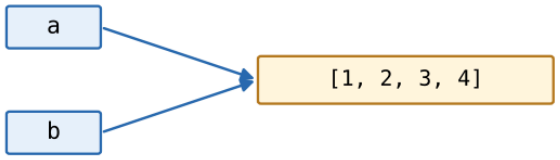
❓ پیش‌بینی کن

خروجی سلول زیر چیست؟ (این همان «نتیجه‌ی غافلگیرکننده»‌ای است که در جلسه‌ی ۲ وعده داده بودیم.)

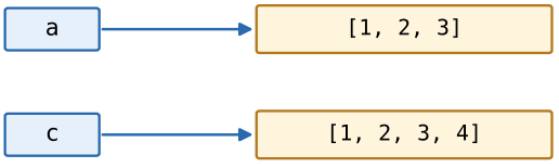
```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)
```

🔍 `b = a` لیست را **کپی نمی‌کند**؛ فقط یک نام دوم برای همان لیست می‌سازد. پس `b.append(4)` همان لیستی را تغییر می‌دهد که `a` به آن اشاره می‌کند. برای داشتن یک لیست **مستقل** باید صریحاً کپی بگیرید:

```
a = [1, 2, 3]
b = a
b.append(4)
```




```
a = [1, 2, 3]
c = a.copy()
c.append(4)
```



چپ: `b = a` دو نام برای یک لیست است. راست: `a.copy()` یک لیست جدید می‌سازد و تغییر آن روی `a` اثری ندارد.

```
a = [1, 2, 3]
c = a.copy()          # also: a[:] or list(a)
c.append(4)
print(a, c)
print(a == c, a is c)
```

- `a.copy()`، `a[:]` و `list(a)` هر سه یک لیست **جدید** با همان اعضا می‌سازند.
- `==` بررسی می‌کند **محتوا** برابر است؛ `is` بررسی می‌کند دو نام به **یک شیء** اشاره می‌کنند یا نه.

💡 لیست به‌عنوان آرگومان تابع

وقتی لیستی را به تابع می‌دهید، پارامتر تابع به **همان** لیست اشاره می‌کند (کپی نمی‌شود). پس اگر تابع لیست را در جا تغییر دهد، تغییر بیرون از تابع هم دیده می‌شود؛ همان نکته‌ای که در پایان جلسه‌ی ۷ اشاره کردیم:

```python
def add_bias(features):
    features.append(1.0)      # modifies the caller's list!
    return features

x = [0.5, 2.0]
x_with_bias = add_bias(x)
print(x, x_with_bias, x is x_with_bias)
```

🔍 بعد از فراخوانی، `x` هم تغییر کرده است؛ شاید این چیزی نباشد که فراخواننده انتظار داشت. تابع‌هایی که **لیست جدید** برمی‌گردانند (مثل `sorted`) معمولاً امن‌تر و قابل پیش‌بینی‌ترند. اگر تابعی ورودی‌اش را تغییر می‌دهد، این را در docstring آن بنویسید.

⚠️ اشتباه رایج: ساختن ماتریس با `*`

`[0] * 3` برای اعداد مشکلی ندارد، اما `[[0] * 3] * 3` سه بار به **یک** لیست درونی اشاره می‌کند:

```python
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)

grid = []
for i in range(3):
    grid.append([0] * 3)      # a NEW inner list each time
grid[0][0] = 1
print(grid)
```

🔍 در حالت اول، هر سه سطر **یک لیست** هستند؛ پس تغییر یک خانه در «سطر اول» در همه‌ی سطرها دیده می‌شود. در حالت دوم، در هر دور حلقه یک لیست درونی **جدید** ساخته می‌شود.

۵. پیمایش لیست و ساختن لیست جدید

💡 چهار روش پیمایش

روش	وقتی که ...
`for x in lst:`	فقط به **مقدار** اعضا نیاز دارید
`for i in range(len(lst)):`	به **اندیس** نیاز دارید (مثلاً برای تغییر `lst[i]`)
`for i, x in enumerate(lst):`	هم اندیس و هم مقدار را می‌خواهید
`for x, y in zip(lst1, lst2):`	می‌خواهید **همزمان** روی دو لیست حرکت کنید

```
colors = ["red", "blue", "green"]

for c in colors:
    print(c, end=" ")
print()

for i in range(len(colors)):
    print(i, colors[i], end=" | ")
print()

for i, c in enumerate(colors):
    print(i, c, end=" | ")
print()

sizes = [3, 4, 5]
for c, s in zip(colors, sizes):
    print(c, s, end=" | ")
```

🔍 کد را بخوان

- `enumerate(colors)` در هر دور یک **جفت** (اندیس، مقدار) می‌دهد که با `i, c` باز می‌شود؛ همان انتساب چندگانه‌ی جلسه‌ی ۲.
- `zip(colors, sizes)` اعضای **هم‌اندیس** دو لیست را جفت می‌کند. اگر طول لیست‌ها برابر نباشد، `zip` بی‌صدا در انتهای **کوتاه‌ترین** لیست متوقف می‌شود (این را به خاطر بسپارید!).

💡 الگوی ساختن لیست جدید

مثل الگوی جمع‌کننده (جلسه‌ی ۶)، این بار با یک **لیست خالی** شروع می‌کنیم و در حلقه `append` می‌کنیم. این الگو برای **فیلتر کردن** و **تبدیل کردن** داده بسیار رایج است:

```
colors = ["red", "blue", "green", "black", "white"]
long_names = []
for c in colors:
    if len(c) > 4:
        long_names.append(c)
print(long_names)

def odds(values):
    result = []
    for v in values:
        if v % 2 == 1:
            result.append(v)
    return result

print(odds([5, 7, 2, 4, 81]))
```

> **نکته برای آینده:** پایتون برای این الگو یک شکل کوتاه یک‌خطی به نام **list comprehension** دارد (`[c for c in colors if len(c) > 4]`) که در جلسه‌ی ۱۳ می‌خوانیم. اگر در کدی دیدید، `[c` را نادیده بگیرید.

> معنی‌اش همین حلقه است.

⚠️ اشتباه رایج: تغییر لیست حین پیمایش

حذف کردن از لیستی که روی آن حلقه می‌زنید، نتیجه‌ی غیرمنتظره می‌دهد:

```
nums = [1, 2, 2, 3, 2]
for n in nums:
    if n == 2:
        nums.remove(n)
print(nums)
```

🔍 هدف حذف همه‌ی `2` ها بود، اما یکی باقی ماند! وقتی عضوی حذف می‌شود، بقیه‌ی اعضا یک خانه به عقب می‌روند و حلقه عضو بعدی را **جا می‌اندازد**. راه درست: یک **لیست جدید** از اعضای مطلوب بسازید (الگوی بالا)، یا روی یک کپی (`for n in nums[:]`) حلقه بزنید.

۶. لیست‌های تودرتو

💡 مفهوم

عضو یک لیست می‌تواند خودش یک لیست باشد. با این کار می‌توان یک **جدول** یا **ماتریس** ساخت: لیستی از سطرها که هر سطر لیستی از اعداد است. برای دسترسی، **دو** اندیس پشت سر هم می‌نویسیم: `m[row][col]`.

```
m = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

print(m[1])          # second row
print(m[1][2])       # row 1, column 2
print(len(m), len(m[0]))   # number of rows, number of columns

column_1 = []
for row in m:
    column_1.append(row[1])
print(column_1)
```

🔍 کد را بخوان

- `m[1]` سطر دوم (یک لیست) است و `m[1][2]` سومین عضو آن سطر: `6`.
- برای گرفتن یک **ستون** باید روی سطرها حلقه بزنیم و از هر سطر عضو مورد نظر را برداریم. (در Pandas و NumPy این کار با یک عبارت کوتاه انجام می‌شود.)

❓ پیش‌بینی کن

کد زیر **ترانهاده‌ی** (transpose) ماتریس را می‌سازد. خروجی را پیش‌بینی کنید و توضیح دهید چرا حلقه‌ی بیرونی روی **ستون‌ها** است.

```
m = [[1, 2, 3],
     [4, 5, 6]]

t = []
for j in range(len(m[0])):
    new_row = []
    for i in range(len(m)):
        new_row.append(m[i][j])
    t.append(new_row)
print(t)
```

🔍 ماتریس `2 × 3` به ماتریس `3 × 2` تبدیل می‌شود: `[[1, 4], [2, 5], [3, 6]]` . سطر `j` ام ترانهاده، همان ستون `j` ام ماتریس اصلی است؛ برای همین حلقه‌ی بیرونی روی اندیس ستون‌ها (`j`) و حلقه‌ی درونی روی سطرها (`i`) است.

📌 آنچه باید بلد باشید

- نتیجه‌ی اندیس (مثبت و منفی) و برش `lst[start:stop:step]` را دقیقاً پیش‌بینی کنید؛ اندیس خارج از محدوده خطا می‌دهد ولی برش نه.
- `+` و `*` روی لیست یعنی چسباندن و تکرار، نه عملیات برداری.
- متدهای اصلی و مقدار برگشتی‌شان؛ تفاوت `append` و `extend` ؛ تفاوت `sort` (در جا، `None`) و `sorted` (لیست جدید).
- `b = a` کپی نمی‌کند؛ `copy` ، `[:]` و `list()` کپی می‌کنند؛ تفاوت `==` و `is` .
- تابعی که لیست ورودی را در جا تغییر می‌دهد، لیست فراخواننده را هم تغییر می‌دهد.
- خطای `[[0] * n] * m` و خطای حذف از لیست حین پیمایش.
- `enumerate` و `zip` (و رفتار `zip` با طول‌های نابرابر).
- الگوی ساختن لیست جدید با `append` برای فیلتر و تبدیل.
- تقسیم آموزش و آزمون: به‌هم‌ریختن روی کپی (یا روی اندیس‌ها) با seed ثابت.
- دسترسی `m[i][j]` ، استخراج ستون و ترانهاده‌ی ماتریس با حلقه‌ی تودرتو.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — اندیس و برش

با فرض `d = [3, 8, -1, 4, 9, 0, 7]`، حاصل هر عبارت را بنویسید.

عبارت	حاصل؟
`d[2:5]`	
`d[-3:]`	
`d[::3]`	
`d[5:1:-2]`	
`d[-1:-4]`	
`d[len(d) // 2]`	
`d[1:-1][::-1]`	

▼ پاسخ

عبارت	حاصل
`d[2:5]`	`[-1, 4, 9]`
`d[-3:]`	`[9, 0, 7]`
`d[::3]`	`[3, 4, 7]` (اندیس‌های ۰، ۳، ۶)
`d[5:1:-2]`	`[0, 4]` (اندیس‌های ۵ و ۳)
`d[-1:-4]`	`[]` (گام مثبت است، ولی شروع بعد از پایان قرار دارد)
`d[len(d) // 2]`	`4` (عضو میانی، اندیس ۳)
`d[1:-1][::-1]`	`[0, 9, 4, -1, 8]` (حذف اولین و آخرین عضو، سپس برعکس)

سؤال ۲ — ذخیره‌ی «بهترین وزن‌ها»

در حلقه‌ی آموزش یک مدل، معمولاً بهترین وزن‌هایی را که تا الان دیده‌ایم ذخیره می‌کنیم. کد زیر را دنبال کنید و خروجی را بنویسید.

```
w = [0.5, -1.0, 2.0]
best = w
history = [w, w.copy()]

w[0] = 9
best.append(1)
```

```
print(w)
print(best)
print(history)
```

الف) خروجی چیست؟ در پایان کار، **چند** لیست مجزا در حافظه وجود دارد؟

ب) برنامه‌نویس می‌خواست `best` وزن‌های لحظه‌ی ذخیره را نگه دارد. خط مربوط را درست کنید.

▼ پاسخ

الف)

```
[9, -1.0, 2.0, 1]
[9, -1.0, 2.0, 1]
[[9, -1.0, 2.0, 1], [0.5, -1.0, 2.0]]
```

فقط **سه** لیست وجود دارد: لیست اصلی (که `w`، `best` و `history[0]` همگی به آن اشاره می‌کنند)، کپی گرفته‌شده در `history[1]`، و خود لیست `history`. هر تغییری از طریق `w` یا `best` در هر سه نام دیده می‌شود.

ب) `best = w.copy()` (یا `w[:]`). در کتابخانه‌های یادگیری عمیق هم دقیقاً همین باگ رایج است و برای ذخیره‌ی وزن‌ها باید یک کپی جدا گرفت.

سؤال ۳ — عملیات برداری

در یادگیری ماشین هر نمونه‌ی داده یک **بردار** است. تابع ضرب داخلی (dot product) را در نظر بگیرید:

```
def dot(u, v):
    total = 0
    for a, b in zip(u, v):
        total += a * b
    return total
```

الف) حاصل `dot([1, 2, 3], [4, -5, 6])` و `dot([1, 2, 3], [1, 1])` چیست؟ چرا دومی خطرناک است؟ تابع را طوری تغییر دهید که در این حالت متوجه شوید (مثلاً به جای جواب، `None` برگرداند).

ب) با استفاده از `dot`، تابع `norm(u)` (طول بردار، یعنی جذر `dot(u, u)`) و تابع `cosine(u, v)` (شباهت کسینوسی: `dot(u, v)` تقسیم بر حاصل‌ضرب طول دو بردار) را بنویسید. شباهت کسینوسی یک بردار با خودش چقدر است؟ با `[-1, -2, -3]` چطور؟

ج) خروجی یک نورون برابر `sigmoid(dot(w, x) + b)` است (جلسه‌ی ۷). اگر `w = [0.2, -0.5, 1.0]`، `x = [1, 2, 0.5]` و `b = 0.3` باشد، `dot(w, x) + b` چقدر است؟

▼ پاسخ

الف) `4 - 10 + 18 = 12` و `1 + 2 = 3`. برای دومی `zip` در انتهای لیست کوتاه‌تر متوقف می‌شود و جواب بدون هیچ خطایی برمی‌گردد؛ در حالی که ضرب داخلی برای بردارهای با طول نابرابر تعریف نشده است. چنین باگ‌های

بی‌صدایی خطرناک‌ترین باگ‌ها هستند.

```
def dot(u, v):
    if len(u) != len(v):
        return None
    total = 0
    for a, b in zip(u, v):
        total += a * b
    return total
```

(در جلسه‌ی ۱۵ یاد می‌گیرید به جای `None` ، یک **خطا** ایجاد کنید که بهتر است.)

ب)

```
def norm(u):
    return dot(u, u) ** 0.5

def cosine(u, v):
    return dot(u, v) / (norm(u) * norm(v))
```

شباهت کسینوسی بردار با خودش `1` و با قرینه‌اش `-1` است (زاویه‌ی ۰ و ۱۸۰ درجه).

ج) `0.2 - 1.0 + 0.5 + 0.3 = 0` ؛ پس خروجی نورون `sigmoid(0) = 0.5` است. (اگر اجرا کنید، پایتون به خاطر دقت محدود اعشاری عددی بسیار نزدیک به صفر، مثل `-5.55e-17` ، نشان می‌دهد.)

سؤال ۴ — نرمال‌سازی

قبل از آموزش بیشتر مدل‌ها، ویژگی‌ها را **مقیاس‌بندی** می‌کنیم. دو تابع زیر را در نظر بگیرید:

```
def min_max(xs):
    lo, hi = min(xs), max(xs)
    for i in range(len(xs)):
        xs[i] = (xs[i] - lo) / (hi - lo)
    return xs

def z_score(xs):
    m = sum(xs) / len(xs)
    ss = 0
    for x in xs:
        ss += (x - m) ** 2
    sd = (ss / len(xs)) ** 0.5
    result = []
    for x in xs:
        result.append((x - m) / sd)
    return result
```

الف) برای `ages = [2, 4, 6, 8]`، خروجی `min_max(ages)` و `z_score(ages)` را حساب کنید (`z_score` را تا سه رقم اعشار).

ب) اگر اول `z = z_score(ages)` و **بعد** `mm = min_max(ages)` اجرا شود، نتیجه با حالتی که ترتیب برعکس باشد فرق دارد؟ چرا؟ کدام تابع «خطرناک» است و چطور درستش می‌کنید؟

ج) هر دو تابع برای چه ورودی‌ای خطا می‌دهند؟ (راهنمایی: مخرج‌ها را نگاه کنید.)

▼ **پاسخ**

الف) `min_max`: `[0.0, 0.333..., 0.666..., 1.0]`. `z_score`: میانگین `5` و انحراف معیار $\sqrt{5} \approx 2.236$؛ نتیجه `[-1.342, -0.447, 0.447, 1.342]`.

ب) بله. `min_max` لیست ورودی را **در جا** تغییر می‌دهد. اگر اول اجرا شود، `ages` به `[0.0, 0.333, 0.667, 1.0]` تبدیل می‌شود و `z_score` بعدی روی داده‌ی نرمال‌شده (نه داده‌ی اصلی) حساب می‌شود. (z-score اتفاقاً به مقیاس‌بندی خطی حساس نیست و همان اعداد را می‌دهد، اما داده‌ی اصلی `ages` برای همیشه از دست رفته است.) `min_max` باید مثل `z_score` یک **لیست جدید** بسازد:

```
def min_max(xs):
    lo, hi = min(xs), max(xs)
    result = []
    for x in xs:
        result.append((x - lo) / (hi - lo))
    return result
```

ج) وقتی همه‌ی اعضا برابر باشند (مثلاً `[5, 5, 5]`): `hi - lo` و `sd` صفر می‌شوند و `ZeroDivisionError` رخ می‌دهد. (و البته برای لیست خالی.)

سؤال ۵ — softmax و argmax

خروجی لایه‌ی آخر یک شبکه‌ی دسته‌بندی، یک لیست **امتیاز** (logit) است. تابع **softmax** آن‌ها را به احتمال تبدیل می‌کند و **argmax** اندیس بزرگ‌ترین مقدار (یعنی کلاس پیش‌بینی‌شده) را می‌دهد:

```
import math

def softmax(scores):
    exps = []
    for s in scores:
        exps.append(math.exp(s))
    total = sum(exps)
    probs = []
    for e in exps:
        probs.append(e / total)
    return probs
```

```
def argmax(values):
    best_i = 0
    for i in range(1, len(values)):
        if values[i] > values[best_i]:
            best_i = i
    return best_i
```

الف) برای `scores = [2.0, 1.0, 0.1]` خروجی `softmax` را تا سه رقم حساب کنید (`e^2 ≈ 7.389`، `e^1 ≈ 2.718`، `e^0.1 ≈ 1.105`). مجموع احتمال‌ها چقدر است؟ `argmax(scores)` و `argmax(softmax(scores))` را مقایسه کنید.

ب) `argmax([3, 7, 7, 1])` چه عددی برمی‌گرداند؟ اگر در `if` به جای `>` از `>=` استفاده شود چطور؟

ج) `softmax([1000, 999])` خطای `OverflowError` می‌دهد. نشان دهید اگر از همه‌ی امتیازها یک عدد ثابت (مثلاً `max(scores)`) کم کنیم، خروجی softmax تغییر نمی‌کند، و تابع را با این ایده اصلاح کنید.

▼ **پاسخ**

الف) مجموع نماها `≈ 11.212`؛ پس احتمال‌ها `[0.659, 0.242, 0.099]` و مجموعشان `1` است. `argmax` هر دو `0` است: softmax ترتیب را حفظ می‌کند، پس کلاس پیش‌بینی‌شده عوض نمی‌شود.

ب) `1` (اولین بیشینه، چون شرط `>` سخت‌گیرانه است). با `>=` نتیجه `2` می‌شود (آخرین بیشینه).

ج) `e^(s - c) / Σ e^(s_k - c) = (e^s · e^-c) / (e^-c · Σ e^(s_k)) = e^s / Σ e^(s_k)`؛ ضریب `e^-c` از صورت و مخرج حذف می‌شود.

```
def softmax(scores):
    m = max(scores)
    exps = []
    for s in scores:
        exps.append(math.exp(s - m))
    total = sum(exps)
    probs = []
    for e in exps:
        probs.append(e / total)
    return probs
```

با این کار بزرگ‌ترین نما `e^0 = 1` است و سرریز رخ نمی‌دهد. کتابخانه‌های یادگیری عمیق دقیقاً همین ترفند را به کار می‌برند.

سؤال ۶ — ارزیابی دسته‌بند

برچسب‌های واقعی و پیش‌بینی‌های یک دسته‌بند دودویی را داریم:

```
y_true = [1, 0, 1, 1, 0, 1, 0, 0]
y_pred = [1, 1, 0, 1, 0, 1, 1, 0]
```

الف) با یک حلقه روی `zip(y_true, y_pred)`، چهار شمارنده‌ی **ماتریس درهم‌ریختگی** را حساب کنید: TP (واقعی ۱، پیش‌بینی ۱)، FP (واقعی ۰، پیش‌بینی ۱)، FN (واقعی ۱، پیش‌بینی ۰) و TN (واقعی ۰، پیش‌بینی ۰). کد را بنویسید و مقدارها را گزارش کنید.

ب) **دقت (accuracy)** `(TP + TN) / n`، **precision** `TP / (TP + FP)` و **recall** `TP / (TP + FN)` را حساب کنید.

ج) مدلی که **همیشه** `0` پیش‌بینی کند، روی داده‌ای که ۹۵٪ آن کلاس `0` است، چه accuracy و چه recallای دارد؟ چه نتیجه‌ای می‌گیرید؟

▼ پاسخ

الف)

```
tp = fp = fn = tn = 0
for t, p in zip(y_true, y_pred):
    if t == 1 and p == 1:
        tp += 1
    elif t == 0 and p == 1:
        fp += 1
    elif t == 1 and p == 0:
        fn += 1
    else:
        tn += 1
```

`TP = 3`، `FP = 2`، `FN = 1`، `TN = 2`.

ب) recall `= 3 / 5 = 0.6`، precision `= 5 / 8 = 0.625`، accuracy `= 3 / 4 = 0.75`.

ج) accuracy `0.95` ولی recall `0` (هیچ نمونه‌ی مثبتی را پیدا نمی‌کند). روی داده‌ی **نامتوازن**، accuracy به‌تنهایی گمراه‌کننده است و باید precision و recall را هم گزارش کرد.

سؤال ۷ — ماتریس‌ها

الف) خروجی کد زیر چیست؟ چرا؟

```
weights = [[0.0] * 2] * 3
weights[1][0] = 0.5
print(weights)
```

ب) تابع ضرب ماتریسی زیر را کامل کنید. `A` ماتریس `n × k` و `B` ماتریس `k × m` است و حاصل `n × m` است. حاصل `matmul([[1, 2], [3, 4]], [[5, 6], [7, 8]])` را هم حساب کنید.

```
def matmul(A, B):
    n, k, m = len(A), len(B), len(B[0])
    C = []
```

```
    for i in range(n):
        row = []
        for j in range(m):
            s = 0
            for t in range(______):
                s += ______
            row.append(s)
        C.append(______)
    return C
```

ج) بدنه‌ی درونی‌ترین حلقه برای ضرب یک ماتریس `100 × 50` در یک ماتریس `50 × 20` چند بار اجرا می‌شود؟

▼ **پاسخ**

الف) `[[0.5, 0.0], [0.5, 0.0], [0.5, 0.0]]` . هر سه سطر به **یک** لیست درونی اشاره می‌کنند؛ تغییر «سطر ۱» در همه دیده می‌شود.

ب)

```
for t in range(k):
    s += A[i][t] * B[t][j]
...
C.append(row)
```

حاصل: `[[19, 22], [43, 50]]` (مثلاً `1*5 + 2*7 = 19`).

ج) `n × m × k = 100 × 20 × 50 = 100,000` بار. لایه‌ی کاملاً متصل (fully connected) در یک شبکه‌ی عصبی دقیقاً همین ضرب ماتریسی است؛ برای همین در عمل از NumPy (بخش ۳) استفاده می‌کنیم که همین محاسبه را صدها برابر سریع‌تر انجام می‌دهد.

سؤال ۸ — تقسیم آموزش و آزمون

الف) کد زیر دو اشکال دارد. خط آخر چه خطایی می‌دهد؟ اشکال دوم (که پیام خطا ندارد) چیست؟

```
import random

data = list(range(10))
random.seed(1)
shuffled = random.shuffle(data)
train = shuffled[:8]
```

ب) در یادگیری ماشین معمولاً ویژگی‌ها (`X`) و برچسب‌ها (`y`) در **دو لیست جدا** هستند و عضو `i` ام آن‌ها به هم مربوط است. تابع زیر را طوری کامل کنید که داده را به‌طور تصادفی تقسیم کند، **جفت‌ها را به هم نریزد** و لیست‌های ورودی را **تغییر ندهد**.

```
import random

def train_test_split(X, y, test_ratio=0.2, seed=None):
```

```
    random.seed(seed)
    idx = list(range(len(X)))
    random._____(idx)
    n_test = int(_____)
    X_train, X_test, y_train, y_test = [], [], [], []
    for k in range(len(idx)):
        i = idx[k]
        if k < n_test:
            X_test.append(X[i])
            y_test.append(_____)
        else:
            X_train.append(_____)
            y_train.append(y[i])
    return X_train, X_test, y_train, y_test
```

ج) چرا به جای به‌هم‌ریختن **اندیس‌ها**، دو بار `random.shuffle(X)` و `random.shuffle(y)` را صدا نمی‌زنیم؟

د) اگر `train_test_split(X, y, seed=7)` را دو بار صدا بزنیم، نتیجه‌ها یکسان‌اند؟ با `seed=None` چطور؟

▼ **پاسخ**

الف) `random.shuffle` در جا کار می‌کند و `None` برمی‌گرداند؛ پس `shuffled` برابر `None` است و `shuffled[:8]` خطای `TypeError: 'NoneType' object is not subscriptable` می‌دهد. اشکال دوم: `data` اصلی بدون اینکه کپی گرفته شود به هم ریخته شده است. اصلاح:

```
shuffled = data.copy()
random.shuffle(shuffled)
train = shuffled[:8]
```

ب)

```
random.shuffle(idx)
n_test = int(test_ratio * len(X))
y_test.append(y[i])
X_train.append(X[i])
```

ج) دو `shuffle` جداگانه، `X` و `y` را با **دو جایگشت متفاوت** به هم می‌ریزند و ارتباط هر نمونه با برچسبش از بین می‌رود (و ورودی‌ها هم در جا تغییر می‌کنند). با به‌هم‌ریختن یک لیست اندیس و استفاده از **همان** اندیس برای هر دو لیست، جفت‌ها حفظ می‌شوند.

د) با `seed=7` هر دو بار دقیقاً همان تقسیم به دست می‌آید. `random.seed(None)` نقطه‌ی شروع را از منبع تصادفی سیستم‌عامل می‌گیرد؛ پس هر بار نتیجه فرق دارد. (تابع `train_test_split` در scikit-learn دقیقاً همین ورودی و خروجی‌ها را دارد و پارامتر `random_state` آن همین نقش seed را بازی می‌کند.)

خلاصه‌ی جلسه

- لیست مجموعه‌ای مرتب و **تغییرپذیر** از مقدارهاست؛ اندیس از صفر شروع می‌شود و اندیس منفی از انتها می‌شمارد.
- برش `lst[start:stop:step]` لیست جدید می‌سازد و قواعدش مثل `range` است.
- متدهایی مثل `append`، `sort` و `reverse` در جا تغییر می‌دهند و `None` برمی‌گردانند؛ `sorted` و برش لیست جدید می‌سازند.
- `b = a` کپی نیست؛ با `copy()` کپی بگیرید. لیستی که به تابع داده می‌شود، کپی نمی‌شود.
- با `for`، `range(len(...))`، `enumerate` و `zip` پیمایش کنید؛ برای فیلتر و تبدیل، لیست جدید بسازید.
- لیست تودرتو یعنی ماتریس: `m[i][j]`؛ مراقب `[[0] * n] * m` باشید.

جلسه‌ی بعد: تاپل‌ها و مجموعه‌ها.

جلسه ۹: تاپل‌ها و مجموعه‌ها

Tuples & Sets

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- **تاپل** بسازید، آن را باز کنید (unpacking) و توضیح دهید چرا و کجا به جای لیست از تاپل استفاده می‌کنیم،
- مفهوم **تغییرناپذیری** را دقیق بفهمید؛ از جمله وقتی یک تاپل شامل یک لیست است،
- با لیستی از تاپل‌ها مثل **جدولی از رکوردها** کار کنید و آن را مرتب کنید،
- **مجموعه** بسازید و از آن برای حذف تکراری‌ها، بررسی سریع عضویت و عملیات اجتماع، اشتراک و تفاضل استفاده کنید،
- تشخیص دهید چه مقدارهایی می‌توانند عضو مجموعه باشند و چرا.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. تاپل و ویژگی‌های آن
2. باز کردن تاپل (unpacking)
3. کاربردهای تاپل
4. مجموعه (set)
5. عملیات روی مجموعه‌ها

۱. تاپل و ویژگی‌های آن

💡 مفهوم

تاپل (tuple) مثل لیست یک دنباله‌ی **مرتب** از مقدارهاست، با یک تفاوت اساسی: **تغییرناپذیر (immutable)** است. بعد از ساختن یک تاپل، نمی‌توان عضوی را عوض، اضافه یا حذف کرد. تاپل را با پرانتز می‌نویسیم:

```
t1 = (1, 2, 3, 4)
t2 = ("apple", "banana", "cherry")
t3 = (1, "hello", 3.14, True)
print(t1, t2, t3)

t = (10, 20, 30, 40, 20)
print(t[0], t[-1], t[1:3])          # indexing and slicing, like lists
print(len(t), max(t), min(t))
print(t.count(20), t.index(30))     # the only two tuple methods
print(t1 + (5, 6), (0,) * 3)        # concatenation and repetition
```

🔍 کد را بخوان

- اندیس‌گذاری، برش، `len` ، `max` ، `min` ، `in` ، `+` و `*` دقیقاً مثل لیست کار می‌کنند (جلسه‌ی ۸).
- تاپل فقط **دو** متد دارد: `count` و `index` . متدهایی مثل `append` ، `remove` یا `sort` وجود ندارند، چون همه‌ی آن‌ها شیء را تغییر می‌دادند.
- `t1 + (5, 6)` تاپل `t1` را تغییر نمی‌دهد؛ یک تاپل **جدید** می‌سازد.

⚠️ تلاش برای تغییر

```
t = (1, 2, 3)
t[0] = 100
```

🔍 خطای `TypeError: 'tuple' object does not support item assignment` یعنی تاپل اجازه‌ی انتساب به عضو را نمی‌دهد.

⚠️ اشتباه رایج: تاپل یک‌عضوی

کاما است که تاپل می‌سازد، نه پرانتز. پرانتز تنها فقط برای گروه‌بندی عبارت‌هاست:

❓ پیش‌بینی کن

نوع هر مقدار چیست؟

```
a = (5)
b = (5,)
c = 5, 6
d = ()
print(type(a), type(b), type(c), type(d))
```

🔍 `(5)` همان عدد `5` است. برای تاپل یک‌عضوی باید بعد از عضو کاما بگذارید: `(5,)` . حتی بدون پرانتز، `5, 6` یک تاپل است. `()` تاپل خالی است.

💡 تغییرناپذیری «سطحی» است

تاپل نمی‌گذارد **خانه‌هایش** به شیء دیگری اشاره کنند؛ اما اگر یکی از اعضا خودش **تغییرپذیر** باشد (مثلاً یک لیست)، محتوای آن عضو را می‌توان تغییر داد:

```
config = ("relu", [64, 32])
config[1].append(16)          # allowed: we change the list, not the tuple
print(config)
```

🔍 تاپل همچنان به **همان** لیست اشاره می‌کند (پیکان‌ها جابه‌جا نشده‌اند؛ شکل جلسه‌ی ۸ را به خاطر بیاورید)، اما خود لیست تغییر کرده است. پس «تاپل تغییرناپذیر است» به این معنی نیست که هر چیزی داخل آن ثابت می‌ماند.

۲. باز کردن تاپل (unpacking)

💡 مفهوم

اعضای یک تاپل (یا هر دنباله‌ای) را می‌توان در یک خط در چند متغیر ریخت. این کار را از جلسه‌ی ۲ (انتساب چندگانه) و جلسه‌ی ۷ (برگرداندن چند مقدار) دیده‌اید؛ حالا می‌دانید پشت آن یک تاپل است:

```
point = (3, 4)
x, y = point
print(x, y)

def min_max(values):
    return min(values), max(values)      # returns a tuple

lo, hi = min_max([4, 7, 1, 9, 3])
print(lo, hi)

a, b = 1, 2
a, b = b, a                              # swap: the right side is a tuple
print(a, b)
```

💡 باز کردن با `*`

اگر تعداد متغیرها با تعداد اعضا برابر نباشد، خطای `ValueError` می‌گیرید. با `*` می‌توانید «بقیه‌ی اعضا» را در یک **لیست** جمع کنید:

❓ پیش‌بینی کن

```
first, *rest = (10, 20, 30, 40)
print(first, rest)

a, *middle, z = range(6)
print(a, middle, z)

header, *rows = ["name,score", "Ali,17", "Sara,19"]
print(header, rows)
```

🔍 `*rest` همه‌ی اعضای باقی‌مانده را به شکل لیست می‌گیرد. مثال آخر الگوی رایجی برای جدا کردن **سطر عنوان** یک فایل CSV از سطرهای داده است.

💡 باز کردن در حلقه

وقتی روی **لیستی از تاپل‌ها** حلقه می‌زنید، می‌توانید هر تاپل را مستقیماً در متغیر حلقه باز کنید (همان کاری که با `enumerate` و `zip` در جلسه‌ی ۸ کردید):

```
records = [("Ali", 17.5), ("Sara", 19), ("Reza", 12)]
for name, score in records:
    print(f"{name:<6}{score:>6.2f}")
```

۳. کاربردهای تاپل

💡 چه وقت تاپل و چه وقت لیست؟

	لیست	تاپل
تغییرپذیر؟	بله	خیر
کاربرد معمول	مجموعه‌ای از **چیزهای هم‌نوع** که تعدادشان تغییر می‌کند (نمره‌های یک کلاس)	یک **رکورد** با ساختار ثابت که هر جایگاهش معنی خاصی دارد (نام، سن، شهر)
مثال	`scores = [17, 12, 19]`	`student = ("Ali", 21, "Bushehr")`
کلید دیکشنری یا عضو مجموعه	نمی‌تواند باشد	می‌تواند باشد

یک قاعده‌ی سرانگشتی: اگر می‌توانید جمله‌ی «**هر جایگاه** این دنباله معنی مشخصی دارد» را بگویید، تاپل مناسب‌تر است. در علوم داده هم شکل یک ماتریس یا جدول به شکل تاپل گزارش می‌شود؛ مثلاً NumPy و Pandas ابعاد داده را به شکل `(1000, 20)` (۱۰۰۰ سطر، ۲۰ ستون) می‌دهند.

💡 مرتب‌سازی تاپل‌ها

تاپل‌ها (و لیست‌ها) **عضو به عضو** مقایسه می‌شوند، مثل کلمات در فرهنگ لغت (جلسه‌ی ۳): اول عضو اول؛ اگر برابر بود، عضو دوم؛ و ... . این ویژگی مرتب‌سازی رکوردها را ساده می‌کند:

```
print((1, 9) < (2, 0))
print((2, "b") < (2, "a"))

results = [(17, "Sara"), (15, "Ali"), (17, "Reza"), (19, "Mina")]
for score, name in sorted(results, reverse=True):
    print(name, score)
```

🔍 کد را بخوان

- `(1, 9) < (2, 0)` درست است، چون عضو اول تعیین‌کننده است و `1 < 2` .
- در `results` نمره را **اول** گذاشته‌ایم تا مرتب‌سازی بر اساس نمره انجام شود. دو نفر با نمره‌ی `17` بر اساس عضو **دوم** (نام) مرتب می‌شوند؛ با `reverse=True` ، `Sara` قبل از `Reza` می‌آید.

> مرتب کردن بر اساس یک عضو دلخواه (بدون جابه‌جا کردن ترتیب اعضای تاپل) را در جلسه‌ی ۱۳ با پارامتر `key` می‌بینیم.

❓ پیش‌بینی کن

خروجی کد زیر چیست؟ چرا «بیشترین» و «کمترین» در این مثال معنی خاصی دارند؟

```
readings = [("2026-10-28", 31.5), ("2026-10-29", 29.0), ("2026-10-30", 33.2)]
print(max(readings))
print(min(readings))
```

🔍 `max` و `min` هم عضو به عضو مقایسه می‌کنند؛ پس اینجا بر اساس **تاریخ** (رشته‌ای که به ترتیب سال، ماه، روز نوشته شده) انتخاب می‌کنند، نه بر اساس دما. `max` آخرین روز و `min` اولین روز را برمی‌گرداند؛ گرم‌ترین روز (`33.2`) تصادفاً همان آخرین روز است، ولی کمترین دما (`29.0`) مال `min(readings)` نیست. اگر دما را جایگاه اول می‌گذاشتیم، نتیجه فرق می‌کرد.

۴. مجموعه (set)

💡 مفهوم

مجموعه (set) گردآوری‌ای از مقدارهاست که:

- **تکراری ندارد:** هر مقدار حداکثر یک بار در آن هست.
- **ترتیب ندارد:** اندیس ندارد و `s[0]` معنی ندارد.
- **عضویت را بسیار سریع بررسی می‌کند:** `x in s` تقریباً مستقل از اندازه‌ی مجموعه، فوراً جواب می‌دهد.

مجموعه را با `{ }` یا با تابع `set()` می‌سازیم:

```
s1 = {1, 2, 3, 4}
s2 = set([1, 2, 2, 3, 4, 4, 4])     # duplicates removed
s3 = set("banana")
print(s1, s2, s3)
print(len(s2))

empty = set()                        # NOT {}  -- that is an empty dictionary
print(type(empty), type({}))
```

🔍 کد را بخوان

- `set([1, 2, 2, 3, 4, 4, 4])` تکراری‌ها را حذف می‌کند: `{1, 2, 3, 4}` .
- `set("banana")` حروف **متمایز** کلمه است. ترتیب نمایش آن ممکن است با ترتیب حروف کلمه فرق کند؛ مجموعه ترتیب ندارد و نباید به ترتیب نمایش آن تکیه کنید.
- `{}` یک **دیکشنری** خالی است (جلسه‌ی ۱۰)، نه مجموعه‌ی خالی.

💡 افزودن و حذف

```
fruits = {"apple", "banana"}
fruits.add("cherry")
fruits.add("apple")            # already there: nothing happens
print(fruits)

fruits.remove("banana")        # KeyError if not present
fruits.discard("kiwi")         # no error if not present
print(fruits)
print("apple" in fruits, "kiwi" not in fruits)
```

💡 کاربرد رایج: مقدارهای یکتا

در علوم داده بارها می‌پرسیم «این ستون چند مقدار **متمایز** دارد؟» یا «چه کلاس‌هایی در داده هست؟»:

```
labels = ["cat", "dog", "cat", "bird", "dog", "cat"]
classes = set(labels)
print(classes, len(classes))
print(sorted(classes))           # sorted() returns a list
```

🔍 `sorted` روی مجموعه یک **لیست** مرتب برمی‌گرداند؛ این راه معمول برای داشتن یک ترتیب ثابت از اعضای یک مجموعه است.

⚠️ چه چیزی می‌تواند عضو مجموعه باشد؟

مجموعه برای جست‌وجوی سریع، از هر عضو یک «اثر انگشت» عددی (**hash**) می‌سازد. این اثر انگشت فقط برای مقدارهای **تغییرناپذیر** معتبر می‌ماند؛ پس اعداد، رشته‌ها و تاپل‌ها (از مقدارهای تغییرناپذیر) می‌توانند عضو مجموعه باشند، اما **لیست نمی‌تواند**:

```
points = {(0, 0), (1, 2), (0, 0)}
print(points)

bad = {[1, 2], [3, 4]}
```

🔍 خطای `TypeError: unhashable type: 'list'` یعنی لیست اثر انگشت ندارد. برای ذخیره‌ی نقطه‌ها یا رکوردها در مجموعه، از **تاپل** استفاده کنید. (همین قاعده برای **کلیدهای دیکشنری** در جلسه‌ی ۱۰ هم برقرار است.)

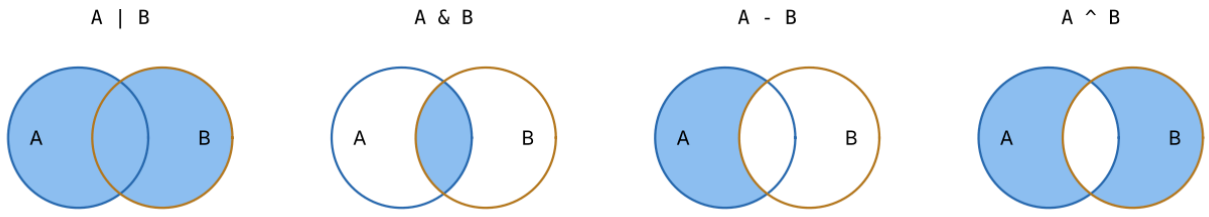
⚠️ اشتباه رایج: حذف تکراری‌ها با از دست دادن ترتیب

`list(set(seq))` تکراری‌ها را حذف می‌کند، اما **ترتیب** اولیه را تضمین نمی‌کند. اگر ترتیب مهم است، از الگوی «دیده‌شده‌ها» استفاده کنید (سؤال ۵).

۵. عملیات روی مجموعه‌ها

💡 مفهوم

مجموعه‌های پایتون همان مجموعه‌های ریاضی هستند و عملیات آن‌ها را پشتیبانی می‌کنند:



چهار عمل اصلی مجموعه‌ها: اجتماع، اشتراک، تفاضل و تفاضل متقارن (بخش رنگی، نتیجه است).

عملگر	متد	معنی
`A | B`	`A.union(B)`	اجتماع: عضوهای A **یا** B
`A & B`	`A.intersection(B)`	اشتراک: عضوهای A **و** B
`A - B`	`A.difference(B)`	تفاضل: در A هست ولی در B نیست
`A ^ B`	`A.symmetric_difference(B)`	تفاضل متقارن: دقیقاً در یکی از دو مجموعه
`A <= B`	`A.issubset(B)`	آیا A زیرمجموعه‌ی B است؟

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B, B - A)
print("Symmetric difference:", A ^ B)
print({1, 2} <= A, A <= B)
```

🔍 کد را بخوان: کاربرد در داده

فرض کنید شناسه‌ی دانشجویانی را که در دو درس ثبت‌نام کرده‌اند دارید:

```
python_course = {"s01", "s02", "s03", "s05", "s08"}
stats_course = {"s02", "s03", "s04", "s08", "s09"}

both = python_course & stats_course
only_python = python_course - stats_course
everyone = python_course | stats_course

print("both:", sorted(both))
print("only python:", sorted(only_python))
print("total students:", len(everyone))
```

- اشتراک: چه کسانی **هر دو** درس را گرفته‌اند.
- تفاضل: چه کسانی **فقط** پایتون را گرفته‌اند (ترتیب مهم است: `A - B` با `B - A` فرق دارد).
- اجتماع: تعداد کل دانشجویان **متمایز**؛ اگر دو لیست را با `+` به هم می‌چسباندیم، دانشجویان مشترک دو بار شمرده می‌شدند.

❓ پیش‌بینی کن

بدون اجرا، خروجی هر خط را بنویسید.

```
X = set("data")
Y = set("science")
print(sorted(X & Y))
print(len(X | Y))
print(sorted(Y - X))
print(X ^ Y == (X | Y) - (X & Y))
```

🔍 `X = {'d', 'a', 't'}` و `Y = {'s', 'c', 'i', 'e', 'n'}` (حرف تکراری فقط یک بار حساب می‌شود). اشتراک خالی است (`[]`)، اجتماع `8` عضو دارد، `Y - X` همان کل `Y` است، و خط آخر یک اتحاد همیشه درست را بررسی می‌کند: `True` .

📌 آنچه باید بلد باشید

- تاپل مرتب و تغییرناپذیر است؛ فقط `count` و `index` دارد؛ `+` و `*` تاپل جدید می‌سازند.
- `(5)` عدد است و `(5,)` تاپل؛ کاما تاپل می‌سازد.
- تاپلی که لیست دارد: خود لیست قابل تغییر است.
- باز کردن تاپل، از جمله با `*` ، و باز کردن در حلقه‌ی `for` .
- انتخاب بین لیست و تاپل؛ مقایسه و مرتب‌سازی تاپل‌ها **عضو به عضو**.
- مجموعه: بدون تکرار، بدون ترتیب، بدون اندیس، عضویت سریع؛ `set()` برای مجموعه‌ی خالی.
- `add` ، `remove` (خطا اگر نباشد)، `discard` (بدون خطا).
- فقط مقدارهای تغییرناپذیر (hashable) می‌توانند عضو مجموعه باشند.
- `|` ، `&` ، `-` ، `^` ، `<=` و کاربرد آن‌ها روی داده‌ی واقعی.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — تاپل‌ها

خروجی هر خط را بنویسید. اگر خطا رخ می‌دهد، نوع خطا را بنویسید. (خط‌ها را **پشت سر هم** در نظر بگیرید.)

```
t = (1, [2, 3], "ab")
t[1].append(4)
print(t)
t[1] = t[1] + [5]
print(len(((),)), len((1, 2) * 3))
a, *b, c = range(6)
print(a, b, c)
x, y = (1, 2, 3)
```

▼ **پاسخ**

خط	نتیجه
`t[1].append(4)`	مجاز؛ خود لیست تغییر می‌کند
`print(t)`	`(1, [2, 3, 4], 'ab')`
`t[1] = t[1] + [5]`	`TypeError` : تاپل انتساب به عضو را نمی‌پذیرد (`t[1] + [5]` یک لیست **جدید** می‌سازد و باید به خانه‌ی `t[1]` انتساب شود)
`print(len(((),)), ...)`	`1 6` : تاپلی که تنها عضوش یک تاپل خالی است، و تاپل شش‌عضوی
`print(a, b, c)`	`0 [1, 2, 3, 4] 5`
`x, y = (1, 2, 3)`	`ValueError: too many values to unpack`

نکته: فرق `t[1].append(4)` و `t[1] = t[1] + [5]` همان فرق «تغییر در جا» و «ساختن شیء جدید» در جلسه‌ی ۸ است.

سؤال ۲ — k نزدیک‌ترین همسایه (kNN)

الگوریتم **k نزدیک‌ترین همسایه** برای دسته‌بندی یک نقطه‌ی جدید، `k` نقطه‌ی نزدیک‌تر از داده‌ی آموزش را پیدا می‌کند و برچسبی را انتخاب می‌کند که بین آن‌ها **بیشترین تکرار** را دارد. داده‌ی آموزش، لیستی از تاپل‌های `(features, label)` است:

```
train = [((1, 1), "A"), ((2, 1), "A"), ((4, 4), "B"),
         ((5, 4), "B"), ((1, 2), "A"), ((4, 5), "B")]

def distance(p, q):
    s = 0
    for a, b in zip(p, q):
        s += (a - b) ** 2
    return s ** 0.5

def knn_predict(train, query, k):
    pairs = []
    for features, label in train:
        pairs.append((distance(features, query), label))
    nearest = sorted(pairs)[:k]
    votes = []
    for d, label in nearest:
        votes.append(label)
    best = None
    for c in sorted(set(votes)):
        if best is None or votes.count(c) > votes.count(best):
```

```
            best = c
    return best
```

الف) برای `query = (3, 3)` فاصله‌ی هر نقطه‌ی آموزش را حساب کنید (با سه رقم اعشار) و لیست مرتب‌شده‌ی `pairs` را بنویسید.

ب) خروجی `knn_predict(train, (3, 3), k)` را برای `k = 1`، `k = 3` و `k = 5` بنویسید.

ج) نتیجه‌ی `k = 3` به یک «تصادف» بستگی دارد. آن تصادف چیست؟ (راهنمایی: چهار نقطه فاصله‌ی **دقیقاً برابر** دارند. `sorted` بین آن‌ها چطور تصمیم می‌گیرد؟) اگر برچسب‌ها `"A"` و `"B"` با هم عوض شوند (یعنی کلاس اول `"B"` نام بگیرد)، پیش‌بینی برای `k = 3` چه می‌شود؟ چه نتیجه‌ای می‌گیرید؟

▼ **پاسخ**

الف) فاصله تا `(3, 3)`: `(1, 1)` : `√8 ≈ 2.828`؛ `(2, 1)` : `√5 ≈ 2.236`؛ `(4, 4)` : `√2 ≈ 1.414`؛ `(5, 4)` : `√5`؛ `(1, 2)` : `√5`؛ `(4, 5)` : `√5`.

```
[(1.414, 'B'), (2.236, 'A'), (2.236, 'A'), (2.236, 'B'), (2.236, 'B'),
(2.828, 'A')]
```

ب) `k = 1` : `B`. `k = 3` : رأی‌ها `B, A, A` پس `A`. `k = 5` : رأی‌ها `B, A, A, B, B` پس `B`.

ج) چهار نقطه با فاصله‌ی برابر `√5` وجود دارد و فقط دو تای آن‌ها در سه همسایه‌ی نزدیک جا می‌شوند. چون تاپل‌ها عضو به عضو مقایسه می‌شوند، در فاصله‌ی برابر، **برچسب** تعیین‌کننده است و `"A"` (به خاطر ترتیب الفبایی) جلوتر می‌آید. اگر نام کلاس‌ها عوض شود، این بار دو نقطه‌ی گروه دیگر (`(5, 4)` و `(4, 5)`) جلو می‌افتند و پیش‌بینی برای `k = 3` به گروه نقطه‌ی `(4, 4)` تغییر می‌کند. یعنی پیش‌بینی به **اسم** کلاس‌ها بستگی دارد که اصلاً نباید اهمیتی داشته باشد! پیاده‌سازی‌های واقعی برای حالت تساوی قاعده‌ی صریح دارند (مثلاً در نظر گرفتن ترتیب داده، یا انتخاب `k` فرد و وزن‌دهی بر اساس فاصله).

سؤال ۳ — واژگان و شباهت متن

برای کار با متن در یادگیری ماشین، اول باید **واژگان (vocabulary)** را بسازیم:

```
docs = ["the cat sat", "the cat ran", "a dog ran"]
```

الف) با یک حلقه و عملگر `|`، مجموعه‌ی همه‌ی کلمات (`vocab`) را بسازید. اندازه‌ی آن چند است؟ `sorted(vocab)` چیست؟

ب) **شباهت ژاکارد** دو مجموعه برابر است با اندازه‌ی اشتراک تقسیم بر اندازه‌ی اجتماع. تابع `jaccard(a, b)` را بنویسید و شباهت هر سه جفت سند را (بر اساس مجموعه‌ی کلماتشان) حساب کنید.

ج) با استفاده از `sorted(vocab)` به‌عنوان ترتیب ثابت ستون‌ها، هر سند را به یک بردار **کیسه‌ی کلمات (bag of words)** تبدیل کنید: عضو `j` ام بردار، تعداد دفعات تکرار کلمه‌ی `j` ام در سند است. بردار سند اول چیست؟ چرا ترتیب ثابت ستون‌ها لازم است و چرا نمی‌توانستیم مستقیم از ترتیب اعضای `vocab` استفاده کنیم؟

▼ پاسخ

الف)

```
vocab = set()
for d in docs:
    vocab = vocab | set(d.split())
```

اندازه `6`؛ `['a', 'cat', 'dog', 'ran', 'sat', 'the']`.

ب)

```
def jaccard(a, b):
    return len(a & b) / len(a | b)
```

جفت	اشتراک	اجتماع	ژاکارد
سند ۱ و ۲	`{the, cat}`	۴ کلمه	`0.5`
سند ۱ و ۳	تهی	۶ کلمه	`0.0`
سند ۲ و ۳	`{ran}`	۵ کلمه	`0.2`

ج)

```
columns = sorted(vocab)
vector = []
for w in columns:
    vector.append(docs[0].split().count(w))
```

بردار سند اول: `[0, 1, 0, 0, 1, 1]`. هر عضو بردار باید در **همه‌ی** سندها به یک کلمه‌ی ثابت اشاره کند تا بردارها با هم قابل مقایسه باشند (مثلاً برای ضرب داخلی یا شباهت کسینوسی جلسه‌ی ۸). مجموعه ترتیب تضمین‌شده ندارد و حتی اندیس‌گذاری را پشتیبانی نمی‌کند.

سؤال ۴ — نشت داده

شناسه‌ی نمونه‌های بخش آموزش و آزمون یک پروژه را داریم:

```
train_ids = ["p01", "p02", "p03", "p04", "p05", "p03", "p06"]
test_ids  = ["p07", "p08", "p02", "p09"]
train_labels = {"cat", "dog"}
test_labels  = {"cat", "dog", "bird"}
```

با عملیات مجموعه‌ها، به این سؤال‌ها جواب دهید (کد و نتیجه):

الف) آیا نمونه‌ای هم در آموزش و هم در آزمون آمده است؟ (به این مشکل **نشت داده** می‌گویند و ارزیابی مدل را بیش از حد خوش‌بینانه می‌کند.)

ب) آیا داده‌ی آموزش نمونه‌ی **تکراری** دارد؟ چند تا؟

ج) کدام کلاس در آزمون هست ولی مدل هرگز در آموزش آن را ندیده است؟ چرا این مشکل است؟

▼ پاسخ

الف) `set(train_ids) & set(test_ids)` برابر `{'p02'}` است؛ پس بله، `p02` نشت کرده است.

ب) `len(train_ids) - len(set(train_ids))` برابر `7 - 6 = 1`؛ `p03` دو بار آمده است.

ج) `test_labels - train_labels` برابر `{'bird'}` است. مدلی که هیچ نمونه‌ای از `bird` ندیده، نمی‌تواند آن را پیش‌بینی کند؛ همه‌ی نمونه‌های `bird` در آزمون حتماً اشتباه دسته‌بندی می‌شوند.

سؤال ۵ — حذف تکراری‌ها با حفظ ترتیب

دو تابع زیر را مقایسه کنید:

```python
def unique_a(seq):
    result = []
    for x in seq:
        if x not in result:
            result.append(x)
    return result

def unique_b(seq):
    seen = set()
    result = []
    for x in seq:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
```

الف) خروجی هر دو تابع برای `[3, 1, 3, 2, 1, 5]` چیست؟ خروجی `list(set([3, 1, 3, 2, 1, 5]))` چه فرقی با آن‌ها می‌تواند داشته باشد؟

ب) هر دو تابع درست‌اند، اما برای یک لیست با یک میلیون عضو متمایز، `unique_a` عملاً تمام نمی‌شود و `unique_b` در کسری از ثانیه تمام می‌شود. چرا؟ تعداد تقریبی مقایسه‌ها را برای هر کدام تخمین بزنید.

▼ پاسخ

الف) هر دو `[3, 1, 2, 5]`: اولین وقوع هر مقدار، به ترتیب ظاهر شدن. `list(set(...))` همان اعضا را دارد اما ترتیبشان تضمین‌شده نیست (مثلاً ممکن است `[1, 2, 3, 5]` باشد).

ب) در `unique_a`، عبارت `x not in result` روی یک **لیست** اجرا می‌شود و باید عضوها را یکی‌یکی بررسی کند؛ برای `n` عضو متمایز، حدود `1 + 2 + ... + n` یعنی حدود `n² / 2` مقایسه (برای یک میلیون، حدود ۵ × ۱۰¹¹). در `unique_b`، بررسی عضویت در **مجموعه** تقریباً یک قدم است؛ پس کل کار حدود `n` قدم است. انتخاب ساختار داده‌ی درست، سرعت را میلیون‌ها برابر تغییر می‌دهد.

سؤال ۶ — چه چیزی hashable است؟

کدام خط‌ها اجرا می‌شوند و کدام خطا می‌دهند؟ برای خطاها دلیل بیاورید. اندازه‌ی مجموعه‌هایی را که ساخته می‌شوند بنویسید.

```
s1 = {(1, 2), (2, 1), (1, 2)}
s2 = {"ab", "ba", "a" + "b"}
s3 = {(1, [2])}
s4 = {1, 1.0, True}
s5 = set([[1, 2], [3]])
```

▼ پاسخ

خط	نتیجه
`s1`	اجرا می‌شود؛ ۲ عضو (`(1, 2)` تکراری است؛ `(2, 1)` ترتیب متفاوتی دارد)
`s2`	اجرا می‌شود؛ ۲ عضو (`"a" + "b"` همان `"ab"` است)
`s3`	`TypeError: unhashable type: 'list'` ؛ تاپلی که لیست دارد، hashable نیست
`s4`	اجرا می‌شود؛ ۱ عضو! `1` ، `1.0` و `True` از نظر پایتون برابرند (`1 == 1.0 == True`) و یکی حساب می‌شوند
`s5`	`TypeError` : اعضای لیست ورودی، خودشان لیست‌اند

نکته‌ی `s4` در داده‌ی واقعی دردسرساز است: اگر ستونی شامل `1` و `True` باشد، شمارش مقدارهای یکتا با مجموعه آن‌ها را یکی می‌کند.

سؤال ۷ — کدگذاری برچسب‌ها

مدل‌های یادگیری ماشین با عدد کار می‌کنند، نه با رشته. جاهای خالی را کامل کنید تا برچسب‌های رشته‌ای به **عدد صحیح** (label encoding) و سپس به بردار **one-hot** تبدیل شوند.

```
labels = ["cat", "dog", "cat", "bird", "dog"]

classes = ______(set(labels))        # fixed, reproducible order
encoded = []
for lbl in labels:
    encoded.append(classes.______(lbl))

one_hot = []
for e in encoded:
    row = [0] * ______
    row[e] = 1
    one_hot.append(row)

print(classes)
```

```
print(encoded)
print(one_hot)
```

بعد از کامل کردن: خروجی را بنویسید. سپس توضیح دهید چرا به یک مدل **نباید** بگوییم `bird = 0` ، `cat = 1` ، `dog = 2` و این اعداد را مثل یک ویژگی عددی معمولی به آن بدهیم، و one-hot چه مشکلی را حل می‌کند.

▼ پاسخ

```
classes = sorted(set(labels))
encoded.append(classes.index(lbl))
row = [0] * len(classes)
```

خروجی:

```
['bird', 'cat', 'dog']
[1, 2, 1, 0, 2]
[[0, 1, 0], [0, 0, 1], [0, 1, 0], [1, 0, 0], [0, 0, 1]]
```

کدگذاری عددی یک **ترتیب و فاصله‌ی ساختگی** ایجاد می‌کند: مدل خطی فکر می‌کند `dog` «دو برابر» `cat` است یا `cat` بین `bird` و `dog` قرار دارد. در one-hot هر کلاس یک بُعد جداگانه دارد و فاصله‌ی همه‌ی کلاس‌ها از هم برابر است. (خروجی softmax در جلسه‌ی ۸ هم دقیقاً با بردار one-hot برچسب واقعی مقایسه می‌شود.)

خلاصه‌ی جلسه

- تاپل یک دنباله‌ی مرتب و تغییرناپذیر است؛ برای رکوردهایی با ساختار ثابت و برگرداندن چند مقدار استفاده می‌شود. تاپل یک‌عضوی کاما لازم دارد.
- باز کردن تاپل (با `*` برای «بقیه») در انتساب و در حلقه بسیار پرکاربرد است.
- تاپل‌ها عضو به عضو مقایسه و مرتب می‌شوند.
- مجموعه: بدون تکرار، بدون ترتیب، با عضویت سریع؛ فقط مقدارهای hashable عضو آن می‌شوند.
- `|` ، `&` ، `-` و `^` اجتماع، اشتراک، تفاضل و تفاضل متقارن‌اند؛ برای مقدارهای یکتا، مقایسه‌ی دو گروه و پیدا کردن تکراری‌ها عالی‌اند.

جلسه‌ی بعد: دیکشنری‌ها.

جلسه ۱۰: دیکشنری‌ها

Dictionaries

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- دیکشنری بسازید و توضیح دهید چرا جست‌وجو با **کلید** از جست‌وجو در لیست سریع‌تر است،
- مقدارها را بخوانید، اضافه، تغییر و حذف کنید و تفاوت `d[key]` و `d.get(key)` را بدانید،
- روی کلیدها، مقدارها و جفت‌ها (`items`) حلقه بزنید،
- الگوهای پرکاربرد **شمارش**، **گروه‌بندی**، **نگاشت (lookup)** و **وارونه‌سازی** را بنویسید و بخوانید،
- داده را به دو شکل «لیستی از دیکشنری‌ها» (سطری) و «دیکشنری از لیست‌ها» (ستونی) نگه دارید؛ همان دو شکلی که Pandas از آن‌ها جدول می‌سازد.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. ساخت دیکشنری
2. دسترسی و به‌روزرسانی
3. متدها و پیمایش
4. الگوهای پرکاربرد
5. دیکشنری‌های تودرتو و داده‌ی جدولی

۱. ساخت دیکشنری

💡 مفهوم

دیکشنری (dictionary) مجموعه‌ای از جفت‌های **کلید: مقدار (key: value)** است. در لیست، به هر عضو با یک **اندیس عددی** (۰، ۱، ۲، ...) دسترسی دارید؛ در دیکشنری، به هر مقدار با یک **کلید معنی‌دار** (مثلاً `"name"` یا `"math"`)

دسترسی پیدا می‌کنید؛ درست مثل یک لغت‌نامه که با «کلمه» به «معنی» می‌رسید.

```
student = {"name": "Ali", "age": 20, "major": "Physics"}
grades = {"math": 18, "physics": 19, "chemistry": 17}
print(student)
print(grades["physics"])
print(len(grades), type(grades))

empty = {}                                  # an empty dict (not a set!)
pairs = dict([("x", 1), ("y", 2)])          # from a list of pairs
zipped = dict(zip(["a", "b", "c"], [1, 2, 3]))
print(empty, pairs, zipped)
```

🔍 کد را بخوان

- هر عضو یک جفت `key: value` است و جفت‌ها با کاما جدا می‌شوند.
- `grades["physics"]` مقدار مربوط به کلید `"physics"` را می‌دهد. اینجا داخل `[ ]` **کلید** می‌آید، نه اندیس.
- `dict(zip(keys, values))` دو لیست هم‌طول را به یک دیکشنری تبدیل می‌کند؛ الگویی بسیار رایج.

💡 قواعد کلید و مقدار

- **کلیدها یکتا هستند.** اگر یک کلید را دو بار بنویسید، مقدار دوم جایگزین اولی می‌شود.
- **کلید باید hashable باشد** (جلسه‌ی ۹): عدد، رشته یا تاپل؛ اما **نه** لیست، مجموعه یا دیکشنری.
- **مقدار** می‌تواند هر چیزی باشد: عدد، رشته، لیست، دیکشنری دیگر، حتی یک تابع.
- از پایتون ۳.۷ به بعد، دیکشنری **ترتیب افزودن** کلیدها را حفظ می‌کند.

💡 چرا دیکشنری سریع است؟

دیکشنری هم مثل مجموعه از **اثر انگشت (hash)** کلیدها استفاده می‌کند؛ پس پیدا کردن مقدار یک کلید، تقریباً مستقل از تعداد اعضا، **فوری** است. پیدا کردن یک عضو در لیست اما ممکن است لازم داشته باشد همه‌ی اعضا را یکی‌یکی بررسی کنید. اگر بارها چیزی را «بر اساس یک شناسه» جست‌وجو می‌کنید، دیکشنری ساختار درست است.

```
D = {"Reiazi": 12, "Amar": 17}
D[["Reiazi", "Amar"]]
```

🔍 کسی که با جدول‌های Pandas کار کرده باشد ممکن است انتظار داشته باشد با یک **لیست** از کلیدها چند مقدار را یک‌جا بگیرد؛ اما در دیکشنری، لیست می‌شود **یک کلید** و چون لیست hashable نیست، خطای `TypeError: unhashable type: 'list'` رخ می‌دهد.

۲. دسترسی و به‌روزرسانی

💡 خواندن: `d[key]` یا `d.get(key)`؟

روش	اگر کلید باشد	اگر کلید نباشد
`d[key]`	مقدار	خطای `KeyError`
`d.get(key)`	مقدار	`None`
`d.get(key, default)`	مقدار	`default`

```
student = {"name": "Ali", "age": 20}
print(student["name"])
print(student.get("major"))
print(student.get("major", "unknown"))
```

```
student["major"]
```

🔍 پیام `KeyError: 'major'` دقیقاً نام کلید گمشده را نشان می‌دهد. وقتی مطمئن نیستید کلید وجود دارد، از `get` با یک مقدار پیش‌فرض استفاده کنید.

💡 افزودن، تغییر و حذف

انتساب به یک کلید، اگر کلید **نباشد** آن را **اضافه** می‌کند و اگر **باشد** مقدارش را **عوض** می‌کند. دیکشنری مثل لیست **تغییرپذیر** است.

```
student = {"name": "Ali", "age": 20}
student["major"] = "Physics"        # add a new key
student["age"] = 21                 # modify an existing key
print(student)

del student["major"]                # remove by key
age = student.pop("age")            # remove and return the value
print(student, "| removed age:", age)

student.update({"age": 22, "city": "Bushehr"})   # add/modify several keys
print(student)
```

❓ پیش‌بینی کن

عملگر `in` روی دیکشنری چه چیزی را بررسی می‌کند؟ هر خط `True` است یا `False`؟

```
D = {"Reiazi": 12, "Amar": 17}
print("Amar" in D)
print(17 in D)
print(17 in D.values())
```

🔍 `in` فقط **کلیدها** را بررسی می‌کند؛ پس `17 in D` نادرست است، چون `17` یک **مقدار** است. برای جست‌وجو در مقدارها باید صریحاً `D.values()` بنویسید (که جست‌وجوی آن، برخلاف کلیدها، سریع نیست).

⚠️ اشتباه رایج

- **دسترسی با اندیس:** `grades[0]` معنی «اولین عضو» نمی‌دهد؛ دنبال کلیدی به نام `0` می‌گردد و `KeyError` می‌دهد.
- **فرض بر وجود کلید:** در داده‌ی واقعی بعضی رکوردها فیلدهایی را ندارند. `record["email"]` برنامه را متوقف می‌کند؛ `record.get("email")` نه.

۳. متدها و پیمایش

💡 سه نمای دیکشنری

متد	چه چیزی می‌دهد
`d.keys()`	همه‌ی کلیدها
`d.values()`	همه‌ی مقدارها
`d.items()`	همه‌ی جفت‌های `(key, value)` به شکل تاپل

این‌ها لیست نیستند، بلکه **نما (view)** هستند؛ یعنی همیشه وضعیت فعلی دیکشنری را نشان می‌دهند. اگر لیست لازم دارید، `list(...)` بنویسید.

```
grades = {"math": 18, "physics": 19, "chemistry": 17}
print(grades.keys())
print(list(grades.values()))
print(list(grades.items()))

for subject in grades:                # looping over a dict gives its KEYS
    print(subject, end=" ")
print()

for subject, score in grades.items():
    print(f"{subject:<10}{score:>3}")
```

🔍 کد را بخوان

- حلقه‌ی `for` مستقیم روی دیکشنری، **کلیدها** را می‌دهد.

- `items()` در هر دور یک تاپل `(key, value)` می‌دهد که با `subject, score` باز می‌شود (جلسه‌ی ۹)؛ رایج‌ترین شکل پیمایش.

❓ پیش‌بینی کن

توابعی مثل `max` ، `sum` و `sorted` روی دیکشنری با کدام بخش کار می‌کنند؟

```
grades = {"math": 18, "physics": 19, "chemistry": 17}
print(max(grades))
print(max(grades.values()))
print(sorted(grades))
print(sum(grades.values()) / len(grades))
```

🔍 `max(grades)` روی **کلیدها** کار می‌کند و بزرگ‌ترین کلید از نظر الفبایی (`"physics"`) را می‌دهد، نه درسی که بیشترین نمره را دارد. برای «درس با بیشترین نمره» باید روی `items()` حلقه بزنید (یا در جلسه‌ی ۱۳ از `key` استفاده کنید).

⚠️ اشتباه رایج: تغییر اندازه‌ی دیکشنری حین پیمایش

```
scores = {"Ali": 8, "Sara": 17, "Reza": 9}
for name in scores:
    if scores[name] < 10:
        del scores[name]
```

🔍 خطای `RuntimeError: dictionary changed size during iteration` ؛ پایتون اجازه نمی‌دهد حین پیمایش، کلیدی اضافه یا حذف کنید (با لیست‌ها در جلسه‌ی ۸ دیدید که بدون خطا نتیجه‌ی غلط می‌گرفتید؛ اینجا حداقل خطا می‌گیرید). راه حل: یک **دیکشنری جدید** از اعضای مطلوب بسازید، یا روی `list(scores)` حلقه بزنید.

۴. الگوهای پرکاربرد

💡 الگوی ۱: شمارش

شمردن تعداد تکرار هر مقدار (فراوانی) شاید رایج‌ترین کار در تحلیل داده باشد:

```
text = "the cat sat on the mat the cat ran"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

🔍 خط کلیدی `counts.get(word, 0) + 1` است: اگر کلمه قبلاً دیده نشده، `get` مقدار `0` را برمی‌گرداند؛ وگرنه شمارش فعلی را. بدون `get` ، اولین بار `counts[word]` خطای `KeyError` می‌داد.

💡 الگوی ۲: گروه‌بندی

جمع کردن اعضا بر اساس یک ویژگی، در **دیکشنری‌ای از لیست‌ها**:

```
students = [("Ali", "A"), ("Sara", "B"), ("Reza", "A"), ("Mina", "C"), ("Hadi",
"B")]
by_grade = {}
for name, grade in students:
    if grade not in by_grade:
        by_grade[grade] = []          # a NEW list for each new key
    by_grade[grade].append(name)
print(by_grade)
```

این همان کاری است که بعدها با `groupby` در Pandas (جلسه‌ی ۲۴) انجام می‌دهید.

💡 الگوی ۳: جدول نگاشت (lookup) و کدگذاری

در جلسه‌ی ۹، برای تبدیل برچسب به عدد از `classes.index(label)` استفاده کردیم که برای هر برچسب کل لیست را جست‌وجو می‌کند. دیکشنری همین کار را فوری انجام می‌دهد:

```
size_mapping = {"XL": 3, "L": 2, "M": 1}
sizes = ["M", "XL", "L", "M"]
encoded = []
for s in sizes:
    encoded.append(size_mapping[s])
print(encoded)
```

💡 الگوی ۴: وارونه‌سازی

برای تبدیل عدد به برچسب (مثلاً برای نمایش خروجی مدل)، دیکشنری را **وارونه** می‌کنیم:

```
inv_size_mapping = {}
for k, v in size_mapping.items():
    inv_size_mapping[v] = k
print(inv_size_mapping)
print(inv_size_mapping[3])
```

⚠️ اشتباه رایج: وارونه‌سازی با مقدارهای تکراری

وارونه‌سازی فقط وقتی بدون از دست دادن اطلاعات کار می‌کند که **مقدارها هم یکتا** باشند. اگر دو کلید مقدار یکسانی داشته باشند، در دیکشنری وارونه فقط **آخرین** آن‌ها باقی می‌ماند (چون کلیدها یکتا هستند).

۵. دیکشنری‌های تودرتو و داده‌ی جدولی

💡 مفهوم

مقدار یک دیکشنری می‌تواند لیست یا دیکشنری دیگری باشد. دو شکل استاندارد برای نگه‌داری یک **جدول داده** وجود دارد:

شکل	ساختار	مثال
سطری (row-oriented)	لیستی از دیکشنری‌ها؛ هر دیکشنری یک رکورد	`[{"name": "Ali", "score": 17}, ...]`
ستونی (column-oriented)	دیکشنری‌ای از لیست‌ها؛ هر کلید یک ستون	`{"name": ["Ali", ...], "score": [17, ...]}`

شکل سطری همان چیزی است که در فایل‌های **JSON** و پاسخ سرویس‌های وب می‌بینید؛ شکل ستونی تقریباً همان چیزی است که `pd.DataFrame` در Pandas (جلسه‌ی ۲۳) می‌سازد.

```
# row-oriented: a list of records
records = [
    {"name": "Ali", "math": 17, "stat": 15},
    {"name": "Sara", "math": 19, "stat": 18},
    {"name": "Reza", "math": 12, "stat": 16},
]
print(records[1]["math"])

# column-oriented: one list per column
table = {
    "name": ["Ali", "Sara", "Reza"],
    "math": [17, 19, 12],
    "stat": [15, 18, 16],
}
print(table["math"][1])
```

🔍 کد را بخوان

هر دو خط چاپ، نمره‌ی ریاضی سارا (`19`) را می‌دهند، اما ترتیب دسترسی برعکس است:

- **سطری:** `records[1]["math"]` یعنی «رکورد دوم، فیلد ریاضی».
- **ستونی:** `table["math"][1]` یعنی «ستون ریاضی، عضو دوم».

هر شکل برای یک کار راحت‌تر است. محاسبه روی یک **ستون** (مثلاً میانگین ریاضی) در شکل ستونی ساده است:

```
means = {}
for column in ["math", "stat"]:
    values = table[column]
    means[column] = sum(values) / len(values)
print(means)
```

❓ پیش‌بینی کن

کد زیر شکل ستونی را به شکل سطری تبدیل می‌کند. خروجی را پیش‌بینی کنید. چرا حلقه‌ی بیرونی روی اندیس سطرهاست؟

```
columns = list(table.keys())
n_rows = len(table["name"])

rows = []
for i in range(n_rows):
    row = {}
    for col in columns:
        row[col] = table[col][i]
    rows.append(row)
print(rows[2])
print(rows == records)
```

🔍 برای هر سطر `i` ، از هر ستون عضو `i` ام برداشته می‌شود؛ همان ایده‌ی **ترانهاده** در جلسه‌ی ۸. `rows[2]` رکورد رضاست و چون محتوا دقیقاً با `records` یکی است، مقایسه‌ی `==` درست است (دو دیکشنری وقتی برابرند که همان کلیدها با همان مقدارها را داشته باشند).

📌 آنچه باید بلد باشید

- دیکشنری جفت‌های `key: value` است؛ کلیدها یکتا و hashableاند؛ مقدارها هر چیزی.
- تفاوت `d[key]` (خطای `KeyError`) و `d.get(key, default)` .
- انتساب به کلید: افزودن یا تغییر؛ `del` ، `pop` و `update` .
- `in` فقط کلیدها را بررسی می‌کند؛ `max` ، `sorted` و حلقه‌ی `for` هم روی کلیدها کار می‌کنند.
- `keys()` ، `values()` و `items()` ؛ پیمایش با `for k, v in d.items()` .
- خطای تغییر اندازه‌ی دیکشنری حین پیمایش.
- الگوهای شمارش (`get(k, 0) + 1`)، گروه‌بندی (دیکشنری از لیست‌ها)، نگاشت/کدگذاری و وارونه‌سازی.
- جدول داده به دو شکل سطری و ستونی و تبدیل آن‌ها به یکدیگر.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی

با فرض `d = {"a": 1, "b": 2}` ، حاصل هر خط را بنویسید (هر خط را مستقل، روی همین `d` اولیه، در نظر بگیرید). اگر خطا می‌دهد، نوع خطا را بنویسید.

عبارت	حاصل؟
`d["c"]`	
`d.get("c", 0)`	
`"1" in d`	
`1 in d.values()`	
`list(d.items())[1]`	
`{1: "x", 1.0: "y", True: "z"}`	
`dict(zip("abc", range(3)))`	
`len({**d, "a": 10, "c": 3})`	

راهنمایی برای خط آخر: `**d` داخل `{ }` اعضای `d` را در دیکشنری جدید پخش می‌کند.

▼ پاسخ

عبارت	حاصل
`d["c"]`	`KeyError`
`d.get("c", 0)`	`0`
`"1" in d`	`False` (کلید رشته‌ی `"1"` نیست)
`1 in d.values()`	`True`
`list(d.items())[1]`	`('b', 2)`
`{1: "x", 1.0: "y", True: "z"}`	`{1: 'z'}` : سه کلید از نظر پایتون برابرند (جلسه‌ی ۹)؛ کلید اول نگه داشته می‌شود و مقدار آخر
`dict(zip("abc", range(3)))`	`{'a': 0, 'b': 1, 'c': 2}`
`len({**d, "a": 10, "c": 3})`	`3` (کلید `"a"` بازنویسی می‌شود، `"c"` اضافه می‌شود)

سؤال ۲ — فراوانی کلمات و واژه‌نامه

متن زیر را در نظر بگیرید:

```
text = "the cat sat on the mat the cat ran"
```

الف) با الگوی شمارش، دیکشنری `counts` را بسازید و آن را بنویسید. **فراوانی نسبی** هر کلمه (تعداد تقسیم بر کل کلمات) را تا سه رقم حساب کنید.

ب) با ساختن یک لیست از تاپل‌های `(count, word)` و مرتب‌سازی نزولی، سه کلمه‌ی پرتکرار را پیدا کنید. با سه کلمه‌ی اولی که پیدا کردید، چه مشکلی پیش آمده است؟

ج) دیکشنری `word2idx` را طوری بسازید که به هر کلمه‌ی **جدید** به ترتیب دیدن، یک شماره‌ی صحیح بدهد (از ۰). جمله‌ی `"the cat ran"` به چه دنباله‌ی اعدادی تبدیل می‌شود؟ (این دقیقاً اولین قدم ورود متن به یک شبکه‌ی عصبی است.) برای جمله‌ی `"the dog ran"` چه اتفاقی می‌افتد و چه راه‌حلی پیشنهاد می‌کنید؟

▼ پاسخ

الف) `{'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}` ؛ کل ۹ کلمه؛ فراوانی نسبی: `the` : `0.333` ، `cat` : `0.222` ، بقیه `0.111` .

ب)

```
pairs = []
for w, n in counts.items():
    pairs.append((n, w))
top3 = sorted(pairs, reverse=True)[:3]
```

نتیجه `[(3, 'the'), (2, 'cat'), (1, 'sat')]` . مشکل: چهار کلمه با فراوانی `1` هم‌رتبه‌اند و انتخاب `sat` فقط به ترتیب الفبایی **برعکس** بستگی دارد (همان مشکل تساوی سؤال kNN در جلسه‌ی ۹). ضمناً پرتکرارترین کلمه، `the` ، اطلاعات چندانی ندارد؛ در پردازش متن معمولاً چنین کلمه‌هایی (stop words) حذف می‌شوند.

ج)

```
word2idx = {}
for w in text.split():
    if w not in word2idx:
        word2idx[w] = len(word2idx)
```

`{'the': 0, 'cat': 1, 'sat': 2, 'on': 3, 'mat': 4, 'ran': 5}` و `"the cat ran"` به `[0, 1, 5]` تبدیل می‌شود. برای `"dog"` کلیدی وجود ندارد و `word2idx["dog"]` خطای `KeyError` می‌دهد. راه‌حل استاندارد: یک کلمه‌ی ویژه مثل `"<unk>"` (کلمه‌ی ناشناخته) با یک شماره‌ی رزرو تعریف کنید و از `word2idx.get(w, unk_id)` استفاده کنید.

سؤال ۳ — ماتریس درهم‌ریختگی چندکلاسه

برای یک دسته‌بند سه‌کلاسه:

```
y_true = ["cat", "dog", "cat", "bird", "dog", "cat", "bird", "dog"]
y_pred = ["cat", "cat", "cat", "bird", "dog", "dog", "dog", "dog"]

cm = {}
for t, p in zip(y_true, y_pred):
    cm[(t, p)] = cm.get((t, p), 0) + 1
```

الف) چرا کلید `cm` یک **تاپل** است و نمی‌توانست لیست `[t, p]` باشد؟ محتوای `cm` را بنویسید.

ب) برای هر کلاس `k` ، **recall** (تعداد `(k, k)` تقسیم بر تعداد نمونه‌هایی که واقعاً `k` هستند) و **precision** (تعداد `(k, k)` تقسیم بر تعداد پیش‌بینی‌های `k`) را حساب کنید. با دیکشنری بنویسید چطور این‌ها را حساب می‌کنید.

ج) چرا `cm[("cat", "bird")]` خطا می‌دهد؟ چطور بدون تغییر دیکشنری، تعداد آن را درست (یعنی `0`) بخوانید؟

▼ پاسخ

الف) کلید باید hashable باشد؛ تاپل هست، لیست نیست.

```
 {('cat', 'cat'): 2, ('dog', 'cat'): 1, ('bird', 'bird'): 1, ('dog', 'dog'): 2, ('cat', 'dog'): 1, ('bird', 'dog'): 1}
```

ب)

```
for k in sorted(set(y_true)):
    tp = cm.get((k, k), 0)
    actual = 0
    predicted = 0
    for (t, p), n in cm.items():
        if t == k:
            actual += n
        if p == k:
            predicted += n
    print(k, tp / actual, tp / predicted)
```

کلاس	recall	precision
`bird`	`1/2 = 0.5`	`1/1 = 1.0`
`cat`	`2/3 ≈ 0.667`	`2/3 ≈ 0.667`
`dog`	`2/3 ≈ 0.667`	`2/4 = 0.5`

ج) چون این ترکیب هیچ‌وقت رخ نداده و کلیدش ساخته نشده است؛ `cm.get(("cat", "bird"), 0)` مقدار `0` را برمی‌گرداند.

سؤال ۴ — باگ گروه‌بندی

دانشجویی برای کوتاه کردن الگوی گروه‌بندی از `dict.fromkeys(keys, value)` استفاده کرده است که یک دیکشنری با کلیدهای داده‌شده می‌سازد و **همه را** به `value` وصل می‌کند:

```
labels = ["a", "b", "a"]
values = [1, 2, 3]

groups = dict.fromkeys(labels, [])
for lbl, v in zip(labels, values):
    groups[lbl].append(v)
print(groups)
```

الف) خروجی را پیش‌بینی کنید. انتظار برنامه‌نویس `{'a': [1, 3], 'b': [2]}` بود. چه اتفاقی افتاده است؟ (راهنمایی: شکل «نام‌ها و مقدارها» در جلسه‌های ۲ و ۸.)

ب) دو راه درست برای این کد بنویسید.

▼ **پاسخ**

الف) `{'a': [1, 2, 3], 'b': [1, 2, 3]}` . `fromkeys` **یک** لیست خالی می‌سازد و هر دو کلید را به **همان** لیست وصل می‌کند؛ پس هر `append` در «هر دو گروه» دیده می‌شود. (همان باگ `[[0] * 3] * 3` در جلسه‌ی ۸.)

ب) الگوی بخش ۴ (ساختن لیست جدید برای هر کلید جدید):

```
groups = {}
for lbl, v in zip(labels, values):
    if lbl not in groups:
        groups[lbl] = []
    groups[lbl].append(v)
```

یا ساختن لیست جدید برای هر کلید از قبل:

```
groups = {}
for lbl in set(labels):
    groups[lbl] = []
for lbl, v in zip(labels, values):
    groups[lbl].append(v)
```

سؤال ۵ — حافظه‌ی نهان (memoization)

تابع بازگشتی زیر جمله‌ی `n` ام دنباله‌ی فیبوناچی را حساب می‌کند؛ متغیر `calls` تعداد فراخوانی‌ها را می‌شمارد:

```
calls = 0

def fib(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(20), calls)
```

الف) خروجی `fib(20)` برابر `6765` و تعداد فراخوانی‌ها `21891` است. چرا این‌قدر زیاد؟ (درخت فراخوانی `fib(5)` را بکشید و بشمارید `fib(2)` چند بار حساب می‌شود.)

ب) با یک دیکشنری `memo` که نتیجه‌ی هر `n` حساب‌شده را نگه می‌دارد، تابع `fib_memo` را بنویسید. حالا `fib_memo(20)` چند بار فراخوانی می‌شود؟

ج) این ایده (ذخیره‌ی نتیجه‌ی محاسبه‌های تکراری) در یادگیری عمیق کجا کاربرد دارد؟ (راهنمایی: محاسبه‌ی گرادیان در یک شبکه‌ی چندلایه به مشتق‌های لایه‌های بعدی نیاز دارد.)

▼ پاسخ

الف) هر فراخوانی دو فراخوانی دیگر می‌سازد و زیرمسئله‌های یکسان بارها از نو حساب می‌شوند. در درخت `fib(5)`، `fib(3)` دو بار، `fib(2)` سه بار و `fib(1)` پنج بار حساب می‌شود؛ تعداد کل فراخوانی‌ها تقریباً نمایی رشد می‌کند.

ب)

```
memo = {}

def fib_memo(n):
    global calls
    calls += 1
    if n in memo:
        return memo[n]
    if n < 2:
        result = n
    else:
        result = fib_memo(n - 1) + fib_memo(n - 2)
    memo[n] = result
    return result
```

با `calls = 0` و `memo` خالی، `fib_memo(20)` فقط `39` فراخوانی دارد (هر `n` یک بار محاسبه می‌شود و بقیه از `memo` خوانده می‌شود).

ج) **پس‌انتشار (backpropagation)** دقیقاً همین ایده است: مشتق هر لایه یک بار حساب و ذخیره می‌شود و برای لایه‌های قبلی دوباره استفاده می‌شود. بدون آن، محاسبه‌ی گرادیان در شبکه‌های عمیق عملاً ناممکن بود. (در جلسه‌ی ۱۴ همین حافظه‌ی نهان را با یک decorator به هر تابعی اضافه می‌کنیم.)

سؤال ۶ — جست‌وجوی ابرپارامترها

برای انتخاب ابرپارامترهای یک مدل، **همه‌ی ترکیب‌های** مقدارهای پیشنهادی را امتحان می‌کنیم (grid search):

```
grid = {"lr": [0.1, 0.01], "batch": [16, 32, 64]}

configs = []
for lr in grid["lr"]:
    for batch in grid["batch"]:
        configs.append({"lr": lr, "batch": batch})
```

الف) `configs` چند عضو دارد؟ دو عضو اول آن را بنویسید. اگر `grid` سه کلید با ۳، ۴ و ۵ مقدار داشت، چند ترکیب وجود داشت؟

ب) بعد از آموزش، دقت هر ترکیب در یک دیکشنری با کلید **تاپل** `(lr, batch)` ذخیره شده است:

```
results = {(0.1, 16): 0.81, (0.1, 32): 0.84, (0.1, 64): 0.79,
           (0.01, 16): 0.86, (0.01, 32): 0.85, (0.01, 64): 0.83}
```

با یک حلقه روی `results.items()`، بهترین ترکیب و دقت آن را پیدا کنید. چرا اینجا از کلید تاپل استفاده شده و نه از خود دیکشنری‌های `configs`؟

ج) دقت میانگین برای هر مقدار `lr` (روی همه‌ی اندازه‌های batch) چقدر است؟ برای این کار از الگوی گروه‌بندی استفاده کنید.

▼ پاسخ

الف) `2 × 3 = 6` عضو؛ دو عضو اول `{'lr': 0.1, 'batch': 16}` و `{'lr': 0.1, 'batch': 32}`. با ۳، ۴ و ۵ مقدار: `3 × 4 × 5 = 60` ترکیب (تعداد ترکیب‌ها با تعداد ابرپارامترها نمایی رشد می‌کند).

ب)

```
best_config = None
best_acc = -1
for config, acc in results.items():
    if acc > best_acc:
        best_config, best_acc = config, acc
print(best_config, best_acc)
```

نتیجه: `(0.01, 16) 0.86`. دیکشنری‌ها تغییرپذیرند و نمی‌توانند **کلید** باشند؛ تاپل می‌تواند.

ج)

```
by_lr = {}
for (lr, batch), acc in results.items():
    if lr not in by_lr:
        by_lr[lr] = []
    by_lr[lr].append(acc)
for lr, accs in by_lr.items():
    print(lr, sum(accs) / len(accs))
```

برای `0.1` : `(0.81 + 0.84 + 0.79) / 3 = 0.813` ؛ برای `0.01` : `(0.86 + 0.85 + 0.83) / 3` .`= 0.847`

سؤال ۷ — آمار روی داده‌ی ستونی

داده‌ی دو متغیر به شکل ستونی:

```
data = {"x": [2, 4, 6, 8, 10],
        "y": [1, 3, 7, 9, 15]}

def mean(v):
    return sum(v) / len(v)

def cov(a, b):
    ma, mb = mean(a), mean(b)
    total = 0
    for ai, bi in zip(a, b):
        total += ______
    return total / (len(a) - 1)

means = {}
for col in data:
    means[col] = ______

r = cov(data["x"], data["y"]) / (cov(data["x"], data["x"]) * cov(data["y"],
data["y"])) ** 0.5
```

الف) جاهای خالی را کامل کنید.

ب) مقدار `means` ، **کوواریانس نمونه‌ای** `cov(x, y)` ، واریانس‌های `x` و `y` و **ضریب همبستگی** `r` را حساب کنید (`r` را تا سه رقم).

ج) چرا تابع `cov(a, a)` همان **واریانس** است؟ چرا مخرج `n - 1` است و نه `n` ؟

▼ پاسخ

الف)

```
total += (ai - ma) * (bi - mb)
means[col] = mean(data[col])
```

ب) `means = {'x': 6.0, 'y': 7.0}`. انحراف‌ها از میانگین: `x`: `-4, -2, 0, 2, 4` و `y`: `-6, -1, 2, 4, 17`. واریانس‌ها: `var(x) = 40 / 4 = 10` و `var(y) = 120 / 4 = 30`. پس `r = 17 / √300 ≈ 0.981`: همبستگی مثبت بسیار قوی.

ج) با `b = a`، هر جمله `(ai - ma) ** 2` می‌شود که همان تعریف واریانس است. مخرج `n - 1` (اصلاح بسل) برآورد **نااریب** واریانس جامعه از روی نمونه را می‌دهد، چون میانگین هم از همان نمونه برآورد شده و یک درجه‌ی آزادی را مصرف کرده است. (Pandas هم به‌طور پیش‌فرض از `n - 1` استفاده می‌کند.)

خلاصه‌ی جلسه

- دیکشنری جفت‌های `key: value` با کلیدهای یکتا و hashable است و جست‌وجو با کلید را فوری انجام می‌دهد.
- `d[key]` برای کلید ناموجود خطا می‌دهد؛ `d.get(key, default)` نه. `in`، `for` و `max` روی کلیدها کار می‌کنند.
- `items()` پرکاربردترین راه پیمایش است؛ اندازه‌ی دیکشنری را حین پیمایش تغییر ندهید.
- الگوهای اصلی: شمارش، گروه‌بندی، نگاشت/کدگذاری و وارونه‌سازی.
- جدول داده را می‌توان سطری (لیستی از دیکشنری‌ها) یا ستونی (دیکشنری از لیست‌ها) نگه داشت؛ پایه‌ی DataFrame در Pandas.

جلسه‌ی بعد: رشته‌ها.

جلسه ۱۱: رشته‌ها

Strings

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

🎯 اهداف جلسه

در پایان این جلسه باید بتوانید:

- رشته را به‌عنوان یک **دنباله‌ی تغییرناپذیر** از کاراکترها بشناسید و اندیس و برش آن را پیش‌بینی کنید،
- با متدهای اصلی رشته کار کنید و بدانید همه‌ی آن‌ها **رشته‌ی جدید** برمی‌گردانند،
- در رشته جست‌وجو و جایگزینی کنید و تفاوت `find` و `index` را بدانید،
- با `split` و `join` متن را به لیست و برعکس تبدیل کنید و یک خط داده‌ی متنی را تجزیه کنید،
- یک **خط لوله‌ی پیش‌پردازش متن** (پاک‌سازی، توکن‌سازی) بنویسید؛ اولین قدم هر کار پردازش زبان طبیعی.

> 📖 روش خواندن این جزوه و معنی علامت‌ها (💡 🔍 ❓ ⚠️ 📌 📝) در ابتدای جلسه‌ی ۱ توضیح داده شده است.

فهرست مطالب

1. رشته به‌عنوان دنباله
2. متدهای رشته
3. جست‌وجو و جایگزینی
4. `split` و `join`
5. پیش‌پردازش متن و نکته‌های قالب‌بندی

۱. رشته به‌عنوان دنباله

💡 مفهوم

در جلسه‌ی ۲ دیدید که رشته (`str`) برای نگه‌داری متن است. حالا که لیست و تاپل را می‌شناسید، می‌توانیم دقیق‌تر بگوییم: رشته یک **دنباله‌ی مرتب و تغییرناپذیر** از کاراکترهاست؛ پس اندیس، برش، `len`، `in`، `+`، `*` و حلقه‌ی `for` دقیقاً مثل تاپل کار می‌کنند.

```
text = "Python"
print(text[0], text[-1])
print(text[0:4], text[2:], text[:3])
print(text[::-1])
print(len(text), "th" in text)

s1 = "Hello"
s2 = "World"
print(s1 + " " + s2, s1 * 3)

for ch in "abc":
    print(ch, end="-")
```

🔍 کد را بخوان

- `text[0:4]` چهار کاراکتر اول را برمی‌دارد (`stop` شامل نیست؛ مثل `range` و برش لیست).
- `text[::-1]` رشته را برعکس می‌کند.
- `"th" in text` بررسی می‌کند آیا `"th"` **زیررشته**ی `text` است (جلسه‌ی ۳).

⚠️ رشته تغییرناپذیر است

```
word = "Jython"
word[0] = "P"
```

🔍 مثل تاپل، نمی‌توان یک کاراکتر را عوض کرد. باید یک رشته‌ی **جدید** ساخت: `word = "P" + word[1:]` .

💡 کاراکترها در واقع عدد هستند

هر کاراکتر یک کد عددی در استاندارد **یونیکد (Unicode)** دارد. `ord` کد یک کاراکتر را می‌دهد و `chr` برعکس عمل می‌کند. همین کدها هستند که ترتیب مقایسه‌ی رشته‌ها را تعیین می‌کنند (جلسه‌ی ۳):

```
print(ord("A"), ord("a"), ord("0"))
print(chr(98))
print("Z" < "a")
```

🔍 کد `"Z"` (۹۰) از کد `"a"` (۹۷) کوچک‌تر است؛ برای همین همه‌ی حروف بزرگ انگلیسی قبل از حروف کوچک مرتب می‌شوند. چون رشته‌های پایتون یونیکد هستند، متن فارسی هم در داده‌ها بدون مشکل نگه‌داری و پردازش می‌شود.

۲. متدهای رشته

💡 مفهوم

رشته ده‌ها متد دارد. نکته‌ی کلیدی: چون رشته تغییرناپذیر است، **هیچ متدی خود رشته را عوض نمی‌کند**؛ همه یک رشته (یا مقدار) **جدید** برمی‌گردانند. (برخلاف `list.sort` یا `list.append` در جلسه‌ی ۸.)

متد	کار	مثال	نتیجه
`lower()` / `upper()`	حروف کوچک / بزرگ	`"Data".lower()`	`'data'`
`title()`	حرف اول هر کلمه بزرگ	`"data science".title()`	`'Data Science'`
`strip()`	حذف فاصله (و `\n`) از دو طرف	`"  hi \n".strip()`	`'hi'`
`lstrip()` / `rstrip()`	فقط از چپ / راست	`"xxhi".lstrip("x")`	`'hi'`
`startswith(p)` / `endswith(p)`	آیا با `p` شروع / تمام می‌شود؟	`"data.csv".endswith(".csv")`	`True`
`isdigit()`	فقط رقم؟	`"2026".isdigit()`	`True`
`isalpha()`	فقط حرف؟	`"abc1".isalpha()`	`False`
`isalnum()`	فقط حرف و رقم؟	`"abc1".isalnum()`	`True`
`isspace()`	فقط فاصله؟	`"  ".isspace()`	`True`

```python
s = "  Python Programming  "
print(s.lower())
print(s.upper())
print(s.strip())
print(s.strip().lower().replace(" ", "_"))     # methods can be chained
print("[" + s + "]")                           # s itself is unchanged
```

🔍 کد را بخوان

- در خط چهارم چند متد **زنجیر** شده‌اند و از چپ به راست اجرا می‌شوند: اول فاصله‌های دو طرف حذف، بعد حروف کوچک، بعد فاصله‌ی وسط با `_` جایگزین می‌شود. این کار برای ساختن **نام ستون‌های تمیز** در داده بسیار رایج است.
- خط آخر نشان می‌دهد `s` بعد از همه‌ی این فراخوانی‌ها دست‌نخورده باقی مانده است.

⚠️ اشتباه رایج: فراموش کردن انتساب

`name.strip()` به‌تنهایی هیچ اثری ندارد؛ نتیجه‌اش دور ریخته می‌شود. باید بنویسید `name = name.strip()`.

❓ پیش‌بینی کن

متدهای `is...` برای **اعتبارسنجی** داده‌ی ورودی پرکاربردند. کدام خط‌ها `True` چاپ می‌کنند؟ (به عدد منفی و عدد اعشاری دقت کنید.)

```
print("2026".isdigit())
print("-12".isdigit())
print("3.5".isdigit())
print("".isdigit())
print(" 42 ".strip().isdigit())
```

🔍 `isdigit` فقط وقتی `True` است که رشته **غیرخالی** باشد و **همه‌ی** کاراکترها رقم باشند؛ پس `"-12"` (علامت منفی) و `"3.5"` (نقطه) نادرست‌اند. بنابراین `isdigit` برای تشخیص «آیا این رشته یک عدد است؟» کافی نیست؛ در جلسه‌ی ۱۵ راه درست را (تلاش برای تبدیل با `float` و مدیریت خطا) می‌بینیم.

۳. جست‌وجو و جایگزینی

💡 مفهوم

متد	کار	اگر پیدا نشود
`s.find(sub)`	اندیس **اولین** وقوع `sub`	`-1`
`s.index(sub)`	مثل `find`	خطای `ValueError`
`s.count(sub)`	تعداد وقوع‌ها (بدون هم‌پوشانی)	`0`
`s.replace(old, new)`	جایگزینی **همه‌ی** وقوع‌ها	همان رشته
`s.replace(old, new, k)`	جایگزینی `k` وقوع اول	همان رشته

```
email = "student.ali@pgu.ac.ir"
at = email.find("@")
print(at)
print("user:", email[:at])
print("domain:", email[at + 1:])
print(email.count("."), email.find("#"))
print(email.replace(".", "[dot]"))
```

🔍 کد را بخوان

الگوی «پیدا کن و برش بزن» بسیار رایج است: `find` موقعیت `@` را می‌دهد و با برش، بخش قبل و بعد از آن جدا می‌شود. چون `@` خودش جزو هیچ‌کدام نیست، بخش دوم از `at + 1` شروع می‌شود.

❓ پیش‌بینی کن

```
s = "banana"
print(s.find("an"), s.find("an", 2))
print(s.count("ana"))
print(s.replace("a", "A", 2))
print(s.index("x"))
```

🔍 آرگومان دوم `find` محل **شروع** جست‌وجو است؛ پس `s.find("an", 2)` وقوع دوم (`3`) را پیدا می‌کند. `count("ana")` برابر `1` است، نه `2` : وقوع‌های **هم‌پوشان** (`ban[ana]` و `b[ana]na`) فقط یک بار شمرده می‌شوند. `replace` با آرگومان سوم فقط دو `a` اول را عوض می‌کند. خط آخر `ValueError` می‌دهد.

⚠️ اشتباه رایج: `if s.find(x):`

`find` در صورت پیدا نکردن `-1` برمی‌گرداند که در شرط **درست** حساب می‌شود، و اگر زیررشته در **اول** رشته باشد، `0` برمی‌گرداند که **نادرست** حساب می‌شود؛ یعنی دقیقاً برعکس منظور شما! برای بررسی وجود، از `in` استفاده کنید.

۴. `split` و `join`

💡 مفهوم

در جلسه‌ی ۸ با این دو آشنا شدید. حالا جزئیات مهم‌تر:

- `s.split()` (بدون آرگومان): روی **هر تعداد فاصله، tab یا خط جدید** می‌شکند و رشته‌های خالی را دور می‌اندازد.
- `s.split(sep)` : دقیقاً روی `sep` می‌شکند؛ جداکننده‌های پشت سر هم، رشته‌ی **خالی** تولید می‌کنند.
- `s.split(sep, maxsplit)` : حداکثر `maxsplit` بار می‌شکند.
- `sep.join(list_of_strings)` : اعضای لیست را با `sep` به هم می‌چسباند.

```
messy = "  the   cat\tsat \n on  the mat "
print(messy.split())
print("a,,b".split(","))
print("key=value=with=equals".split("=", 1))

words = ["2026", "10", "30"]
print("-".join(words))
print("".join(["P", "y", "t", "h", "o", "n"]))
```

🔍 کد را بخوان

- `messy.split()` فاصله‌های اضافه، tab و خط جدید را همه با هم نادیده می‌گیرد؛ معمولاً همین را برای جدا کردن کلمات می‌خواهید.
- `"a,,b".split(",")` بین دو کامای پشت سر هم یک رشته‌ی خالی تولید می‌کند: `['a', '', 'b']`. در فایل‌های CSV یعنی **مقدار گمشده**.
- با `maxsplit=1` ، فقط اولین `=` جداکننده است؛ بقیه جزو مقدار می‌مانند.

💡 تجزیه‌ی یک خط داده

ترکیب `split` ، `strip` و تبدیل نوع، اساس خواندن داده‌ی متنی است:

```
line = "Ali, 17.5 ,Bushehr\n"
fields = line.split(",")
print(fields)

name = fields[0].strip()
score = float(fields[1])
city = fields[2].strip()
print(name, score, city)
```

🔍 بعد از `split` ، فیلدها فاصله‌های اضافه و `\n` انتهای خط را دارند. `float` خودش فاصله‌های دو طرف را تحمل می‌کند، اما برای رشته‌ها باید صریحاً `strip` کنید؛ وگرنه `"Bushehr\n"` با `"Bushehr"` برابر نیست و مثلاً در شمارش (جلسه‌ی ۱۰) دو شهر جدا حساب می‌شوند.

⚠️ اشتباه رایج: ساختن رشته با `+=` در حلقه

چون رشته تغییرناپذیر است، هر `result += piece` یک رشته‌ی **کاملاً جدید** می‌سازد و همه‌ی محتوای قبلی را کپی می‌کند. برای چند ده قطعه مهم نیست، اما برای هزاران قطعه بسیار کند است. الگوی درست: قطعه‌ها را در یک **لیست** جمع کنید و در آخر یک بار `join` کنید.

```
parts = []
for i in range(5):
    parts.append(str(i ** 2))
result = ", ".join(parts)
print(result)
```

۵. پیش‌پردازش متن و نکته‌های قالب‌بندی

💡 خط لوله‌ی پیش‌پردازش

قبل از اینکه متن وارد هر مدل یادگیری ماشین شود، باید **یکنواخت** شود: «data»، «Data» و «data,» باید یک کلمه حساب شوند. یک خط لوله‌ی ساده:

1. همه‌ی حروف کوچک شوند،
2. علامت‌های نگارشی با فاصله جایگزین شوند،
3. متن به کلمه‌ها (**توکن**ها) شکسته شود،
4. کلمه‌های بسیار رایج و کم‌اطلاعات (**stop words**) حذف شوند.

ماژول `string` (از کتابخانه‌ی استاندارد) فهرست آماده‌ای از علامت‌های نگارشی انگلیسی دارد:

```
import string
print(string.punctuation)

def tokenize(text):
    text = text.lower()
    clean = []
    for ch in text:
        if ch in string.punctuation:
            clean.append(" ")
        else:
            clean.append(ch)
    return "".join(clean).split()

stop_words = {"the", "a", "an", "on", "is", "and"}
review = "The movie was GREAT, and the music... the music is AMAZING!"

tokens = tokenize(review)
print(tokens)

content = []
for t in tokens:
    if t not in stop_words:
        content.append(t)
print(content)
```

🔍 کد را بخوان

- `tokenize` کاراکترها را یکی‌یکی بررسی می‌کند و علامت‌های نگارشی را با فاصله عوض می‌کند. قطعه‌ها در یک لیست جمع و در آخر `join` می‌شوند (الگوی بخش ۴).
- `split()` بدون آرگومان، فاصله‌های اضافه‌ای را که از حذف علامت‌ها به وجود آمده نادیده می‌گیرد.
- `stop_words` یک **مجموعه** است؛ بررسی `t not in stop_words` فوری است (جلسه‌ی ۹).

نتیجه، فهرستی از کلمه‌های پرمعناست که حالا می‌توان آن را شمرد (جلسه‌ی ۱۰)، به عدد تبدیل کرد (`word2idx`) و به یک مدل داد.

💡 یادآوری قالب‌بندی و دو نکته‌ی تازه

قالب‌بندی با f-string را در جلسه‌ی ۴ کامل دیدید (`:.2f` ، `:,` ، `:>8` و ...). دو نکته‌ی تکمیلی:

رشته‌ی خام (raw string): در مسیرهای ویندوز، `\n` و `\t` معنی خط جدید و tab دارند و مسیر را خراب می‌کنند. با گذاشتن `r` قبل از نقل‌قول، علامت `\` معمولی در نظر گرفته می‌شود:

```
print("C:\new\table.csv")
print(r"C:\new\table.csv")
```

رشته‌ی چندخطی با سه نقل‌قول، شکسته شدن خط‌ها را حفظ می‌کند؛ برای متن‌های طولانی و docstringها (جلسه‌ی ۷) استفاده می‌شود.

📌 آنچه باید بلد باشید

- رشته دنباله‌ای تغییرناپذیر است؛ اندیس، برش، `in` ، `+` ، `*` و حلقه مثل تاپل؛ انتساب به `s[i]` خطاست.
- `ord` و `chr` و ترتیب یونیکد.
- همه‌ی متدهای رشته مقدار **جدید** برمی‌گردانند؛ نتیجه را انتساب کنید؛ زنجیر کردن متدها.
- رفتار `isdigit` ، `isalpha` و `isalnum` (از جمله برای `"-12"` ، `"3.5"` و `""`).
- `find` (و `-1`)، `index` (و `ValueError`)، `count` (بدون هم‌پوشانی)، `replace` (همه یا `k` تا).
- تفاوت `split()` و `split(sep)` ، رشته‌های خالی، `maxsplit` ؛ `join` روی لیستی از رشته‌ها.
- تجزیه‌ی یک خط داده با `split` ، `strip` و تبدیل نوع.
- چرا `+=` در حلقه کند است و الگوی لیست و `join` .
- یک تابع پیش‌پردازش متن (حروف کوچک، حذف علامت‌ها، توکن‌سازی، حذف stop words)؛ رشته‌ی خام.

📝 سؤالات مرور / نمونه سؤال امتحان

> اول خودتان جواب دهید (روی کاغذ، بدون اجرای کد)، بعد پاسخ را باز کنید و در آخر کد را اجرا کنید.

سؤال ۱ — پیش‌بینی

با فرض `s = "Data Science"`، حاصل هر عبارت را بنویسید.

عبارت	حاصل؟
`s[5:]`	
`s[-7:-3]`	
`s[::-1]`	
`s.find("a"), s.find("z")`	
`s.count("a")`	
`s.replace("a", "4", 1)`	
`s.split("a")`	
`s.lower().index("s")`	
`s.strip("De")`	

▼ پاسخ

عبارت	حاصل
`s[5:]`	`'Science'`
`s[-7:-3]`	`'Scie'`
`s[::-1]`	`'ecneicS ataD'`
`s.find("a"), s.find("z")`	`(1, -1)`
`s.count("a")`	`2` (فقط حروف کوچک؛ `"Data"` دو `a` دارد)
`s.replace("a", "4", 1)`	`'D4ta Science'`
`s.split("a")`	`['D', 't', ' Science']`
`s.lower().index("s")`	`5`

عبارت	حاصل
`s.strip("De")`	`'ata Scienc'` : `strip` با آرگومان، **هر کاراکتری** از مجموعه‌ی `{'D', 'e'}` را از دو سر حذف می‌کند (نه رشته‌ی `"De"` را)

سؤال ۲ – توکن‌ساز

تابع `tokenize` بخش ۵ را در نظر بگیرید.

الف) خروجی آن را برای این سه ورودی بنویسید:

```
tokenize("Hello, World! Hello... AI.")
tokenize("Pi is 3.14 in the U.S.")
tokenize("I don't know")
```

ب) برای ورودی‌های دوم و سوم، کدام توکن‌ها «غلط» به نظر می‌رسند و چرا؟ هر کدام در یک مدل متنی چه مشکلی ایجاد می‌کند؟

ج) تابع را طوری تغییر دهید که نقطه‌ای که **بین دو رقم** قرار دارد حذف نشود (تا `3.14` یک توکن بماند). راهنمایی: به جای `for ch in text` روی اندیس‌ها حلقه بزنید تا به کاراکتر قبلی و بعدی دسترسی داشته باشید.

▼ پاسخ

الف)

```
['hello', 'world', 'hello', 'ai']
['pi', 'is', '3', '14', 'in', 'the', 'u', 's']
['i', 'don', 't', 'know']
```

ب) `3.14` به دو عدد بی‌ربط `3` و `14` شکسته شده، `U.S.` به دو حرف بی‌معنی، و `don't` به `don` و `t`. مدل این‌ها را کلمه‌های جدا می‌بیند: معنی از دست می‌رود و واژگان با توکن‌های بی‌معنی شلوغ می‌شود. (توکن‌سازهای واقعی قاعده‌های خاص برای اعداد، اختصارها و ضمیرهای متصل دارند.)

ج)

```python
def tokenize(text):
    text = text.lower()
    clean = []
    for i in range(len(text)):
        ch = text[i]
        keep_dot = (ch == "." and 0 < i < len(text) - 1
                    and text[i - 1].isdigit() and text[i + 1].isdigit())
        if ch in string.punctuation and not keep_dot:
            clean.append(" ")
        else:
            clean.append(ch)
    return "".join(clean).split()
```

حالا `tokenize("Pi is 3.14")` برابر `['pi', 'is', '3.14']` است. شرط `0 < i < len(text)` جلوی `IndexError` را در اول و آخر متن می‌گیرد. `- 1`

سؤال ۳ — n-gram

بسیاری از مدل‌های زبانی به جای تک‌کلمه‌ها، دنباله‌های `n` تایی از توکن‌ها (**n-gram**) را می‌شمارند:

```
def ngrams(tokens, n):
    result = []
    for i in range(len(tokens) - n + 1):
        result.append(tuple(tokens[i:i + n]))
    return result
```

الف) خروجی `ngrams(["i", "love", "data", "science"], 2)` و `ngrams(["a", "b"], 5)` چیست؟ چرا حلقه تا `len(tokens) - n + 1` است؟ اگر به جای آن `len(tokens)` بود، چه می‌شد؟

ب) `ngrams(list("data"), 3)` چه چیزی برمی‌گرداند؟ (به این‌ها **character n-gram** می‌گویند.)

ج) چرا n-gramها به شکل **تاپل** ذخیره شده‌اند و نه لیست؟ (راهنمایی: قدم بعدی، شمردن آن‌ها با یک دیکشنری است.)

د) یک متن `N` توکنی چند `n`-gram دارد؟ اگر واژگان `V` کلمه داشته باشد، حداکثر چند bigram **متفاوت** ممکن است وجود داشته باشد؟ این چه مشکلی برای مدل‌های مبتنی بر شمارش ایجاد می‌کند؟

▼ پاسخ

الف) `[('i', 'love'), ('love', 'data'), ('data', 'science')]` و `[]`. آخرین `n`-gram از اندیس `len - n` شروع می‌شود؛ پس `range` باید تا `len - n + 1` باشد (`stop` شامل نیست). با `len(tokens)`، برش‌های آخر کوتاه‌تر از `n` می‌شدند (مثلاً `('science',)`)؛ بدون هیچ خطایی، n-gramهای ناقص تولید می‌شد.

ب) `[('d', 'a', 't'), ('a', 't', 'a')]`.

ج) چون می‌خواهیم آن‌ها را **کلید دیکشنری** (برای شمارش) یا عضو مجموعه کنیم و فقط تاپل hashable است.

د) `N - n + 1` تا (اگر `N >= n`). bigramهای ممکن `V²` تاست؛ برای واژگان ۱۰٬۰۰۰ کلمه‌ای، ۱۰۰ میلیون. تقریباً همه‌ی آن‌ها هرگز در داده‌ی آموزش دیده نمی‌شوند (**پراکندگی داده**) و شمارششان صفر است؛ یکی از انگیزه‌های اصلی مدل‌های مبتنی بر شبکه‌ی عصبی و embedding.

سؤال ۴ — تجزیه‌ی CSV

الف) خروجی کد زیر را بنویسید. چرا تجزیه‌ی خط دوم اشتباه است؟

```
line1 = "Ali,17.5,Bushehr"
line2 = 'Sara,"Tehran, Iran",19'
```

```
print(line1.split(","))
print(line2.split(","), len(line2.split(",")))
```

ب) فایلی داریم که هر خط آن به شکل `name,score` است، اما بعضی خط‌ها مشکل دارند. تابعی بنویسید که یک لیست از خط‌ها بگیرد و یک دیکشنری `{name: score}` برگرداند، به‌طوری که: فاصله‌های اضافه حذف شوند، خط‌های خالی نادیده گرفته شوند، و خط‌هایی که دقیقاً دو فیلد ندارند یا نمره‌ی آن‌ها فقط از رقم و حداکثر یک نقطه تشکیل نشده، نادیده گرفته شوند. خروجی تابع برای ورودی زیر چیست؟

```
lines = ["Ali, 17.5", "", "Sara,19", "Reza,abc", "Mina,15,extra", "  Hadi ,
12 "]
```

▼ **پاسخ**

الف)

```
['Ali', '17.5', 'Bushehr']
['Sara', '"Tehran', ' Iran"', '19'] 4
```

کامای داخل نقل‌قول بخشی از **مقدار** است، اما `split` آن را جداکننده حساب می‌کند و رکورد سه‌فیلدی چهار تکه می‌شود. برای همین برای خواندن CSV از ماژول `csv` یا Pandas استفاده می‌کنیم (جلسه‌های ۱۶ و ۲۳).

ب)

```
def parse_scores(lines):
    result = {}
    for line in lines:
        line = line.strip()
        if line == "":
            continue
        fields = line.split(",")
        if len(fields) != 2:
            continue
        name = fields[0].strip()
        score_text = fields[1].strip()
        if score_text.count(".") > 1 or not score_text.replace(".", "",
1).isdigit():
            continue
        result[name] = float(score_text)
    return result
```

خروجی: `{'Ali': 17.5, 'Sara': 19.0, 'Hadi': 12.0}`. `Reza` نمره‌ی نامعتبر دارد و `Mina` سه فیلد. (در جلسه‌ی ۱۵ همین کار را با `try` / `except` ساده‌تر و دقیق‌تر انجام می‌دهیم.)

سؤال ۵ — کدگذاری در سطح کاراکتر

مدل‌های زبانی سطح کاراکتر، هر کاراکتر را به یک عدد تبدیل می‌کنند:

```
text = "banana"
chars = sorted(set(text))
```

```
char2idx = {}
for i, c in enumerate(chars):
    char2idx[c] = i

encoded = []
for c in text:
    encoded.append(char2idx[c])
```

الف) `chars`، `char2idx` و `encoded` را بنویسید.

ب) با ساختن دیکشنری وارونه (`idx2char`) و `join`، تابع `decode(encoded)` را بنویسید که رشته‌ی اصلی را بازسازی کند.

ج) **فاصله‌ی همینگ** دو رشته‌ی هم‌طول، تعداد موقعیت‌هایی است که کاراکترهایشان فرق دارد. تابع `hamming(a, b)` را با `zip` بنویسید و `hamming("karolin", "kathrin")` و `hamming("data", "date")` را حساب کنید. چرا این فاصله برای مقایسه‌ی `"data"` و `"ata"` مناسب نیست؟

▼ **پاسخ**

الف) `chars = ['a', 'b', 'n']`، `char2idx = {'a': 0, 'b': 1, 'n': 2}`، `encoded = [1, 0, 2, 0, 2, 0]`.

ب)

```
idx2char = {}
for c, i in char2idx.items():
    idx2char[i] = c

def decode(encoded):
    pieces = []
    for i in encoded:
        pieces.append(idx2char[i])
    return "".join(pieces)
```

ج)

```
def hamming(a, b):
    d = 0
    for x, y in zip(a, b):
        if x != y:
            d += 1
    return d
```

`"ata"` از حذف یک حرف `"data"` به دست می‌آید، اما مقایسه‌ی موقعیت به موقعیت همه‌ی کاراکترها را نامتناظر می‌کند (و `zip` بی‌صدا یکی را کوتاه می‌کند). برای این حالت از **فاصله‌ی ویرایش (Levenshtein)** استفاده می‌شود که درج و حذف را هم در نظر می‌گیرد.

`hamming("karolin", "kathrin")` و `hamming("data", "date")` به ترتیب `3` و `1`. فاصله‌ی همینگ فقط برای رشته‌های **هم‌طول** تعریف شده و فقط **جایگزینی** را می‌بیند.

سؤال ۶ — پالیندروم و ترتیب پردازش

تابع زیر بررسی می‌کند که یک متن (با نادیده گرفتن حروف بزرگ و کوچک و هر کاراکتر غیر از حرف و رقم) از دو طرف یکسان خوانده می‌شود یا نه:

```
def is_palindrome(s):
    clean = []
    for ch in s.lower():
        if ch.isalnum():
            clean.append(ch)
    c = "".join(clean)
    return c == c[::-1]
```

الف) خروجی برای `"A man, a plan, a canal: Panama"`، `"Data"` و `""` چیست؟

ب) دانشجویی ترتیب را عوض کرده و **اول** فیلتر `isalnum` و **بعد** `lower` را روی نتیجه اعمال کرده است. آیا نتیجه فرق می‌کند؟ اگر به جای `lower` از `strip` استفاده می‌کرد چطور؟ به طور کلی، چه وقت ترتیب مراحل یک خط لوله‌ی پیش‌پردازش مهم است؟

▼ پاسخ

الف) `True`، `False`، `True` (رشته‌ی خالی با برعکس خودش برابر است).

ب) برای `lower` و فیلتر `isalnum` ترتیب اهمیتی ندارد، چون کوچک کردن حروف، حرف یا رقم بودن یک کاراکتر را تغییر نمی‌دهد. `strip` هم بعد از فیلتر بی‌اثر است، چون فاصله‌ای باقی نمانده است. ترتیب وقتی مهم است که یک مرحله **ورودی** مرحله‌ی دیگر را تغییر دهد؛ مثلاً اگر علامت‌های نگارشی را **قبل از** `split` با فاصله عوض نکنید، `"data,"` و `"data"` دو توکن متفاوت می‌شوند، و اگر stop words را **قبل از** کوچک کردن حروف حذف کنید، `"The"` حذف نمی‌شود.

سؤال ۷ — تکمیل کد: شباهت دو جمله

با ترکیب `tokenize` (بخش ۵)، الگوی شمارش (جلسه‌ی ۱۰) و شباهت کسینوسی (جلسه‌ی ۸)، شباهت دو جمله را حساب می‌کنیم. جاهای خالی را کامل کنید.

```
import math

def bag_of_words(text):
    counts = {}
    for w in tokenize(text):
        counts[w] = ______
    return counts

def cosine(a, b):
    dot = 0
    for w in a:
```

```
        dot += a[w] * ______
    norm_a = math.sqrt(sum(v * v for v in a.values()))
    norm_b = math.sqrt(sum(v * v for v in b.values()))
    return dot / (norm_a * norm_b)

a = bag_of_words("The cat sat.")
b = bag_of_words("the CAT ran!")
print(round(cosine(a, b), 3))
```

الف) جاهای خالی را کامل کنید و خروجی را حساب کنید. (عبارت `sum(v * v for v in ...)` شکل کوتاه یک حلقه‌ی جمع‌کننده است که در جلسه‌ی ۱۳ می‌خوانید.)

ب) چرا نوشتن `b[w]` به جای جای خالی دوم خطا می‌دهد؟

ج) شباهت `"the cat the cat"` با `"cat the"` چقدر است؟ چه نتیجه‌ای درباره‌ی این روش (کیسه‌ی کلمات) می‌گیرید؟

▼ پاسخ

الف)

```
counts[w] = counts.get(w, 0) + 1
dot += a[w] * b.get(w, 0)
```

ضرب `a = {'the': 1, 'cat': 1, 'sat': 1}` و `b = {'the': 1, 'cat': 1, 'ran': 1}`؛ داخلی `2` و طول هر دو `√3`؛ شباهت `2/3 ≈ 0.667`. دقت کنید پیش‌پردازش باعث شد `"The"` و `"the"`، و `"CAT"` و `"cat"` یکی حساب شوند.

ب) کلمه‌ی `"sat"` در `b` وجود ندارد و `b["sat"]` خطای `KeyError` می‌دهد؛ کلمه‌ای که در یک سند نیست، شمارش صفر دارد.

ج) بردارها `{'the': 2, 'cat': 2}` و `{'cat': 1, 'the': 1}` هم‌جهت‌اند و شباهت `1.0` است. کیسه‌ی کلمات **ترتیب** کلمات را نادیده می‌گیرد و شباهت کسینوسی **تعداد تکرار** را نرمال می‌کند؛ پس «سگ گربه را گاز گرفت» و «گربه سگ را گاز گرفت» از نظر این روش یکسان‌اند. (n-gramها بخشی از اطلاعات ترتیب را برمی‌گردانند.)

خلاصه‌ی جلسه

- رشته دنباله‌ی تغییرناپذیری از کاراکترهای یونیکد است؛ اندیس، برش و حلقه مثل تاپل.
- متدهای رشته همیشه مقدار جدید برمی‌گردانند؛ نتیجه را انتساب کنید و برای خوانایی آن‌ها را زنجیر کنید.
- `find` / `index` / `count` / `replace` برای جست‌وجو و جایگزینی؛ برای بررسی وجود از `in` استفاده کنید.
- `split` و `join` پل بین رشته و لیست‌اند؛ برای ساختن رشته‌های بزرگ، لیست بسازید و یک بار `join` کنید.

- پیش‌پردازش متن: حروف کوچک، حذف علامت‌ها، توکن‌سازی و حذف stop words؛ پایه‌ی هر کار NLP.

جلسه‌ی بعد: پروژه‌ی ۱، مدیریت یک کتابخانه‌ی کوچک با همه‌ی ابزارهای بخش اول.

جلسه ۱۲: پروژه‌ی ۱ — تحلیل کارنامه‌ی دانشجویان یک دانشکده

Project 1: Analyzing Student Transcripts

بخش ۱: مبانی زبان پایتون — دوره‌ی پایتون برای علوم داده

مدرس: دکتر حسین حق‌بین

فهرست

1. صورت مسئله
2. داده
3. الزامات برنامه
4. محدودیت‌ها
5. آنچه تحویل می‌دهید
6. دفاع
7. معیارهای ارزیابی

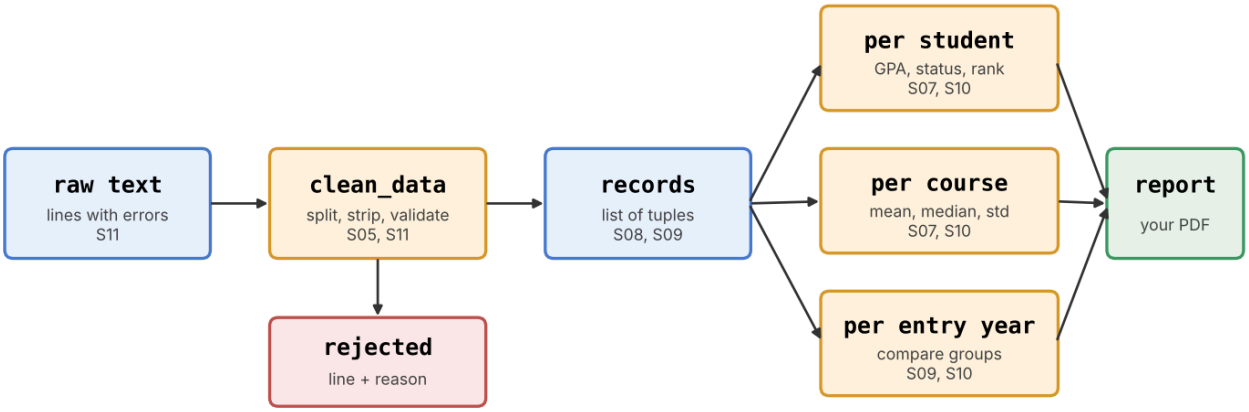
۱. صورت مسئله

معاون آموزشی یک دانشکده، نمره‌های پایان ترم سه ورودی (۱۴۰۰، ۱۴۰۱ و ۱۴۰۲) را از سامانه‌ی آموزشی بیرون کشیده است. بخشی از نمره‌ها را کارمندان دستی وارد کرده‌اند و داده **بی‌نقص نیست**: سطرهای تکراری، نمره‌های نامعتبر، سطرهای ناقص و نوشتار ناهمسان در آن دیده می‌شود.

معاون از شما خواسته است:

- داده را **پاک‌سازی** کنید و دقیقاً بگویید با هر ایراد چه کرده‌اید،
- وضعیت تحصیلی هر دانشجو را مشخص کنید (معدل، مشروطی، ممتاز بودن، رتبه)،
- عملکرد هر **درس** و هر **ورودی** را تحلیل کنید و هر چیز غیرعادی را که می‌بینید گزارش دهید،
- و به یک **پرسش سیاستی** پاسخ دهید (الزام ۷).

همه‌ی نتایج را در یک **گزارش** ارائه می‌کنید که معاون، بدون دانستن برنامه‌نویسی، بتواند بر اساس آن تصمیم بگیرد.



مراحل اصلی برنامه و جلسه‌هایی که ابزار هر مرحله را در آن‌ها یاد گرفته‌اید.

مقررات آموزشی (برای این پروژه)

مفهوم	تعریف
نمره	عددی بین `0` و `20`
قبولی در درس	نمره‌ی `10` یا بیشتر
غیبت در امتحان	با `ABS` ثبت شده و نمره‌ی آن `0` حساب می‌شود
معدل	میانگین **وزنی** نمره‌ها با وزن تعداد واحد هر درس؛ درس‌های مردودی هم در معدل حساب می‌شوند
مشروط	معدل کمتر از `12`
ممتاز	معدل `17` یا بیشتر
گزارش معدل	معدل با **دو رقم اعشار** گزارش و رتبه‌بندی می‌شود

۲. داده

داده یک رشته‌ی چندخطی به نام `RAW_DATA` است (سلول پایین). سطر اول عنوان ستون‌هاست و هر سطر بعدی یک نمره است، با پنج بخش جداشده با ویرگول:

```
student_id,name,course,units,grade
4018507,Mehdi Tangestani,Programming,3,16.75
```

- `student_id` : شماره‌ی دانشجویی هفت‌رقمی؛ **سه رقم اول** سال ورود است (`400` ، `401` یا `402`).
- `name` : نام دانشجو.
- `course` : نام درس؛ فقط درس‌های جدول زیر مجازند.
- `units` : تعداد واحد درس؛ باید با جدول زیر یکی باشد.
- `grade` : نمره، یا `ABS` برای غیبت.

درس	واحد
`Calculus 1`	۳
`Physics 1`	۳
`Programming`	۳
`Statistics 1`	۳
`Linear Algebra`	۳
`Discrete Math`	۳
`General English`	۲
`Physical Education`	۱

داده ساختگی است، اما ایرادهای آن از نوع ایرادهای داده‌های واقعی است. سلول زیر را در ابتدای نوت‌بوک خود کپی کنید و **آن را دستی ویرایش نکنید**: پیدا کردن و رفع ایرادها باید با **کد** انجام شود.

```
RAW_DATA = """student_id,name,course,units,grade
4009414,Parisa Abbasi,Calculus 1,4,15.5
4014257,Mohammad Dashti,Linear Algebra,3,ABS
4018507,Mehdi Tangestani,Programming,3,16.75
 4006062 , OMID BUSHEHRI , physics 1 , 3 , 16.25
4018850,Amir Najafi,Statistics 1,3,17.25
4016853,Babak Tangestani,Calculus 1,3,15.5
4004678,Yasaman Ahmadi,Linear Algebra,3,ABS
4007891,Hossein Ghasemi,Physics 1,3,13.5
4018420,Arash Rahimi,Calculus 1,3,8.5
4009414,Parisa Abbasi,General English,2,18.25
4029925,Amir Mohammadi,Discrete Math,3,16.5
4024088,Reza Salehi,Linear Algebra,3,ABS
4009975,Yasaman Bushehri,Programming,3,20
4027379,Ali Rezaei,Physical Education,1,14.75
4009045,Yasaman Dashti,Discrete Math,3,12.25
4028600,Amir Rahimi,Physics 1,3,17
4018420,Arash Rahimi,Programming,3,12.5
4029925,Amir Mohammadi,Discrete Math,3,16.5
4012583,Leila Bagheri,Physics 1,3,12.5
4007917,Parisa Salehi,Calculus 1,3,15.75
4027969,Hamid Rahimi,General English,2,17.5
4029513,Mehdi Bushehri,Physics 1,3,13
4023797,Leila Salehi,Statistics 1,3,12.5
4015261,Parisa Jafari,Discrete Math,3,7.5
4019955,Ali Salehi,Programming,3,15
4025713,Leila Najafi,Discrete Math,3,11.5
4025975,Leila Sadeghi,Calculus 1,3,10
4019060,Shirin Bushehri,Programming,3,16
4019145,Neda Shirazi,General English,2,17.75
4023623,Mohammad Najafi,General English,2,19.25
4008035,Elham Dashti,Programming,3,12.75
4009233,Hossein Sadeghi,Calculus 1,3,16
4009414,Parisa Abbasi,Linear Algebra,3,13.25
4006062,Omid Bushehri,General English,2,18
4012583,Leila Bagheri,Statistics 1,3,14.75
4022288,Kaveh Najafi,Statistics 1,3,7.5
4028600,Amir Rahimi,Physical Education,1,20
4003308,Fatemeh Salehi,Statistics 1,3,12.25
4024822,Neda Dashti,Linear Algebra,3,16.75
4018420,Arash Rahimi,General English,2,10.75
4029596,Hossein Shirazi,Calculus 1,3,14
4004444,babak sadeghi,  Statistics 1  ,3,8.25
4029513,Mehdi Bushehri,Physical Education,1,17.75
4019060,Shirin Bushehri,Physics 1,3,15.25
4009975,Yasaman Bushehri,General English,2,18.75
4026685,Mohammad Mohammadi,Linear Algebra,3,8
4024088,Reza Salehi,Statistics 1,3,3.75
4029596,Hossein Shirazi,Statistics 1,3,7.75
4018183,Neda Rezaei,Physical Education,1,17.75
4015261,Parisa Jafari,Physics 1,3,6.5
4028600,Amir Rahimi,Discrete Math,3,15
4015762,Mina Shirazi,General English,2,18.75
4008226,Leila Bushehri,Physical Education,1,14.25
4006062,Omid Bushehri,Discrete Math,3,19.5
```

```
4024822,Neda Dashti,Physics 1,3,15.25
4002680,Elham Shirazi,Discrete Math,3,11.25
4009233,Hossein Sadeghi,Discrete Math,3,145
4005237,Hamid Shirazi,Linear Algebra,3,17
4002322,Maryam Ahmadi,Linear Algebra,3,18
4015261,Parisa Jafari,Calculus 1,3,7.25

4008035,Elham Dashti,Physical Education,1,15.5
4016188,Reza Kazemi,General English,2,14
4011542,Zahra Ahmadi,Physics 1,3,12.5
4009045,Yasaman Dashti,Physics 1,3,11.25
4019060,Shirin Bushehri,Linear Algebra,3,14
4018183,Neda Rezaei,Calculus 1,3,12
4016188,Reza Kazemi,Physics 1,3,11.5
4029925,Amir Mohammadi,Physical Education,1,16.5
4009486,Leila Shirazi,Discrete Math,3,10
4023623,Mohammad Najafi,Statistics 1,3,15.25
4029967,Babak Hosseini,Calculus 1,3,12.25
4018507,Mehdi Tangestani,General English,2,17
4023623,Mohammad Najafi,Physics 1,3,19.25
4018183,Neda Rezaei,Discrete Math,3,12
4029513,Mehdi Bushehri,Statistics 1,3,11.75
4012583,Leila Bagheri,Calculus 1,3,14.5
4007891,Hossein Ghasemi,Linear Algebra,3,10.75
4018507,Mehdi Tangestani,Calculus 1,3,14.75
4025713,Leila Najafi,STATISTICS 1,3,8.5
4011542,Zahra Ahmadi,Programming,3,15.25
4002322,Maryam Ahmadi,Calculus 1,3,19
4015762,Mina Shirazi,Physical Education,1,19.75
4019060,Shirin Bushehri,Discrete Math,3,14.25
4019939,Babak Heydari,Programming,3,14.75
4029967,Babak Hosseini,Discrete Math,3,12.25
4009486,Leila Shirazi,Programming,3,11.75
4009975,Yasaman Bushehri,Discrete Math,3,18.25
4008484,Maryam Rezaei,Programming,3,15.75
4015399,Nima Tavakoli,Linear Algebra,3,17.25
4014257,Mohammad Dashti,General English,2,14.75
4022288,Kaveh Najafi,Physical Education,1,15.75
4007891,Hossein Ghasemi,Programming,3,15.5
4028600,Amir Rahimi,General English,2,16
4008226,Leila Bushehri,Discrete Math,3,11
4025713,Leila Najafi,Physical Education,1,13.75
4013397,Sara Najafi,Programming,3,16
4009166,Mohammad Abbasi,Discrete Math,3,17
4027379,Ali Rezaei,Calculus 1,3,13.75
4024088,Reza Salehi,Calculus 1,3,8.25
4019223,Fatemeh Tavakoli,Programming,3,16.75
4023797,Leila Salehi,Programming,3,17.5
4029318,Shirin Kazemi,Calculus 1,3,11.75
4027969,Hamid Rahimi,Statistics 1,3,16.75
4006577,Omid Hosseini,Calculus 1,3,18.25
4025975,Leila Sadeghi,Discrete Math,3,7.25
4016188,Reza Kazemi,Physical Education,1,14
4009032,Neda Ahmadi,Programming,3,17.5
4011542,Zahra Ahmadi,Statistics 1,3,10
4002680,Elham Shirazi,Statistics 1,3,9.5
```

```
4007917,Parisa Salehi,Physical Education,1,19.25
4007917,Parisa Salehi,Statistics 1,3,10.75
4024822,Neda Dashti,Discrete Math,3,16.25
4023256,Sara Shirazi,Linear Algebra,3,16
4029513,Mehdi Bushehri,Programming,3,14.5
4009414,Parisa Abbasi,Physics 1,3,15.75
4008035,Elham Dashti,Discrete Math,3,12.75
4029925,Amir Mohammadi,Physics 1,3,15.5
4029513,Mehdi Bushehri,Discrete Math,3,15.5
4007891,Hossein Ghasemi,Statistics 1,3,11
4018420,Arash Rahimi,Physics 1,3,10.75
4026685,Mohammad Mohammadi,Programming,3,7.5
 4016188 , REZA KAZEMI , programming , 3 , 9.75
4023623,Mohammad Najafi,Linear Algebra,3,16
4009032,Neda Ahmadi,Calculus 1,3,19.75
4022288,Kaveh Najafi,Calculus 1,3,10.75
4013397,Sara Najafi,Linear Algebra,3,14
4029925,Amir Mohammadi,Calculus 1,3,15.5
4017585,Elham Heydari,Discrete Math,3,11.5
4027379,Ali Rezaei,Linear Algebra,3,14
4005237,Hamid Shirazi,General English,2,17.25
4018183,Neda Rezaei,Linear Algebra,3,12.25
4015261,Parisa Jafari,Physical Education,1,10
4004444,Babak Sadeghi,Physical Education,1,12.75
4025975,Leila Sadeghi,Programming,3,7.25
4009166,Mohammad Abbasi,Calculus 1,3,16.25
4027969,Hamid Rahimi,Physical Education,1,19.25
4025713,Leila Najafi,General English,2,12.25
4009486,Leila Shirazi,General English,2,11.5
4018507,Mehdi Tangestani,Linear Algebra,3,16.5
4006577,Omid Hosseini,General English,2,19.5
4019223,Fatemeh Tavakoli,Physics 1,3,15
4007891,Hossein Ghasemi,Discrete Math,3,14.5
4023433,Saeed Abbasi,Linear Algebra,3,10.75
4027379,Ali Rezaei,Programming,3,14
4003308,Fatemeh Salehi,General English,2,17.75
4019145,Neda Shirazi,Calculus 1,3,13
4009414,Parisa Abbasi,Statistics 1,3,12.25
4009032,Neda Ahmadi,Discrete Math,3,17
4009045,Yasaman Dashti,Physical Education,1,15.5
4023623,Mohammad Najafi,Physical Education,1,20
4004678,Yasaman Ahmadi,Discrete Math,3,15.75
4017585,Elham Heydari,Statistics 1,3,11.25
4023256,Sara Shirazi,Programming,3,17.5
4004678,Yasaman Ahmadi,Physics 1,3,13.5
4027969,Hamid Rahimi,Discrete Math,3,17.25
4024088,Reza Salehi,Discrete Math,3,8
4019145,Neda Shirazi,Linear Algebra,3,12.25
4004678,Yasaman Ahmadi,Discrete Math,3,15.75
4004678,Yasaman Ahmadi,General English,2,18
4029682,Ali Karimi,General English,2,13.75
4005237,Hamid Shirazi,Physical Education,1,20
4022288,Kaveh Najafi,General English,2,14.25
4029596,Hossein Shirazi,Discrete Math,3,13
4009233,Hossein Sadeghi,General English,2,16.25
4026685,Mohammad Mohammadi,Discrete Math,3,9
```

```
4009233,Hossein Sadeghi,Statistics 1,3,9
4018183,Neda Rezaei,General English,2,14.75
4025713,Leila Najafi,Linear Algebra,3,11.5
4004678,Yasaman Ahmadi,Calculus 1,3,15.25
4003308,Fatemeh Salehi,Calculus 1,3,16.5
4023623,Mohammad Najafi,Discrete Math,3,18
4023433,Saeed Abbasi,Programming,3,12.25
4029362,Ali Ahmadi,Physical Education,1,8.25
4008484,Maryam Rezaei,Physical Education,1,17.5
4012583,Leila Bagheri,General English,2,14.5
4019060,Shirin Bushehri,Statistics 1,3,7.75
4006577,Omid Hosseini,Discrete Math,3,19.5
4023797,Leila Salehi,Physics 1,3,18.5
4009032,Neda Ahmadi,Physical Education,1,19.25
4018420,Arash Rahimi,Physical Education,1,11.5
4015399,Nima Tavakoli,Statistics 1,3,14.5
4018183,Neda Rezaei,Programming,3,12.25
4029362,Ali Ahmadi,Calculus 1,3,-3
4017585,Elham Heydari,Physics 1,3,10.25
4029682,Ali Karimi,Programming,3,15
4002322,Maryam Ahmadi,Programming,3,14.5
4018850,Amir Najafi,Linear Algebra,3,16
4019942,Elham Karimi,Calculus 1,3,15
4009975,Yasaman Bushehri,Calculus 1,3,16
4025975,Leila Sadeghi,Physical Education,1,11.5
4003308,Fatemeh Salehi,Linear Algebra,3,14.5
4002680,Elham Shirazi,Calculus 1,3,8.75
4019145,Neda Shirazi,Physical Education,1,18.25
4008226,Leila Bushehri,Statistics 1,3,9.5
4027969,Hamid Rahimi,Calculus 1,3,16
4023797,Leila Salehi,Discrete Math,3,17
4019223,Fatemeh Tavakoli,Linear Algebra,3,15.5
4023433,Saeed Abbasi,General English,2,12.75
4019939,Babak Heydari,Physics 1,3,12.5
4025713,Leila Najafi,Physics 1,3,9.5
4003308,Fatemeh Salehi,Discrete Math,3,15
4017585,Elham Heydari,Physical Education,1,14.75
4018183,Neda Rezaei,STATISTICS 1,3,8
4029682,Ali Karimi,Physics 1,3,12.75
4009166,Mohammad Abbasi,Linear Algebra,3,14.75
4006062,Omid Bushehri,Programming,3,19.25
4029596,Hossein Shirazi,Programming,3,17
4007891,Hossein Ghasemi,Physical Education,1,18.25
4009975,Yasaman Bushehri,Physical Education,1,20
4029513,Mehdi Bushehri,Linear Algebra,3,12.75
4008226,Leila Bushehri,Programming,3,13
4029682,Ali Karimi,Statistics 1,3,8.5
4024088,Reza Salehi,General English,2,14
4029967,Babak Hosseini,Physical Education,1,14.25
4029318,Shirin Kazemi,Statistics 1,3,6.25
4023797,Leila Salehi,Physical Education,1,19.25
4029596,Hossein Shirazi,Physics 1,3,14
4004678,Yasaman Ahmadi,Statistics 1,3,9.75
4014257,Mohammad Dashti,Physics 1,3,15.75
4019223,Fatemeh Tavakoli,Physical Education,1,20
4002322,Maryam Ahmadi,General English,2,17.5
```

```
4019942,Elham Karimi,Linear Algebra,3,15.25
4015261,Parisa Jafari,Linear Algebra,3,10
4013397,Sara Najafi,Physics 1,3,13.25
4006062,Omid Bushehri,Statistics 1,3,17,25
4003308,Fatemeh Salehi,Programming,3,17.5
4009486,Leila Shirazi,Physics 1,3,11.25
4019942,Elham Karimi,Statistics 1,3,10.25
4028600,Amir Rahimi,Programming,3,12.5
4011542,Zahra Ahmadi,Calculus 1,3,18
4023256,Sara Shirazi,Statistics 1,3,13.5
4014257,Mohammad Dashti,Programming,3,13.25
4007917,Parisa Salehi,Programming,3,15.5
4026685,Mohammad Mohammadi,General English,2,11.75
4019060,Shirin Bushehri,General English,2,17.75
4019939,Babak Heydari,Physical Education,1,18.5
4009166,Mohammad Abbasi,Programming,3,18.5
4009233,Hossein Sadeghi,Physics 1,3,16.5
4023256,Sara Shirazi,Physical Education,1,19.25
4014257,Mohammad Dashti,Physical Education,1,18.25
4019955,Ali Salehi,General English,2,13.5
4019955,Ali Salehi,Statistics 1,3,9.75
4009166,Mohammad Abbasi,Physical Education,1,19.75
4008226,Leila Bushehri,Calculus 1,3,8
4027969,Hamid Rahimi,Physics 1,3,17
4008035,Elham Dashti,Physics 1,3,10.25
4019939,Babak Heydari,Discrete Math,3,15.75
4023256,Sara Shirazi,Calculus 1,3,14.75
4019223,Fatemeh Tavakoli,Statistics 1,3,11.75

4008484,Maryam Rezaei,Linear Algebra,3,16.5
4016188,Reza Kazemi,Discrete Math,3,12.5
4019223,Fatemeh Tavakoli,Calculus 1,3,17.5
4019955,Ali Salehi,Physical Education,1,14.75
4029362,Ali Ahmadi,Discrete Math,3,8
4019145,Neda Shirazi,Physics 1,3,15.25
4005237,Hamid Shirazi,Programming,3,15.75
4018850,Amir Najafi,Discrete Math,3,17.5
4016853,Babak Tangestani,Programming,3,19.5
4023623,Mohammad Najafi,Calculus 1,3,18
4023433,Saeed Abbasi,Discrete Math,3,11.5
4009166,Mohammad Abbasi,Physics 1,3,16.75
4018850,Amir Najafi,Calculus 1,3,15
4009486,Leila Shirazi,CALCULUS 1,3,13.25
4002322,Maryam Ahmadi,Discrete Math,3,16.5
4029318,Shirin Kazemi,Programming,3,15
4029967,Babak Hosseini,General English,2,12.75
4022288,Kaveh Najafi,Physics 1,3,12.5
4023433,Saeed Abbasi,Physical Education,1,15
4024088,Reza Salehi,Physics 1,3,8
4018420,Arash Rahimi,Discrete Math,3,13
4019223,Fatemeh Tavakoli,General English,2,16
4014257,Mohammad Dashti,Discrete Math,3,13.75
4009032,Neda Ahmadi,3,18.5
4009166,Mohammad Abbasi,Statistics 1,3,15.5
4027379,Ali Rezaei,Physics 1,3,14
4013397,Sara Najafi,General English,2,14
```

```
4002680,Elham Shirazi,Linear Algebra,3,7
4019942,Elham Karimi,Physical Education,1,17
4015762,Mina Shirazi,Statistics 1,3,13.25
4026685,Mohammad Mohammadi,Statistics 1,3,6.5
4005237,Hamid Shirazi,Calculus 1,3,17.5
4029682,Ali Karimi,Physical Education,1,17.75
4025975,Leila Sadeghi,Linear Algebra,3,9
4026685,Mohammad Mohammadi,Physics 1,3,12
4023433,Saeed Abbasi,Physics 1,3,12
4009045,Yasaman Dashti,Linear Algebra,3,11.75
4018850,Amir Najafi,Physical Education,1,20
4007891,Hossein Ghasemi,General English,2,13.75
4008035,Elham Dashti,Linear Algebra,3,9.75
4016853,Babak Tangestani,Statistics 1,3,11.5
4029318,Shirin Kazemi,General English,2,12.75
4004444,Babak Sadeghi,Calculus 1,3,13
4029362,Ali Ahmadi,General English,2,10
4018850,Amir Najafi,Programming,3,17.5
4013397,Sara Najafi,Physical Education,1,16.75
4002680,Elham Shirazi,Physics 1,3,9.75
4011542,Zahra Ahmadi,Linear Algebra,3,13
4029967,Babak Hosseini,Linear Algebra,3,16
4016853,Babak Tangestani,Physics 1,3,14.75
4029682,Ali Karimi,Linear Algebra,3,11.75
4016853,Babak Tangestani,General English,2,17.5
4025975,Leila Sadeghi,General English,2,9.75
4009414,Parisa Abbasi,Discrete Math,3,14.75
4015261,Parisa Jafari,Programming,3,9.25
4004678,Yasaman Ahmadi,Physical Education,1,19.25
4029362,Ali Ahmadi,Linear Algebra,3,6.5
4008484,Maryam Rezaei,General English,2,14.75
4016853,Babak Tangestani,General English,2,17.5
4018850,Amir Najafi,Physics 1,3,18
4004678,Yasaman Ahmadi,Programming,3,16.25
4024822,Neda Dashti,Physical Education,1,20
4006577,Omid Hosseini,Physics 1,3,18.25
4029925,Amir Mohammadi,Linear Algebra,3,12.75
4009045,Yasaman Dashti,Programming,3,14
4007917,Parisa Salehi,Discrete Math,3,9.75
4029967,Babak Hosseini,Statistics 1,3,11.75
4018850,Amir Najafi,General English,2,18.25
4009233,Hossein Sadeghi,Physical Education,1,20
4019060,Shirin Bushehri,Physical Education,1,17.5
4008226,Leila Bushehri,Linear Algebra,3,8.75
4024822,Neda Dashti,Statistics 1,3,13
4023797,Leila Salehi,Linear Algebra,3,18.25
4006577,Omid Hosseini,Programming,3,14.5
4002680,Elham Shirazi,General English,2,12.25
4008484,Maryam Rezaei,Physics 1,3,11.75
4009032,Neda Ahmadi,Linear Algebra,3,18.25
4019145,Neda Shirazi,Statistics 1,3,11.75
4026685,Mohammad Mohammadi,Physical Education,1,13.25
4013397,Sara Najafi,Statistics 1,3,13
4024822,Neda Dashti,Calculus 1,3,14.25
4025975,Leila Sadeghi,Physics 1,3,7
4023797,Leila Salehi,Calculus 1,3,19.75
```

```
4029925,Amir Mohammadi,Programming,3,14.25
4002322,Maryam Ahmadi,Physics 1,3,18.25
4017585,Elham Heydari,Programming,3,12.25
4008035,Elham Dashti,Calculus 1,3,15.5
 4019939 , BABAK HEYDARI , general english , 2 , 15.25
4018507,Mehdi Tangestani,Physical Education,1,17.75
4029596,Hossein Shirazi,Linear Algebra,3,15
4006577,Omid Hosseini,Physical Education,1,20
4004444,Babak Sadeghi,General English,2,11.5
4019942,Elham Karimi,Physics 1,3,15
4019955,Ali Salehi,Physics 1,3,14
4022288,Kaveh Najafi,Programming,3,14
4018420,Arash Rahimi,Linear Algebra,3,10
4013397,Sara Najafi,Calculus 1,3,13.25
4006577,Omid Hosseini,Statistics 1,3,14
4023433,Saeed Abbasi,Calculus 1,3,12.25
4003308,Fatemeh Salehi,Physical Education,1,18.25
4029513,Mehdi Bushehri,General English,2,14.5
4029682,Ali Karimi,Discrete Math,3,14.25
4007917,Parisa Salehi,General English,2,17
4005237,Hamid Shirazi,Physics 1,3,17
4016853,Babak Tangestani,Linear Algebra,3,15
4007917,Parisa Salehi,Linear Algebra,3,13.5
4009045,Yasaman Dashti,General English,2,12.25
4009486,Leila Shirazi,Linear Algebra,3,12.75
4013397,Sara Najafi,Discrete Math,3,16
4029318,Shirin Kazemi,Physical Education,1,11.5
4024088,Reza Salehi,Physical Education,1,16
4011542,Zahra Ahmadi,General English,2,17.5
4011542,Zahra Ahmadi,Discrete Math,3,13.75
4017585,Elham Heydari,Calculus 1,3,11.75
4014257,Mohammad Dashti,Calculus 1,3,13.25
4016853,Babak Tangestani,Physical Education,1,17.75
4002680,Elham Shirazi,Physical Education,1,13.5
4018507,Mehdi Tangestani,Statistics 1,3,10.5
4004444,Babak Sadeghi,Physics 1,3,12.5
4016853,Babak Tangestani,Discrete Math,3,13.25
4008035,Elham Dashti,Statistics 1,3,9
4029513,Mehdi Bushehri,Calculus 1,3,16.5
4012583,Leila Bagheri,Discrete Math,3,13.5
4008484,Maryam Rezaei,Statistics 1,3,9.5
4015399,Nima Tavakoli,Physics 1,3,17.25
4029362,Ali Ahmadi,Programming,3,9
4023623,Mohammad Najafi,Programming,3,18.25
4029362,Ali Ahmadi,Physics 1,3,9
4009032,Neda Ahmadi,Statistics 1,3,12.5
4009486,Leila Shirazi,Physical Education,1,16.5
4009045,Yasaman Dashti,Calculus 1,3,10.25
4014257,Mohammad Dashti,Statistics 1,3,8.5
4006062,Omid Bushehri,Physical Education,1,19.5
4004444,Babak Sadeghi,Linear Algebra,3,12.5
4028600,Amir Rahimi,Calculus 1,3,12
4015399,Nima Tavakoli,Physical Education,1,20
4028600,Amir Rahimi,Statistics 1,3,8.5
4006062,Omid Bushehri,Linear Algebra,3,17
4015762,Mina Shirazi,Discrete Math,3,17.75
```

```
4027379,Ali Rezaei,General English,2,16.25
4009975,Yasaman Bushehri,Physics 1,3,17.5
4002322,Maryam Ahmadi,Statistics 1,3,11.25
4015399,Nima Tavakoli,Programming,3,19.75
4023256,Sara Shirazi,Discrete Math,3,14.25
4019942,Elham Karimi,General English,2,17.25
4019145,Neda Shirazi,Programming,3,16
4025713,Leila Najafi,Calculus 1,3,11.5
4019223,Fatemeh Tavakoli,Discrete Math,3,16.5
4029318,Shirin Kazemi,Linear Algebra,3,11.75
4029596,Hossein Shirazi,Physical Education,1,15
4015762,Mina Shirazi,Physics 1,3,17.5
4009166,Mohammad Abbasi,Discrete Math,3,20
4006062,Omid Bushehri,Calculus 1,3,19
4012583,Leila Bagheri,Linear Algebra,3,13
4029967,Babak Hosseini,Programming,3,14
4005237,Hamid Shirazi,Discrete Math,3,14.5
4019939,Babak Heydari,Calculus 1,3,13.25
4016188,Reza Kazemi,Statistics 1,3,5.5
4015762,Mina Shirazi,Calculus 1,3,18
4009233,Hossein Sadeghi,Linear Algebra,3,16.5
4004444,Babak Sadeghi,Programming,3,15
4027379,Ali Rezaei,Discrete Math,3,12
4023256,Sara Shirazi,General English,2,17
4019955,Ali Salehi,Calculus 1,3,14.25
4019939,Babak Heydari,Statistics 1,3,10.75
4019955,Ali Salehi,Discrete Math,3,13
4029925,Amir Mohammadi,Statistics 1,3,12
4024822,Neda Dashti,Programming,3,19.75
4008035,Elham Dashti,General English,2,14.25
4023256,Sara Shirazi,General English,2,17
4018507,Mehdi Tangestani,Discrete Math,3,16.5
4016188,Reza Kazemi,Linear Algebra,3,10.25
4009414,Parisa Abbasi,Physical Education,1,20
4015762,Mina Shirazi,Linear Algebra,3,17.25
4015399,Nima Tavakoli,Calculus 1,3,17

"""
```

۳. الزامات برنامه

چگونگی تقسیم کد به توابع با خودتان است، اما **دو تابع زیر باید دقیقاً با همین امضا وجود داشته باشند**، چون در جلسه‌ی دفاع با داده‌ی جدید آزمایش می‌شوند:

```
records, rejected = clean_data(raw_text)
# records:  list of (student_id, name, course, units, grade) tuples, cleaned
# rejected: list of (line_number, line, reason) tuples

value = gpa(records, student_id)   # weighted GPA of one student (float)
```

1. **پاک‌سازی:** `clean_data` باید ایرادهای داده را پیدا کند. سطری که با اطمینان قابل اصلاح است (مثلاً فاصله‌ی اضافه یا حروف بزرگ و کوچک ناهمسان) اصلاح شود؛ سطری که قابل اصلاح نیست، با شماره‌ی سطر و **دلیل** در `rejected` ثبت شود. هیچ سطری نباید بی‌صدا حذف شود. نام‌ها را به یک شکل یکسان (مثلاً `Parisa Abbasi`) و نام درس‌ها را دقیقاً به شکل جدول بالا برگردانید. در گزارش، برای هر نوع ایراد بگویید چند بار دیدید و چه تصمیمی گرفتید و چرا.
2. **وضعیت هر دانشجو:** تعداد واحد اخذشده، تعداد واحد قبول‌شده، معدل و وضعیت (مشروط، عادی، ممتاز).
3. **رتبه‌بندی:** رتبه‌ی همه‌ی دانشجویان بر اساس معدل. دانشجویان با معدل برابر رتبه‌ی **برابر** می‌گیرند و رتبه‌ی بعدی به تعداد آن‌ها جلو می‌رود (مثلاً `1, 2, 2, 4`). ده نفر اول را گزارش کنید.
4. **آمار هر درس:** تعداد نمره، میانگین، میانه، انحراف معیار، کمینه، بیشینه و درصد مردودی. توزیع نمره‌های هر درس را با یک **نمودار میله‌ای متنی** (با کاراکتر `#`) در بازه‌های `[0,10)` ، `[10,12)` ، `[12,14)` ، `[14,16)` ، `[16,18)` و `[18,20]` نمایش دهید.
5. **مقایسه‌ی ورودی‌ها:** برای هر ورودی، تعداد دانشجو، میانگین معدل، تعداد و درصد مشروطی.
6. **موارد غیرعادی:** هر درس، ورودی یا الگوی غیرعادی را که در داده می‌بینید (پس از پاک‌سازی) گزارش و با عدد مستند کنید. توضیح دهید چه چیزی آن را غیرعادی می‌کند و معاون باید درباره‌اش چه کند.
7. **پرسش سیاستی:** معاون پیشنهاد کرده است که به نمره‌های درسی که **بیشترین درصد مردودی** را دارد، `2` نمره اضافه شود (سقف `20`). حساب کنید این کار چند دانشجو را از مشروطی خارج می‌کند، درصد مردودی آن درس چقدر می‌شود و رتبه‌های بالای جدول چه تغییری می‌کنند. نظر خود را درباره‌ی این پیشنهاد، با دلیل، بنویسید.
8. **فهرست‌های نهایی:** فهرست دانشجویان مشروط (از کمترین معدل) و فهرست دانشجویان ممتاز (از بیشترین معدل)، با شماره‌ی دانشجویی، نام و معدل.

۴. محدودیت‌ها

- فقط پایتون خالص و ماژول‌های `math` و `string` . استفاده از `statistics` ، `pandas` ، `numpy` ، `collections` یا هر کتابخانه‌ی دیگر **مجاز نیست**؛ میانگین، میانه و انحراف معیار را خودتان بنویسید.
- داده‌ی `RAW_DATA` را دستی تغییر ندهید.
- همه‌ی اعداد گزارش باید از اجرای همان نوت‌بوکی بیایند که تحویل می‌دهید و با اجرای دوباره‌ی آن (از بالا به پایین) عیناً تکرار شوند.
- پروژه **انفرادی** است.

۵. آنچه تحویل می‌دهید

مهلت تحویل و زمان دفاع در کلاس اعلام می‌شود. دو فایل تحویل دهید:

الف) نوت‌بوک کد با نام `S12_Project1_StudentID.ipynb` (شماره‌ی دانشجویی خود را به جای `StudentID` بگذارید)؛ اجراشده، با خروجی همه‌ی سلول‌ها، و با توضیح کوتاه برای هر بخش.

ب) گزارش با نام `S12_Project1_StudentID.pdf` ، به فارسی، حداکثر ۶ صفحه، با این بخش‌ها:

1. **خلاصه‌ی مدیریتی** (حداکثر نیم صفحه): مهم‌ترین یافته‌ها و پیشنهادها به زبان ساده.
2. **داده و پاک‌سازی:** اندازه‌ی داده، انواع ایرادها، تعداد هر کدام و تصمیم شما درباره‌ی هر نوع.
3. **وضعیت دانشجویان:** آمار کلی، ده رتبه‌ی اول، تعداد مشروط و ممتاز.
4. **تحلیل درس‌ها و ورودی‌ها:** جدول آمار درس‌ها، مقایسه‌ی ورودی‌ها و موارد غیرعادی.
5. **پرسش سیاستی:** نتیجه‌ی محاسبه و نظر شما.
6. **محدودیت‌ها:** چه چیزهایی را این داده و این تحلیل **نمی‌تواند** نشان دهد.
7. **پیوست:** فهرست‌های نهایی (الزام ۸) و فهرست سطرهای کنارگذاشته.

گزارش باید **خودبسنده** باشد: خواننده بدون باز کردن نوت‌بوک بفهمد چه کرده‌اید و به چه نتیجه‌ای رسیده‌اید. کد را در گزارش نیاورید.

۶. دفاع

جلسه‌ی دفاع حدود ۱۰ دقیقه است:

- ارائه‌ی کوتاه یافته‌ها (حدود ۳ دقیقه، از روی گزارش)،
- اجرای `clean_data` و `gpa` شما روی چند سطر داده‌ی جدید،
- توضیح بخش‌هایی از کد که پرسیده می‌شود (**هر خط** کد باید برای شما قابل توضیح باشد)،
- اعمال یک تغییر کوچک در کد در همان جلسه.

نمره‌ی هر بخش فقط در صورتی داده می‌شود که بتوانید در دفاع از آن بربیایید.

۷. معیارهای ارزیابی

معیار	سهم
پاک‌سازی داده و مستندسازی تصمیم‌ها (الزام ۱)	۲۵٪
درستی محاسبات (الزامات ۲ تا ۵ و ۸)	۲۵٪
تحلیل: موارد غیرعادی و پرسش سیاستی (الزامات ۶ و ۷)	۲۰٪
کیفیت گزارش: شفافیت، ساختار، دقت در اعداد	۲۰٪
خوانایی کد: نام‌گذاری، تقسیم به توابع، توضیح‌ها	۱۰٪

معیار	سهم
امتیازی: تابعی که کارنامه‌ی یک دانشجو را به شکل یک جدول متنی مرتب چاپ کند (با f-string)، یا هر تحلیل اضافه‌ی مفیدی که با عدد مستند شده باشد	تا ۱۰٪ اضافه